

Security Handbook : Incident Response

06

Security Handbook Series

GRANT SUPPORT • 2025-2026



Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)



Contents

Frontispiece	4
About This Handbook	4
Copyright and License	5
Author and Creative Credits	5
Purpose and Scope	6
Target Audience	6
How to Use This Handbook	6
Relationship to SCSVS, SCSTG, and SCWE	7
Part I: Foundations	8
1. Introduction to Incident Management	8
2. Incident Response Lifecycle	10
3. Roles and Responsibilities	19
Part II: Communication Strategies	24
4. Communication During Incidents	24
Part III: Detection and Response	38
5. Incident Detection	38
6. Incident Response Procedures	43
7. Preparation Checklist	51
Part IV: Playbooks	55
8. Playbook Overview	55
9. Smart Contract and Protocol Hack	57
10. Wallet Drainer and Phishing Incidents	63
11. Malware and Compromised Endpoints	65
12. Insider and Nation-State-Style Attacks (e.g., DPRK Vectors)	67
13. Decentralized Incident Response	69
14. War Room and Crisis Coordination	74
15. Safe Harbor and Whitehat Recovery	76
16. Vulnerability Disclosure and Bug Bounty Linkage	77
Part V: Lessons Learned and Improvement	79
17. Lessons Learned	79
18. Continuous Improvement	87

Part VI: Appendices 92

19. Appendix A: Glossary..... 92

20. Appendix B: Severity and Triage Matrix..... 95

21. Appendix C: Suggested Tools and Platforms 98

22. Appendix D: Message and Communication Templates 100

23. Appendix E: References and Further Reading 101

Appendix C: References and Further Reading 103

Standards and Frameworks 103

Incidents and Case Studies..... 103

Tools and Documentation..... 104

Further Reading..... 105

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

Incident Response

About This Handbook

Incident Response is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Copyright © 2026 The OWASP Foundation and contributors.



This document is released under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). You may share and adapt the material with appropriate attribution and under the same license. For reuse or distribution, make the license terms clear and identify material changes.

OWASP publications are community-developed educational resources. References to organizations, products, or services do not constitute endorsement, and the material is provided without warranty.

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook prepares security teams, protocol operators, and incident responders to detect, contain, and recover from security incidents that hit smart contracts, protocols, infrastructure, and the people who run them. It follows the National Institute of Standards and Technology (NIST) Special Publication (SP) 800-61 Revision 3 model, aligned to the Cybersecurity Framework (CSF) 2.0 Community Profile: incident response as a continuous cycle across Govern, Identify, Protect, Detect, Respond, and Recover, not a document that only gets opened once an alert fires. Supplemental NIST guidance on log management (SP 800-92 Rev. 1), event recovery (SP 800-184), and threat information sharing (SP 800-150) fills in implementation detail the core publication leaves to the reader.

The handbook works through communication strategy (who talks to whom, when, and on which channel), detection and response mechanics (on-chain monitoring, triage, evidence collection, containment), and a library of incident-specific playbooks: smart contract and protocol hacks, wallet drainer and phishing campaigns, malware and compromised endpoints, insider threats and nation-state-style attacks including vectors attributed to the Democratic People's Republic of Korea (DPRK), and the coordination patterns unique to decentralized teams with no single incident commander. It closes with the lessons-learned loop that feeds fixes back into playbooks and detection rules, because an incident response plan that never changes after an incident is not really a plan.

Target Audience

Security specialists, operations and strategy leads, DevOps and site reliability engineering (SRE) staff, protocol teams, and anyone holding incident response or crisis coordination duties inside a Web3 organization.

How to Use This Handbook

Start with Part I to fix the incident response lifecycle, team roles, and how this handbook maps to the SCS verification and testing standards. Part II sets the internal and external communication rules that hold regardless of incident type. Part III is the operational core: detection sources, triage, evidence collection, and containment mechanics. Part IV is the playbook library, one chapter per incident type, from a live protocol hack to an insider threat; pull the matching playbook the moment an incident is declared. Part V closes the loop with post-incident review

and continuous improvement, and the appendices hold the glossary, severity matrix, tool list, and message templates a responder needs mid-incident. A team already mid-incident should skip straight to Part IV and the templates in Appendix D.

Relationship to SCSVS, SCSTG, and SCWE

This handbook complements the OWASP smart contract standards rather than duplicating them. The Smart Contract Security Verification Standard (SCSVS) defines the controls a system must satisfy before an incident; the detection, monitoring, and evidence-collection requirements here are the operational counterpart for the moment one of those controls fails anyway. The Smart Contract Security Testing Guide (SCSTG) describes how to test for weaknesses before deployment; the playbooks in Part IV describe what to do when a weakness the testing guide catalogs gets exploited in production instead. The Smart Contract Weakness Enumeration (SCWE) names the root-cause weakness classes; this handbook's job starts at the moment one of those classes turns into a live incident. The EVM Forensics and DeFi Recovery Handbook (04) picks up the post-incident chain analysis and fund-recovery work referenced throughout the playbooks here, and the Web3 Attack Vectors Mapping Handbook (10) supplies the WA01 to WA15 taxonomy used to classify an incident once it has been triaged.

Part I: Foundations

This part fixes the vocabulary a Web3 incident responder needs before an alert ever fires: the lifecycle model the rest of the handbook builds on, the specific goals that make a Web3 incident different from a conventional IT breach, and the team structure that turns a plan on paper into action at 3 a.m. It anchors that model in the National Institute of Standards and Technology (NIST) Special Publication (SP) 800-61 Revision 3 and the Cybersecurity Framework (CSF) 2.0, then translates each function into what it means for a protocol holding user funds on an irreversible ledger.

1. Introduction to Incident Management

1.1 Why Plan Before an Incident

An incident response plan (IRP) is insurance bought before the loss, not after. On-chain finality removes the safety nets a traditional breach responder relies on: there is no chargeback, no bank able to freeze a wire mid-flight, and no weekend grace period before an attacker converts stolen tokens into assets a court order cannot reach. Every minute spent deciding who is allowed to pause a contract, whose multisig signature is required, or what the first public statement should say is a minute the attacker does not have to wait for.

The Ronin Network bridge hack is the reference case for this chapter, because it shows exactly what an absent plan costs in time. An attacker forged five of nine required validator signatures, four from compromised Sky Mavis infrastructure and a fifth obtained through an Axie DAO gas-free transaction allowlist granted in November 2021 and never revoked, then drained roughly 173,600 ETH and 25.5 million USDC, about \$624 million at the time ([Halborn incident analysis](#)). Nobody noticed until a user's withdrawal failed six days later ([Wikipedia, Ronin Network](#)); the FBI attributed the theft to the North Korea-linked Lazarus Group three weeks after that.

Date	Event
Nov 2021	Axie DAO grants Sky Mavis a temporary gas-free signing allowlist

Date	Event
Dec 2021	Allowlist program ends; the delegated authority is never revoked
Mar 23, 2022	Attacker forges five of nine validator signatures, drains the bridge
Mar 29, 2022	A user's failed withdrawal surfaces the breach, six days late
Apr 14, 2022	FBI attributes the theft to the Lazarus Group

Every failure in that table is a preparation failure, not a detection-tooling failure: an unrevoked permission nobody tracked, and no monitoring that would have flagged \$624 million in unscheduled outflows in real time. Sections 2.2 and 3.1 turn each of those gaps into a control you put in place before, not during, your own incident.

1.2 Goals of Incident Response

NIST frames incident response goals around limiting damage and restoring normal operations at minimum cost ([NIST SP 800-61 Rev. 3](#)). A Web3 program keeps those goals but rewrites what “damage” and “normal operations” mean when the ledger is public, immutable, and adversarial by default.

Goal	Web3-specific expression	Primary metric
Limit further loss	Pause contracts, revoke approvals, coordinate exchange freezes on stolen funds	Time from detection to first containment action
Preserve evidence	Capture off-chain logs (node access, indexer, admin panel) before rotation; on-chain data is permanent, its surrounding context is not	Percentage of relevant off-chain logs retained
Maintain trust	Accurate, timely public communication that does not speculate ahead of facts	Time to first verified statement
Meet legal and regulatory duties	Law enforcement referral, breach notification where required, sanctions screening on recovered funds	Compliance with applicable disclosure windows
Reduce recurrence	Root-cause fix shipped and verified before funds move again	Findings closed per post-incident review

These goals are frequently in tension. Preserving evidence can slow containment; a fast public statement can outrun verified facts. Part 2 of this handbook works through that tension in detail; here, the point is that a responder who has not decided the priority order in advance will make that call, badly, under pressure.

1.3 Relationship to SCSVS, SCSTG, and SCWE

This handbook is the operational counterpart to the OWASP Smart Contract Security (SCS) Project's other standards, not a replacement for any of them (scs.owasp.org). The Smart Contract Security Verification Standard (SCSVS) defines the controls a system must satisfy before an incident; this handbook's detection and evidence requirements pick up the moment one of those controls fails anyway. The Smart Contract Security Testing Guide (SCSTG) describes how to test for weaknesses pre-deployment; the playbooks in Part 4 describe what to do when a weakness slips past that testing and is exploited live. The Smart Contract Weakness Enumeration (SCWE) catalogs the root-cause classes, reentrancy, access control, oracle manipulation, and so on; this handbook's job starts the instant one of those classes turns into a live exploit rather than an audit finding.

Two sibling handbooks extend this one outward: the EVM Forensics and DeFi Recovery Handbook (04) carries the post-incident chain analysis and fund-recovery work referenced in Part 4's playbooks, and the Web3 Attack Vectors Mapping Handbook (10) supplies the WA01 through WA15 taxonomy used once an incident is triaged and classified ([OWASP Web3 Attack Vectors Top 15](#)).

1.4 How to Use This Handbook

Read Part 1 once, straight through, to fix the lifecycle model and team roles; everything after this is reference material you jump into by need. Part 2 sets the communication rules that apply regardless of incident type. Part 3 is the operational core: detection sources, triage, evidence collection, containment. Part 4 is a playbook library, one chapter per incident type, from a live protocol hack to an insider threat to a decentralized team with no single incident commander; pull the matching chapter the moment an incident is declared. Part 5 closes the loop with post-incident review. A team already mid-incident should skip straight to the matching Part 4 playbook and the appendix templates, then circle back here once the fire is out.

2. Incident Response Lifecycle

Every phase in this chapter maps onto the six functions of NIST CSF 2.0, so the model below is worth internalizing before the detail: three functions describe what you build before an incident, three describe what you do during one, and the sixth loop is what carries lessons back to the start.

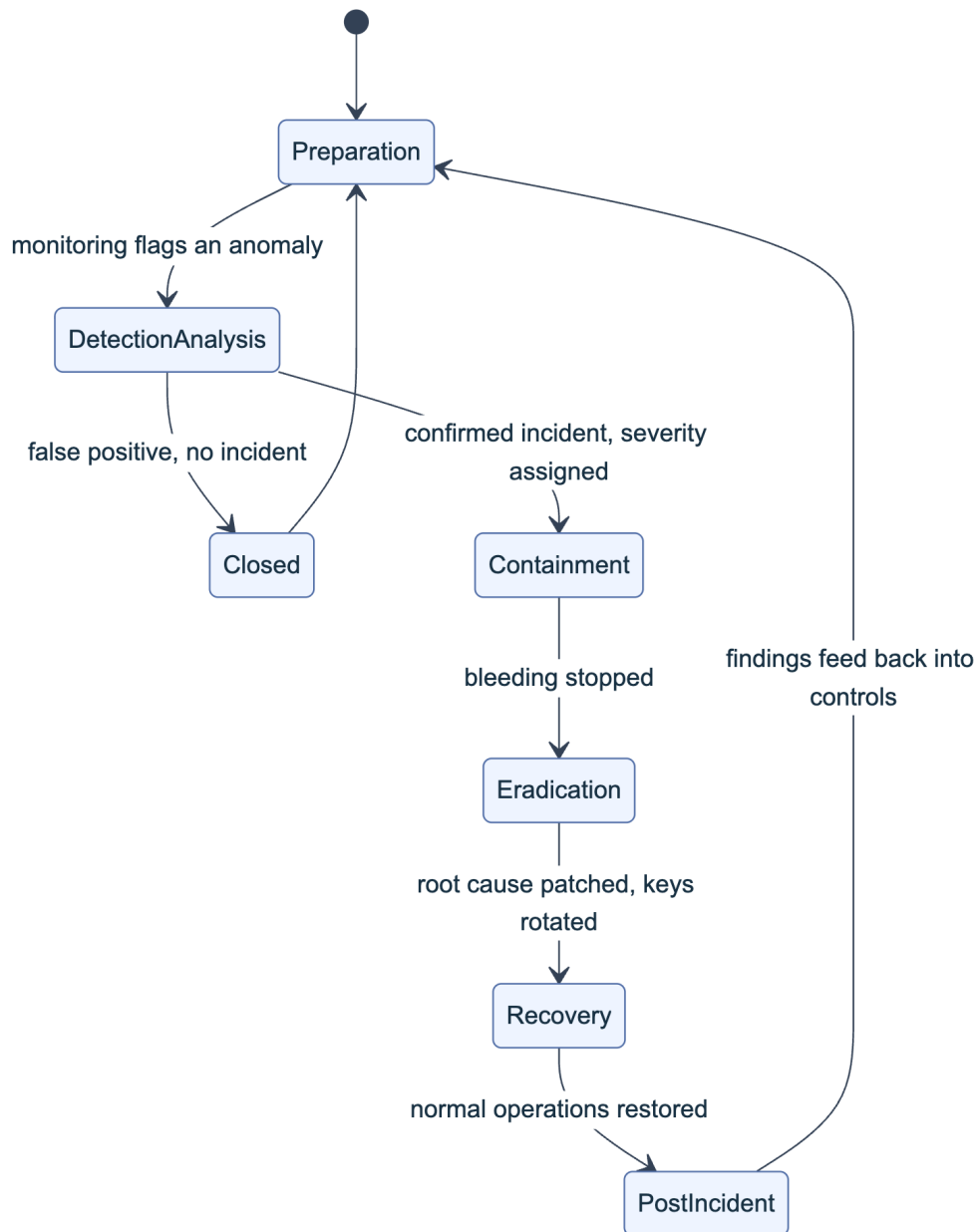


Figure 1. The operational incident lifecycle (Sections 2.2 through 2.5). A false positive exits early at triage; a confirmed incident runs the full containment-eradication-recovery arc before feeding its findings back into preparation.

2.1 NIST SP 800-61r3 and CSF 2.0 Model

NIST published SP 800-61 Revision 3, titled *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*, in April 2025 ([NIST SP 800-61 Rev. 3](#)). It departs from the older four-phase model (Preparation; Detection and Analysis; Containment, Eradication, and Recovery; Post-Incident Activity) that Revision 2 popularized, not by discarding it but by nesting it inside CSF 2.0's six functions ([NIST Cybersecurity Framework](#)), so incident response stops being a standalone document and becomes one thread inside the organization's overall risk management program.

Those six functions are Govern (risk strategy, policy, oversight), Identify (asset and risk visibility), Protect (safeguards), Detect (finding anomalies), Respond (acting on a confirmed incident), and Recover (restoring capability). CSF 2.0's headline change over CSF 1.1 (2018) was adding Govern as an explicit function, putting organizational accountability on the same footing as technical controls.

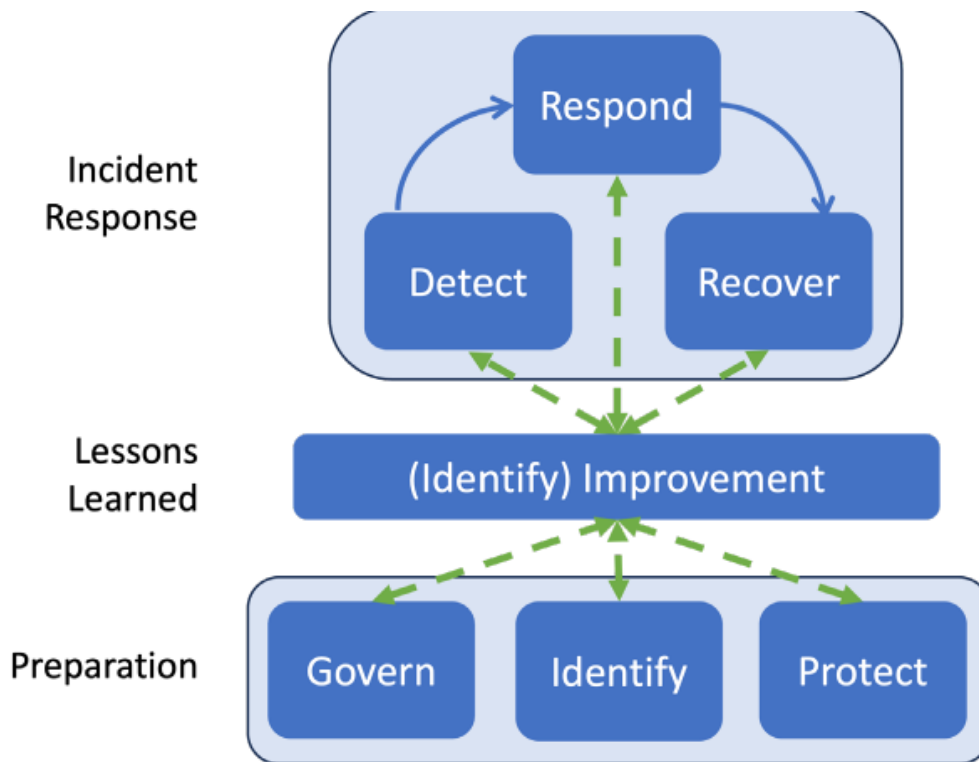


Fig. 2. Incident response life cycle model based on CSF 2.0 Functions

Figure. NIST's current incident-response lifecycle, not the retired four-phase wheel. Preparation spans Govern, Identify, and Protect; active incident handling spans Detect, Respond, and Recover; and lessons learned feed an explicit improvement layer that changes every function. Source: *NIST SP 800-61 Rev. 3, Figure 2*, public domain (U.S. government work).

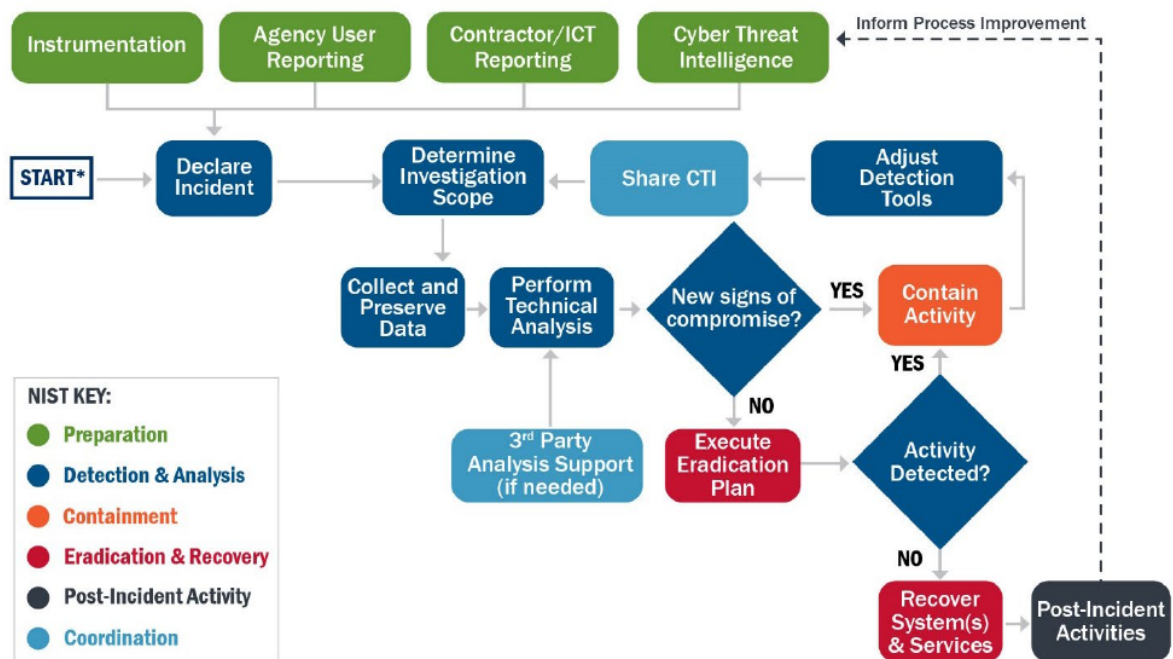


Figure 1: Incident Response Process

Figure. CISA's operational flow adds the decisions hidden inside a lifecycle wheel: declare, scope, collect, analyze, ask whether compromise remains, contain, eradicate, verify no activity remains, recover; and feed the result back into detection. It is a useful execution test for every playbook in Part 4. Source: *CISA Federal Government Cybersecurity Incident and Vulnerability Response Playbooks*, Figure 1, public domain (U.S. government work).

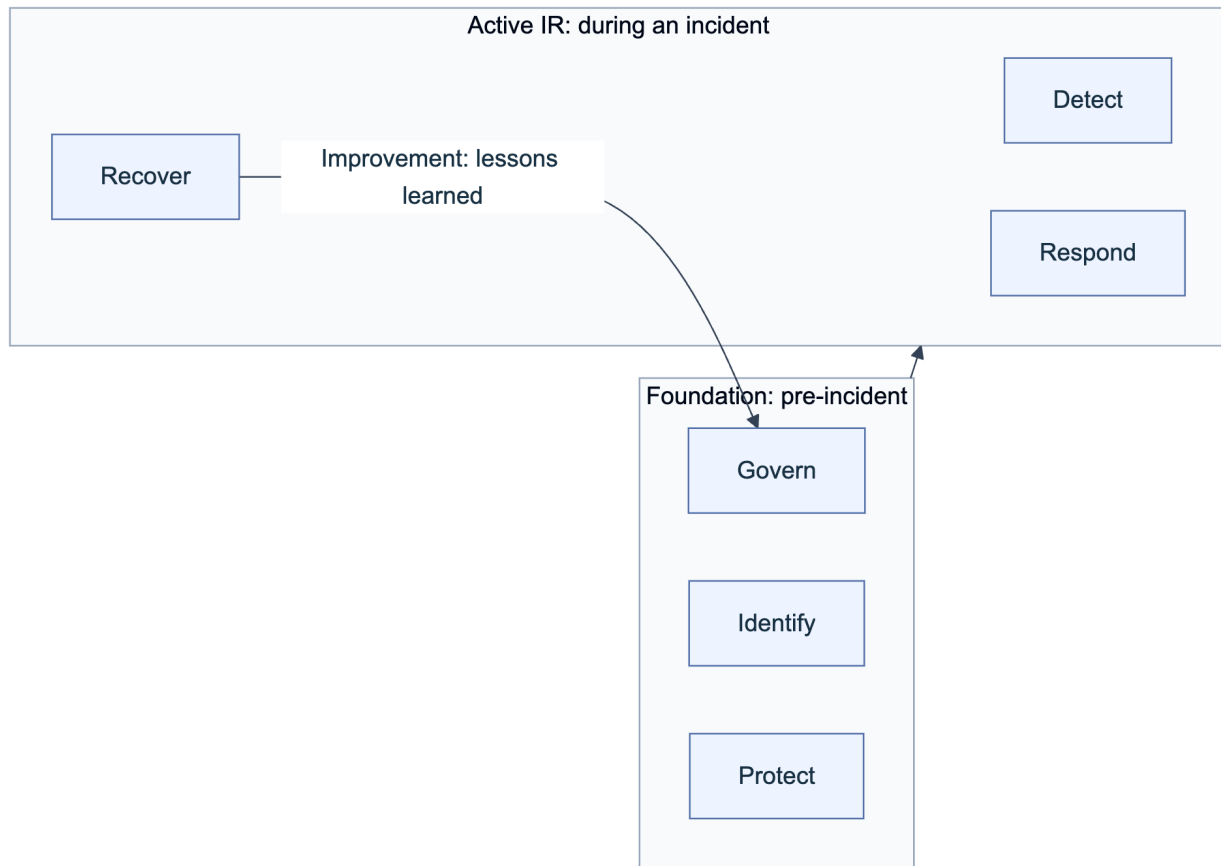


Figure 2. CSF 2.0's six functions grouped by this handbook into Foundation, Active IR, and the Improvement loop that closes back to Govern.

2.1.1 Active IR: Detect, Respond, Recover

Detect, Respond, and Recover are the functions that run once an anomaly appears. Detect covers everything in Section 2.3: turning a raw signal, a spike in outbound transfers, a failed withdrawal, a bug bounty report, into a confirmed, scoped, severity-assigned incident. Respond covers Section 2.4's containment and eradication work: stopping the bleeding, then removing the root cause. Recover covers restoring normal operation, which Section 2.4 also addresses, and which in Web3 specifically means redeployed contracts, rotated keys, and a treasury that is verifiably solvent again, not merely a status page returning to green.

2.1.2 Foundation: Govern, Identify, Protect

Govern, Identify, and Protect are what you have already built by the time Detect fires. Govern is the IRP itself, the authority chain (who can approve a pause, who can speak publicly), and the risk appetite the organization has actually agreed to, not the one it assumes. Identify is the asset inventory that makes Detect possible at all: every contract address, every admin key, every treasury wallet, every third-party dependency and its owner, kept current, not reconstructed from memory during a crisis. Protect is the safeguards layer: access control on privileged functions, circuit breakers and pausability, monitoring coverage, and the incident response plan's own tooling (paging, secure channels, pre-staged templates) kept ready to fire.

2.1.3 Improvement: Continuous Lessons Learned

The sixth loop is not one of the six CSF functions by name; it is the arrow this handbook draws from Recover back to Govern. An incident that does not change a control, a monitor threshold, a key-holder list, or a line in the IRP has been survived, not learned from. Part 5 of this handbook (Lessons Learned and Improvement) is where that loop gets formal structure: blameless review, tracked remediation items, and updated playbooks. It is referenced here because the loop only closes if Govern is staffed and empowered to receive the findings, which is a Foundation-function responsibility, not an afterthought bolted onto the end of a postmortem document.

2.2 Preparation (Identify, Protect)

Preparation is judged by what exists before the pager goes off, not by what gets written afterward. The following is the minimum baseline this handbook assumes a team has in place; Part 3, Chapter 7 expands it into a full checklist.

- ☐ A written IRP naming every role in Chapter 3, with backups for each
- ☐ Current inventory of every contract, admin key, multisig, and treasury address, with owners
- ☐ Pausability or emergency-exit mechanisms tested on a fork, not just written into the contract
- ☐ Monitoring coverage for large transfers, minting events, and ownership or role changes (Part 3, Chapter 5)
- ☐ Pre-established relationships: forensics retainer, legal counsel, at least one exchange security contact, and a channel into SEAL 911 or an equivalent crypto incident-response network ([Security Alliance](#))
- ☐ Draft communication templates for the first hour, first day, and first week (Part 2, Chapter 4; Appendix D)
- ☐ A defined severity and triage matrix agreed on before, not during, an incident (Appendix B)

2.3 Detection and Analysis

Detection and analysis is the work of turning a raw signal into a confirmed, scoped incident: is this real, how bad is it, and who needs to be paged. Part 3, Chapter 5 covers detection sources in depth (on-chain monitors, bug bounty intake, community reports, off-chain SIEM-style logging); this section fixes the concept the rest of the handbook depends on. Analysis has two jobs that run in parallel: confirm the anomaly is an actual security event rather than a benign spike (a large legitimate withdrawal looks identical to an attack at the transaction-hash level until context is added), and establish an initial severity so the right people get paged rather than everyone or no one.

Section 1.1's Ronin case study is the cautionary example again: a six-day exploit-to-detection gap is what an absent tooling layer costs when nobody but an affected user is watching. A functioning detection phase converts that window into minutes, and the difference, a fully drained bridge versus a contained partial loss, is the entire value of this phase.

2.4 Containment, Eradication, and Recovery

These three activities are distinct even though they often run back to back within the same hour. Containment stops the loss from growing: pausing affected contracts, revoking token approvals, coordinating with exchanges and bridges to flag or freeze addresses holding stolen funds. Eradication removes the root cause so the attack cannot immediately repeat: patching the vulnerable code, rotating any compromised key or signer, revoking any stale permission (an unrevoked allowlist, as in Ronin's case) that made the attack possible. Recovery restores verified-normal operation: redeploying fixed contracts, confirming treasury solvency against an independent snapshot, and resuming service only once monitoring confirms the attacker no longer has a foothold.

Phase	Primary question	Typical Web3 action
Containment	How do we stop the loss right now?	Pause, revoke approvals, exchange freeze requests
Eradication	What made this possible, and is it gone?	Patch, rotate keys, revoke stale permissions
Recovery	Are we verifiably back to normal?	Redeploy, reconcile treasury, resume monitoring at heightened sensitivity

Whitehat negotiation and fund recovery, an increasingly common path after containment (attacker returns funds for a bounty, sometimes under a published Safe Harbor policy), is its own playbook and is covered in Part 4, Chapter 15 rather than here.

2.5 Post-Incident Activity and Recovery

Post-incident activity is where the Improvement loop from Section 2.1.3 gets executed. At minimum it produces a blameless root-cause analysis, a public post-mortem proportional to the incident's visibility, concrete remediation items with owners and deadlines, and, where users

suffered loss, a restitution plan. It also closes the legal thread opened in Section 1.2: law enforcement referral where warranted, mandated breach notifications, and sanctions screening on recovered or frozen assets before redistribution. Part 5 develops the postmortem format and improvement-tracking mechanics in full; this section exists so the lifecycle does not silently end at “Recovery” and imply the work is finished once the service is back online.

2.6 Supplemental NIST Resources

SP 800-61 Revision 3 deliberately defers implementation detail to companion NIST publications rather than restating them. Three are directly relevant to a Web3 incident program, because they cover exactly the gaps a purely on-chain view of “the record” leaves open: logs that are not on any chain, recovery planning that has to be rehearsed before it is needed, and the sharing relationships that make Section 2.3’s detection window shorter than Ronin’s six days.

Publication	Title	Status	Web3 relevance
SP 800-92 Rev. 1	Cybersecurity Log Management Planning Guide	Initial public draft, Oct. 2023	Off-chain logs (RPC, indexer, admin panel) are the perishable evidence the chain does not keep
SP 800-184	Guide for Cybersecurity Event Recovery	Final, Dec. 2016	Recovery playbooks and rehearsed failover for pausable, upgradeable systems
SP 800-150	Guide to Cyber Threat Information Sharing	Final, Oct. 2016	Sharing attacker addresses, drainer contracts, and phishing domains across protocols

2.6.1 SP 800-92 Rev. 1 (Log Management)

SP 800-92 Revision 1, an initial public draft dated October 2023, is a planning playbook covering the full log lifecycle: generation, transmission, storage, access, and disposal, across physical, virtual, and cloud environments ([NIST SP 800-92 Rev. 1 IPD](#)). For a Web3 team, the relevant insight is what falls outside the chain’s own permanence: Remote Procedure Call (RPC) access logs, indexer query logs, admin-panel session logs, oracle and keeper-bot activity. Chain state is immutable once finalized, but the operational context around an exploit, who accessed what infrastructure and when, lives in systems with default retention measured in days. A log plan set before an incident, with retention long enough to survive a gap like Ronin’s, keeps that context from rotating out before anyone knows to look.

2.6.2 SP 800-184 (Event Recovery)

SP 800-184, finalized in December 2016, guides recovery planning: identifying and prioritizing the resources a plan depends on, developing and testing playbooks, and feeding lessons learned back in ([NIST SP 800-184](#)). Its core argument, that recovery capability degrades if never re-

hearsed, applies with extra force to smart contract systems: a pause function nobody has tested on a mainnet fork, or a multisig recovery procedure that has never actually rotated a signer, is a plan on paper, not a proven capability. Section 2.2's checklist item to test pausability on a fork applies this guidance directly.

2.6.3 SP 800-150 (Threat Information Sharing)

SP 800-150, finalized in October 2016, guides establishing and joining cyber threat information sharing relationships: setting sharing goals, identifying sources, scoping what gets shared, engaging with sharing communities ([NIST SP 800-150](#)). Web3 has built its own version: crypto-specific sharing centers such as Security Alliance, whose SEAL 911 service provides emergency incident response and whose network shares indicators (attacker wallets, drainer bytecode, phishing infrastructure) across otherwise-competing protocols ([Security Alliance](#)). A protocol outside one of these channels relies entirely on its own detection; one inside can benefit from another team's earlier sighting of the same attacker infrastructure.

3. Roles and Responsibilities

A lifecycle model is inert without people, or automated systems, assigned to execute each phase. This chapter defines the minimum role set and, in Section 3.7, the concrete mechanisms that bind each role to a verifiable, revocable identity.

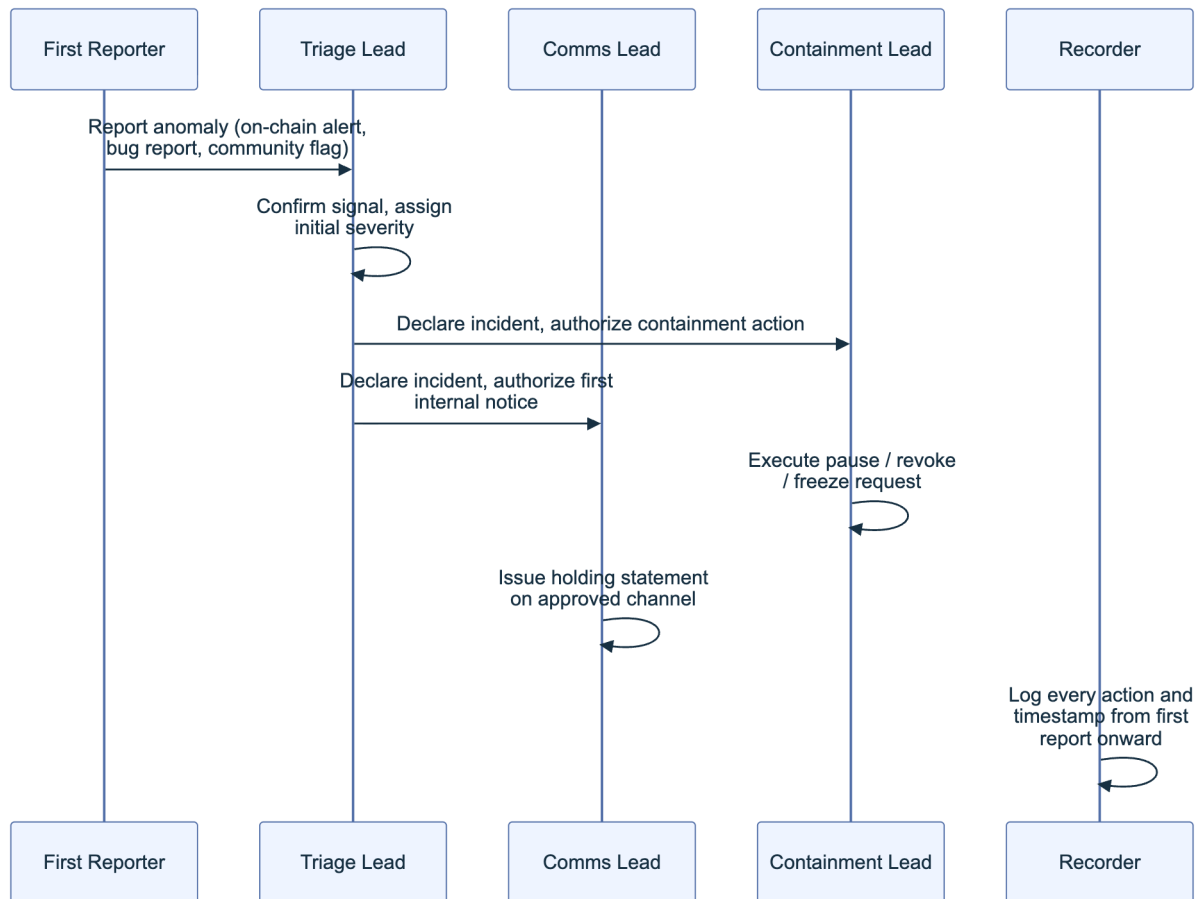


Figure 3. Role handoffs in the first response window. The Recorder listens throughout and is not a downstream step; it captures every message on this diagram as it happens.

3.1 Incident Response Team (IRT) and Incident Response Plan (IRP)

The Incident Response Team (IRT) is the cross-functional group with pre-assigned authority to act during an incident: security engineering, protocol developers, communications, legal, treasury or multisig signers, and, for community-governed protocols, moderator representation. The Incident Response Plan (IRP) is the living document naming who holds each role, what they may do without further approval, and how to reach them if primary channels are compromised. NIST SP 800-61 describes three organizational models for structuring this team, each fitting a different Web3 org shape.

Model	Structure	Typical Web3 fit
Centralized	Single team handles all incidents across the organization	Single-protocol team with one codebase and one treasury
Distributed	Multiple teams, each responsible for its own systems	DAO with independently operated sub-protocols or chains

Model	Structure	Typical Web3 fit
Coordinating	A central team advises and coordinates independent teams, without full authority over them	Cross-protocol alliances such as Security Alliance coordinating member responses

An IRP with no named IRT is a document nobody is accountable to; an IRT with no IRP is a group of capable people who will spend the first critical hour arguing about authority instead of using it.

3.2 First Reporter

The First Reporter is whoever or whatever raises the first flag, and in Web3 that is frequently not an employee: an automated on-chain monitor, a whitehat who noticed an exploitable pattern, an anonymous bug bounty submission, or a community member on Discord or X flagging a transaction that looks wrong. Unlike a traditional security operations center where the first reporter is almost always internal tooling, a Web3 protocol's most valuable first reporters are frequently external, sometimes pseudonymous, first-time contacts. The intake channel this role depends on has to accept a report without demanding identity verification as a precondition; requiring KYC-style proof before triage begins is how a six-day-late report becomes a seven-day-late one. Part 2, Chapter 4 covers that intake channel's design; here it is enough to name the role and staff the Triage Lead to receive it around the clock.

3.3 Triage Lead

The Triage Lead is the first internal decision-maker: confirm whether the report describes a real security event, assign an initial severity using the matrix in Appendix B, and decide whether to formally declare an incident and page the rest of the IRT. This role absorbs the ambiguity a First Reporter's message always carries, an anonymous Discord message reading "is this normal?" is not yet actionable, and converts it into a scoped, severity-tagged incident the Comms and Containment Leads can act on without re-litigating credibility. The role needs judgment, authority to page others, and availability, not seniority, which is why every IRP names a backup.

3.4 Comms Lead (Primary and Backup Spokespersons)

The Comms Lead owns every word the organization says about an incident, internally and externally, and is the single point of truth preventing two team members from posting contradictory statements. A backup spokesperson is not optional: the primary may be asleep, traveling, or unreachable when an incident starts at an inconvenient hour, and in some incidents the primary's own account is the attack surface, protocol and executive social accounts have themselves been hijacked mid-incident to spread disinformation under a trusted name. Part 2, Chapter 4 develops the message discipline this role executes; here the point is that the role and its backup must be named before an incident, not volunteered for during one.

3.5 Containment Lead

The Containment Lead holds the technical authority to execute the actions Section 2.4 describes: pausing a contract, submitting a multisig transaction to revoke a role, or coordinating an emergency upgrade. This role works directly off the Triage Lead's severity call and does not independently re-decide whether action is warranted mid-crisis; the IRP should specify in advance which severity levels authorize which containment actions, so the Containment Lead executes a pre-agreed decision rather than a novel judgment call under pressure. Because these actions are irreversible on-chain the moment they are submitted, access must be pre-provisioned exactly as Section 3.7 describes, not granted ad hoc mid-incident.

3.6 Recorder and Timeline Keeper

The Recorder maintains the single authoritative timeline: every report, decision, and action, timestamped, captured as it happens rather than reconstructed afterward from memory or chat scrollback. This record becomes the backbone of the post-incident review in Section 2.5, the evidentiary basis for any law enforcement referral, and the documentation an insurer or auditor will ask for. Contemporaneous logging matters because memory under crisis conditions is unreliable and participants are simultaneously busy doing the work the timeline needs to capture; the Recorder's only job mid-incident is to listen and log, not to also be the Triage Lead or Containment Lead.

3.7 Mapping Roles to Identities (GPG, Multisig, OIDC, etc.)

A role name is not an access control. "Containment Lead" has to resolve to a specific, durable, revocable credential before an incident, because issuing new credentials during one is itself a security risk: how does the rest of the team verify that a newly added multisig signer is the actual Containment Lead's backup and not the attacker exploiting the chaos to add a sixth key. Bind each role to a mechanism chosen for what that role actually needs to prove.

Role	Identity mechanism	What it proves	Revocation path
Containment Lead	Multisig signer key (e.g., a Safe{Wallet} owner)	Authorization to co-sign pause, upgrade, or fund-movement transactions	Remove from the Safe owner list; requires the remaining signer threshold
Comms Lead	GNU Privacy Guard (GPG) key (RFC 4880) with a fingerprint published in advance	A public statement genuinely came from the organization, checkable even if official channels are themselves compromised	Revoke and publish revocation to keyserver; rotate published fingerprint
Triage Lead, Recorder	OpenID Connect (OIDC)-backed single sign-on to internal IR tooling (OpenID Connect Core 1.0)	Access to monitoring dashboards and paging systems	Deprovision at the identity provider; immediate for employees, slower for contractors without SSO
First Reporter	None required by design	N/A: identity is deliberately not a precondition for intake	Not applicable

A minimal role registry, kept alongside the IRP and reviewed every time the team changes, makes this mapping auditable rather than tribal knowledge:

```
# irp/role_registry.yaml
roles:
  containment_lead:
    primary: alice.eth
    backup: bob.eth
    identity: multisig_signer
    safe_address: "0xSAFE...treasury"
  comms_lead:
    primary: carol
    backup: dave
    identity: gpg_key
    gpg_fingerprint: "AAAA BBBB CCCC DDDD EEEE FFFF 0000 1111 2222 3333"
  triage_lead:
    primary: erin
    backup: frank
    identity: oidc_sso
    idp: "https://idp.example.org"
```

The GPG-signed statement solves a problem specific to a live incident: if an attacker has taken over the protocol’s official Twitter or Discord account, the fastest verification path for a worried user is not “does this look official” but “does this message verify against a fingerprint published somewhere the attacker cannot also control.” Publishing that fingerprint is Section 2.2 preparation work; using it under pressure is what makes it worth having done.

Key controls for Part 1

- Adopt NIST SP 800-61 Rev. 3’s CSF 2.0 model: Govern, Identify, and Protect before an incident; Detect, Respond, and Recover during one; lessons learned closing the loop back to Govern.
- Maintain a current asset inventory (contracts, keys, treasury addresses, third-party dependencies) as the Identify function’s concrete output, not a document that exists only in someone’s memory.
- Staff and name, with backups, every role in Chapter 3 before an incident: First Reporter intake, Triage Lead, Comms Lead, Containment Lead, and Recorder.
- Bind every role to a durable, revocable identity mechanism (multisig signer, GPG key, OIDC-backed SSO) chosen before the incident, never provisioned during one.
- Join at least one crypto-specific threat information sharing channel (SEAL 911 or equivalent) so detection does not depend solely on in-house monitoring.

Part II: Communication Strategies

Poly Network recovered \$610 million from its own attacker in fifteen days by talking to him in public; Multichain lost its users' trust for good by saying almost nothing while roughly \$1.5 billion sat frozen behind a CEO who had gone silent months earlier. The exploit was not what separated those two outcomes: the communication plan was. This part sets the rules for incident communication, who is allowed to speak, on which channel, to which audience, and on what clock, from the first internal page through the final public retrospective. It treats disclosure as a control with its own access model and approval workflow, because a message sent on the wrong channel, to the wrong regulator, or on the wrong timeline can cost a protocol as much as the bug that started the incident.

4. Communication During Incidents

Every incident runs two parallel workstreams: the technical response and the story about the technical response. The second workstream needs its own plan, because in a public, permissionless, always-watching ecosystem, silence and inconsistency get read as confirmation of whatever worst-case rumor is already circulating on crypto Twitter within minutes of an unusual on-chain transaction.

4.1 Internal Communication

Internal communication has to be solved before external communication, because a team that cannot coordinate with itself cannot present a coherent face to anyone else. Two failure modes recur across real incidents: coordinating over a channel the attacker can also read, and routing a decision through nobody in particular.

4.1.1 Secure Channels and Access Control

The default assumption during a live incident should be that the primary company chat tool might itself be compromised. This is not a hypothetical: in the September 2022 Uber breach, an attacker who had socially engineered a contractor's multi-factor authentication prompt escalated to broad internal access and, in the company's own words, "posted a message to a company-wide Slack channel, which many of you saw," while also downloading internal Slack message history ([Uber security update](#)). A response team coordinating containment steps in the same workspace the attacker is reading is handing over the play-by-play in real time.

The fix is a pre-provisioned, out-of-band channel that does not depend on the same identity provider, ticketing system, or chat workspace as production. Create it before an incident, not during one, so responders are added by role rather than improvised under pressure. Scope membership tightly: not every engineer needs war-room access, and comms channel membership should never double as production system access. [NIST SP 800-61 Revision 3](#) frames this under Govern function element GVRM-05, "lines of communication across the organization are established for cybersecurity risks," and under RS.CO-02, which requires following an established procedure for who is notified, when, and through what channel, rather than deciding in the moment.

Severity	Channel	Membership	Retention during investigation
Sev1 (active fund loss)	Pre-provisioned Signal "war room" group	Incident Commander, on-call responders, legal, spokesperson, exec sponsor	Disappearing messages disabled
Sev2 (contained, urgent)	Same Signal group, reduced roster	Incident Commander, technical responders, legal	Disabled
Sev3/4 (low or no fund risk)	Primary company chat, dedicated incident channel	Response team and relevant engineering	Standard retention
Post-incident retrospective	Primary company chat or wiki	Full response team and stakeholders	Standard retention

4.1.2 Escalation Paths and On-Call

Escalation exists so that severity, not seniority or luck, determines who gets paged. In Web3 the first trigger is often a bot, not a human: an on-chain monitor (Chapter 5 covers detection sources) fires a webhook the moment an anomalous transaction lands, and the paging system needs to accept that webhook directly rather than waiting for a person to notice a dashboard. Pair every

page with an acknowledgment timer and an automatic escalation to a secondary responder if nobody acks in time, because a responder who is asleep, off-grid, or simply missed a notification cannot be the single point of failure for a Sev1.

Adopt the Incident Commander model borrowed from the Incident Command System and refined by Google's Site Reliability Engineering practice: one named person owns the decision authority for the duration of the incident, distinct from whoever is fixing the bug. For decentralized and globally distributed teams, the escalation policy has to assume no single timezone is reliably staffed; Part IV Chapter 14 (War Room and Crisis Coordination) and Chapter 13 (Decentralized Incident Response) build directly on the roster this section defines.

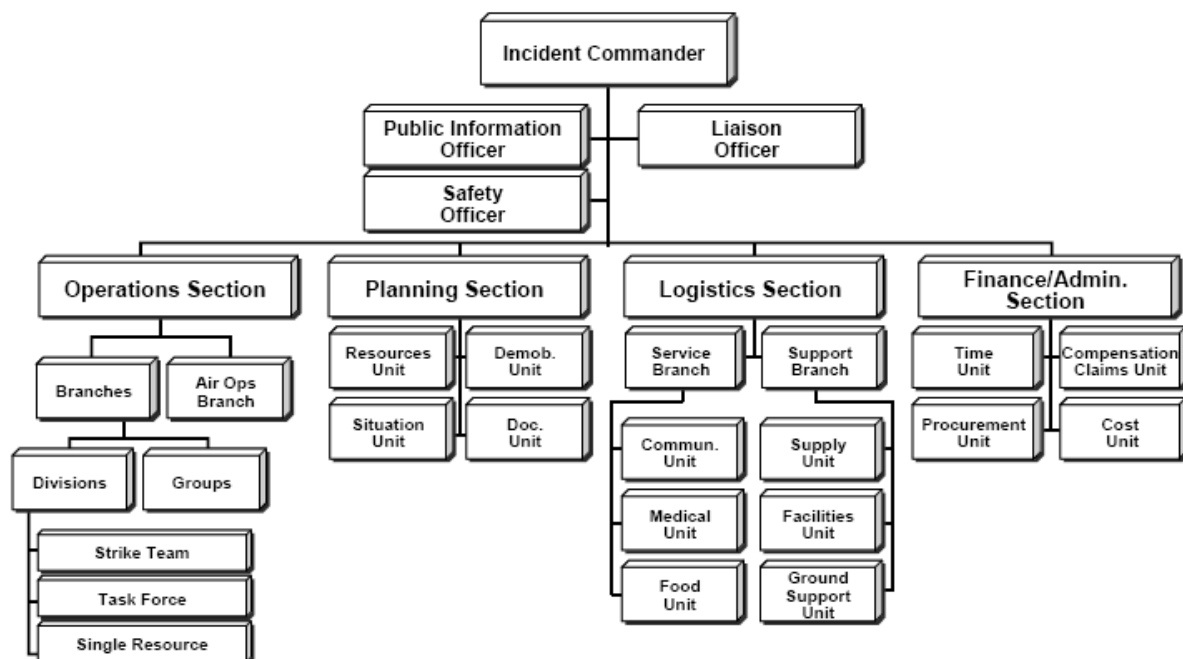


Figure. The FEMA-standardized Incident Command System structure that the Incident Commander model borrows its single-decision-owner principle from, adapted here to name a security incident's roles rather than a physical disaster response's. Source: FEMA, via Wikimedia Commons, public domain (US government work).

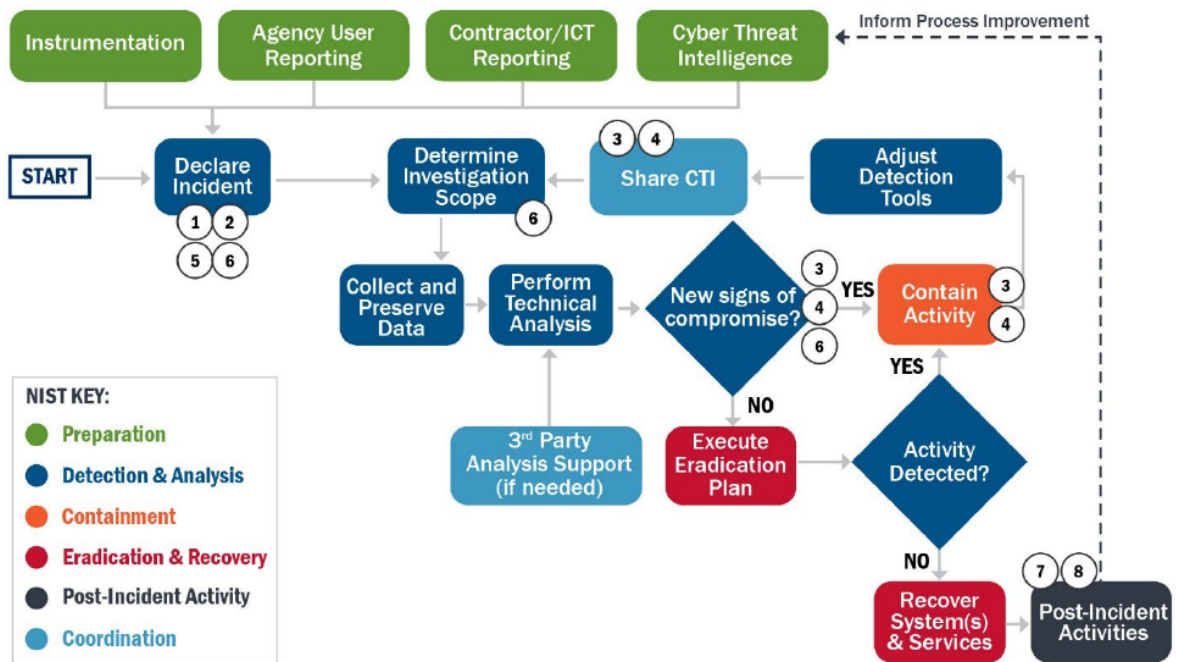


Figure 2: Numbered IR Coordination Activities

Figure. Coordination is continuous, not a single notification at declaration. CISA marks separate reporting and information-sharing actions at the points where scope, indicators, containment status, recovery, and lessons become known; an external update sent once and never refreshed leaves every partner acting on stale facts. Source: *CISA Federal Government Cybersecurity Incident and Vulnerability Response Playbooks*, Figure 2, public domain (U.S. government work).

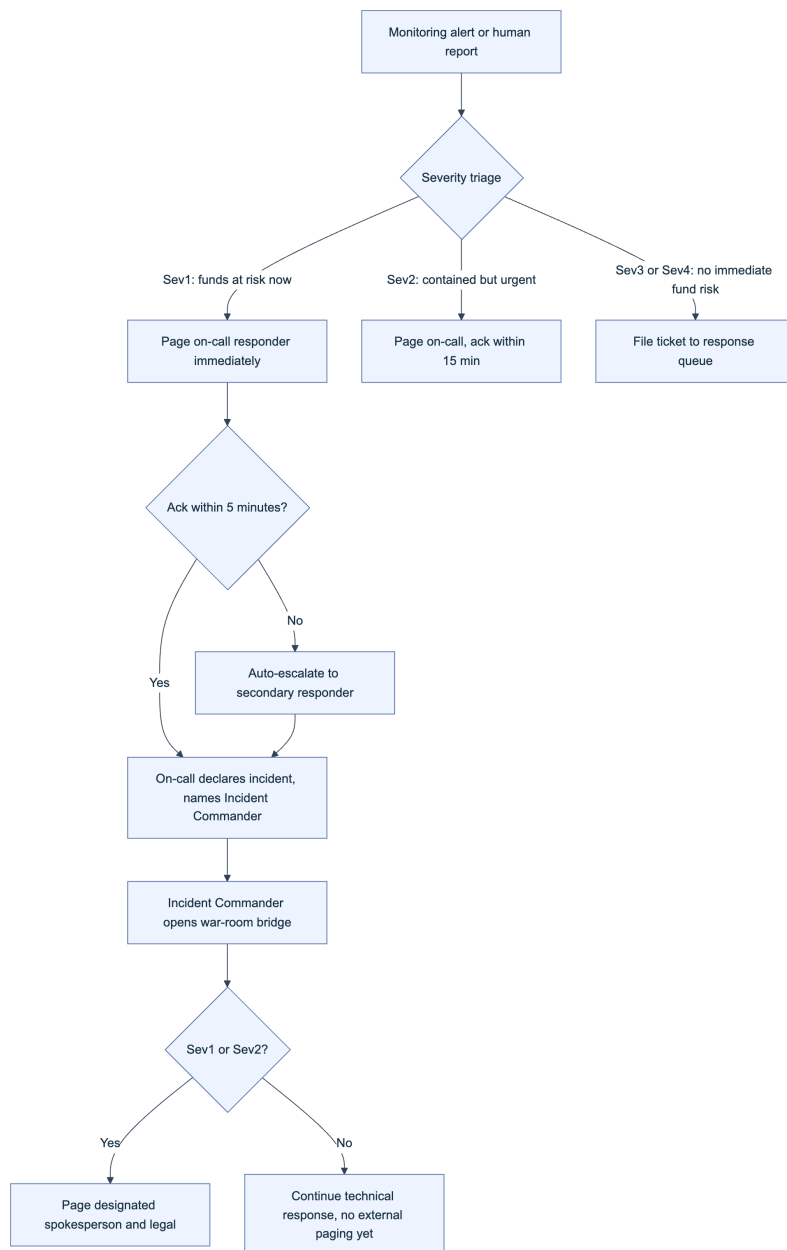


Figure 6. Escalation flow from first alert to spokesperson activation. The acknowledgment timer at F prevents a single unresponsive on-call engineer from stalling a Sev1.

```
# Example escalation policy, PagerDuty/Opsgenie-style pseudo-configuration
escalation_policy:
  name: "protocol-security-sev1"
  rules:
    - level: 1
      target: on_call_primary
      timeout_minutes: 5
    - level: 2
      target: on_call_secondary
      timeout_minutes: 5
    - level: 3
      target: incident_commander_pool
      timeout_minutes: 10
  triggers:
    - source: monitoring_webhook      # e.g. Forta, OpenZeppelin Defender, custom bot
      severity: [sev1, sev2]
      auto_page: true
    - source: manual_report
      severity: [sev1, sev2, sev3, sev4]
      auto_page: false               # routed to response queue instead
```

4.2 External Communication

Once a message crosses the organization's boundary it is effectively permanent and can move markets, so the audience, channel, and timing all need to be decided in advance rather than improvised. NIST SP 800-61 Rev. 3 groups external communication into distinct categories under RS.CO and RC.CO: incident notification (formally informing affected parties), public communication (status updates and media response), and incident information sharing (voluntary threat intelligence exchange) (NIST SP 800-61r3, Section 3). The three subsections below map to three audiences that need distinctly different content, tone, and legal handling: the community, regulators and law enforcement, and integrated partners.

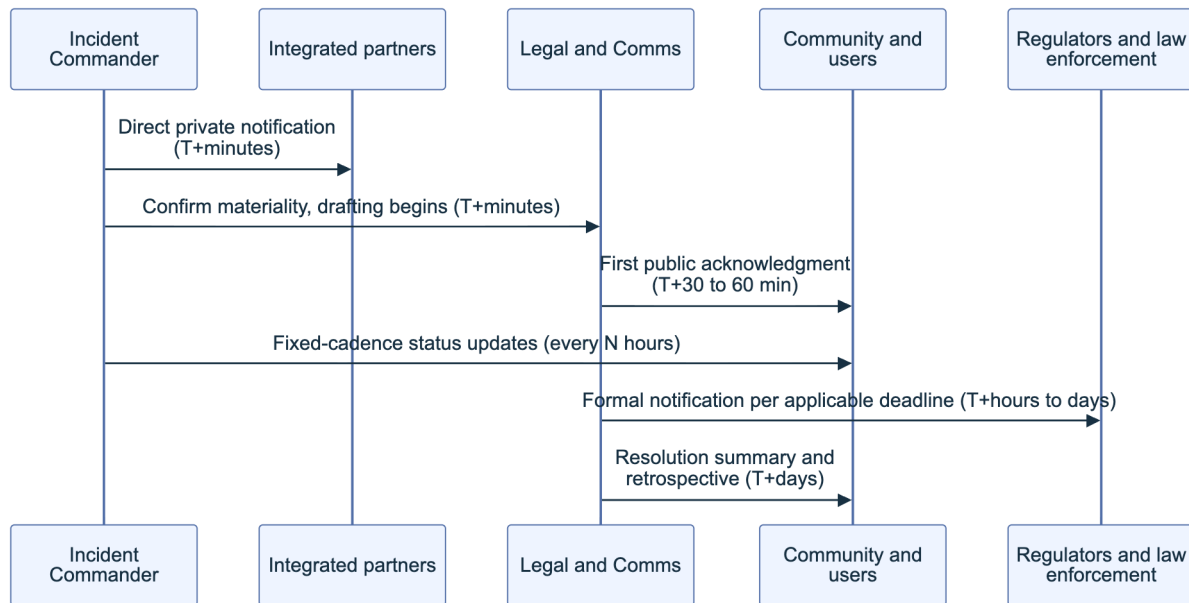


Figure 7. Relative order of external notifications. Partners are told before the public because they must make their own containment decisions; regulators are told on a legally defined clock that may run longer than the public acknowledgment clock.

4.2.1 Community and User Updates

Speed of acknowledgment matters more than completeness of information. The first public message should state what is confirmed, what is not yet confirmed, and the one immediate action users can take to protect themselves, such as revoking a token approval or pausing interaction with a specific contract. Route every channel through a single source of truth, typically a status page, and mirror it to X (formerly Twitter), the Discord announcement channel, and Telegram rather than letting three people paraphrase the same fact slightly differently in three places.

Poly Network’s response to its August 2021 exploit is the reference case for engaging an attacker in public. After roughly \$610 million moved to attacker-controlled addresses across Ethereum, BNB Chain, and Polygon, the team posted an open letter on Twitter addressed directly to the attacker, and the two sides continued the exchange through messages embedded in on-chain transaction data, with the team publicly offering a bug bounty and a security advisor role in exchange for returning the funds. Nearly all assets were returned over the following fifteen days ([Poly Network exploit, Wikipedia](#)). Ronin Network’s March 2022 bridge hack shows the cost of the opposite choice: the roughly \$620 million theft, later attributed by the FBI to North Korea’s Lazarus Group, was not publicly disclosed until about six days after the exploit occurred, a gap that left users bridging and trading against an insolvent system without knowing it ([Ronin Network, Wikipedia](#)). Silence is itself a message, and during an active incident it reads as either “there is something to hide” or “we do not know what is happening,” both worse than a prompt, honest “we are investigating.”

Channel	Purpose	Update cadence (Sev1)	Owner
Status page	Single source of truth, timestamped	First, and on every material change	Spokesperson
X / Twitter	Widest public reach	Mirrors status page	Spokesperson
Discord announcement	Active user and developer community	Mirrors status page, pinned	Community lead
Telegram	Regional and community reach	Mirrors status page	Community lead
Blog or Mirror post	Full retrospective	Once, at resolution	Spokesperson and legal

```
[TIME UTC]: We are investigating [a reported anomaly / a confirmed exploit] affecting
[contract or component].
Confirmed: [fact 1]. [fact 2].
Not yet confirmed: [open question].
Immediate action for users: [pause interactions / revoke approvals / do not bridge].
Next update: within [N] hours, or sooner if the situation changes materially.
```

4.2.2 Regulators and Law Enforcement

Regulatory obligations vary by legal form and jurisdiction, so the incident response plan needs a pre-built contact sheet rather than a research project started mid-incident. A US public company (or one that voluntarily files with the SEC) must disclose a material cybersecurity incident under Item 1.05 of Form 8-K within four business days of determining the incident is material, a rule the Securities and Exchange Commission (SEC) adopted in July 2023 and made effective in December 2023 ([SEC cybersecurity disclosure rule](#)). A US-registered money services business (MSB) or virtual asset service provider (VASP) must file a Suspicious Activity Report (SAR) with the Financial Crimes Enforcement Network (FinCEN) within 30 days of detecting a reportable transaction, extendable to 60 days if no suspect has been identified, under 31 CFR 1022.320 ([FinCEN SAR filing guidance](#)). Entities that fall within a designated critical infrastructure sector face the Cyber Incident Reporting for Critical Infrastructure Act (CIRCIA), which the Cybersecurity and Infrastructure Security Agency (CISA) is implementing with a 72-hour deadline for a substantial cyber incident and 24 hours for a ransom payment ([CISA CIRCIA](#)).

Law enforcement contact points belong on the same sheet: the FBI's Internet Crime Complaint Center (IC3) for US-based reporting ([ic3.gov](#)), Europol's European Cybercrime Centre (EC3) for EU-relevant incidents, and national police units elsewhere. If stolen funds route through a sanctioned mixing service, US persons and entities carry an immediate blocking obligation under Office of Foreign Assets Control (OFAC) rules. Framing the initial technical report against a shared taxonomy, such as [MITRE ATT&CK](#) or MITRE's crypto-focused [AADAPT](#) matrix, speeds up

the law enforcement conversation by removing ambiguity about the technique used. The EVM Forensics and DeFi Recovery Handbook (04) covers the freeze-request and chain-analysis mechanics that typically follow this first contact.

Agency or framework	Trigger	Deadline	Jurisdiction
SEC Item 1.05, Form 8-K	Cybersecurity incident determined material	4 business days after materiality determination	US public/reporting companies
FinCEN SAR	Suspicious transaction detected by an MSB or VASP	30 days (60 if no suspect identified)	US-registered MSBs/VASPs
CISA CIRCIA	Substantial cyber incident at a covered entity	72 hours (24 hours for ransom payment)	US critical infrastructure sectors
FBI IC3	Any cyber-enabled theft	No fixed deadline; earlier improves recovery odds	US, with cross-border referral
OFAC	Funds routed through a sanctioned address or mixer	Blocking obligation attaches on match	US persons and entities
National equivalents (Europol EC3, local police)	Jurisdiction-dependent	Varies	Non-US

4.2.3 Partners and Dependencies

An incident rarely stays contained to one protocol. Integrated partners need direct, private notification ahead of the public timeline, because they must make their own risk decisions, pausing a contract, disabling a UI route, halting a bridge leg, before the news breaks and turns that decision into a race against every other holder of the same exposure. The March 2023 Euler Finance hack illustrates the pattern: protocols that had deployed capital into Euler's markets, including pools on Balancer and collateral backing Angle Protocol's agEUR stablecoin, carried downstream exposure to the frozen funds and needed a direct heads-up rather than learning about it from social media.

Build the partner list around categories, not individual names that turn over. Oracle providers may need to pause a price feed a compromised contract still consumes. Bridge counterparties need synchronized pausing on both chains, since a one-sided pause leaves the other chain accepting deposits against a broken bridge. Exchanges can freeze or flag an attacker-controlled deposit address and halt withdrawals of an affected wrapped asset if notified quickly. Custodians and market makers need to know before they keep quoting an asset whose backing is in question, and

audit firms that reviewed the affected code often hold prior context that shortens triage. Assign one owner to fire every partner notification from the Incident Commander's log, so partners never receive contradictory messages from different employees.

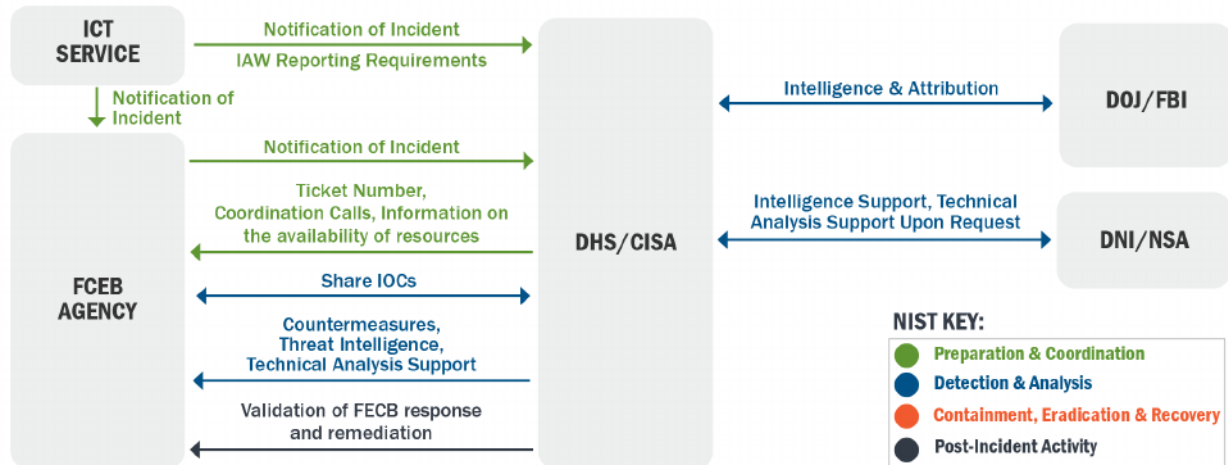


Figure 3: Whole of Government Asset Response

Figure. CISA's federal example generalizes directly to a Web3 response network: the affected protocol remains the evidence owner, a central coordinator routes requests, and specialized partners return different products—indicators, countermeasures, attribution, or technical analysis. The diagram is a reminder to name the information exchanged on each edge, not merely list organizations in a phone tree. Source: *CISA Federal Government Cybersecurity Incident and Vulnerability Response Playbooks*, Figure 3, public domain (U.S. government work).

Partner category	Notification trigger	Expected action	Owner
Oracle providers	Compromised contract consumes their feed	Pause or deprecate the feed for the affected consumer	Incident Commander
Bridge counterparties	Bridge contract exploited on either leg	Synchronized pause on both chains	Incident Commander and partner engineering lead
Centralized exchanges	Attacker address identified	Freeze or flag deposit address, halt affected withdrawals	Legal and comms
Custodians and market makers	Treasury or liquidity-provider funds affected	Risk-adjust exposure, pause quoting	Executive sponsor
Auditors of record	Exploit falls within previously audited scope	Share prior findings, assist triage	Incident Commander

4.3 Best Practices

Four cross-cutting practices apply to every message sent during an incident, regardless of which audience receives it.

4.3.1 Pre-Approved Templates for Notifications, Updates, and Press

Draft and legally review message templates before an incident, not during one. Legal review under active-incident time pressure both slows the message and raises the odds that something inaccurate reaches the public. Maintain a small library covering the recurring moments: initial acknowledgment, periodic status update, resolution and all-clear, and a press statement for inbound media inquiries, each with bracketed placeholders and a “facts only, no speculation” instruction built into the template itself so the constraint travels with the document rather than living only in someone’s memory. Store the library alongside the rest of the incident response plan; Appendix D of this handbook is exactly this set of templates, meant to be one click away rather than buried in a wiki nobody can find at 2 a.m.

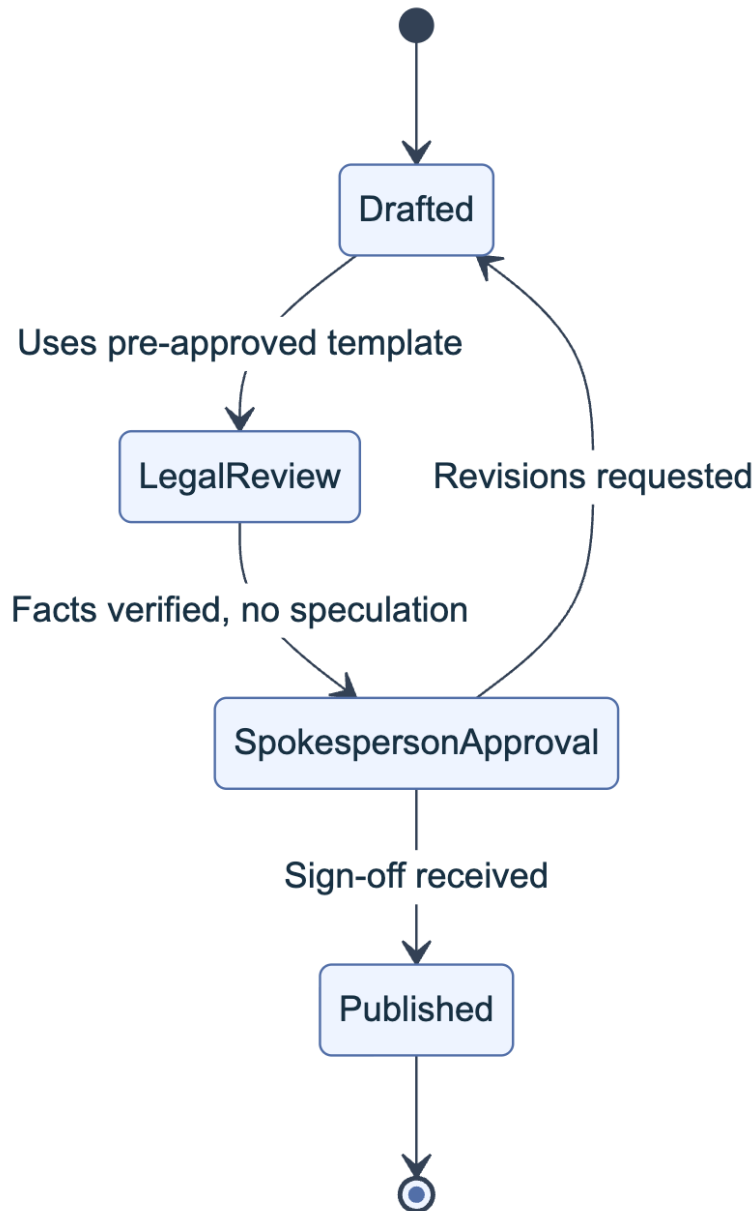


Figure 8. Message approval workflow. Starting from a pre-approved template shortens the loop between drafting and publication without skipping legal or spokesperson sign-off.

4.3.2 Message Consistency and Timing; Fixed-Schedule Updates

Every channel should mirror one source-of-truth document rather than being independently authored, because small paraphrasing differences across Discord, X, and a blog post read to an outside observer as contradictions, which fuels exactly the speculation the update was meant to prevent. Commit publicly to a fixed cadence during an active Sev1, for example “next update within two hours or sooner if material,” and honor it even when the update is “no material change.” A missed expected update reads as bad news whether or not anything actually changed. This mirrors NIST’s RC.CO-03 and RC.CO-04 elements, which call for recovery progress and public updates to be shared through approved methods and messaging on a defined cadence

rather than opportunistically (NIST SP 800-61r3). Trade speed for accuracy as the stakes rise: the first acknowledgment should go out fast, but a statement carrying legal or financial weight, such as a confirmed loss figure, should wait for verification.

Severity	Cadence during active incident	Cadence during recovery
Sev1	Every 1 to 2 hours, or immediately on material change	Every 4 to 6 hours until resolved
Sev2	Every 4 hours	Daily
Sev3	Daily	On resolution
Sev4	On resolution only	Not applicable

4.3.3 Designated Spokesperson; Avoid Speculation, Provide Factual Information

Name exactly one authorized external voice per incident, with a tightly scoped one- or two-deep backup, documented in the plan in advance rather than improvised in the moment. Every other employee, including the engineer who found the bug, routes inbound press and community questions to that person; state this explicitly during onboarding, because a well-meaning engineer answering a Discord direct message with a guess becomes the quote a reporter runs with. Content discipline matters as much as channel discipline: report only confirmed facts, label anything unverified as unverified, never speculate publicly about root cause or attacker identity before confirmation, and never commit to a recovery timeline the team does not control. Attribution errors made under pressure, naming the wrong address or the wrong root cause, have forced public retractions in past incidents that did more reputational damage than a slower, correct statement would have.

Do	Don't
State only confirmed facts, label estimates as estimates	Speculate on root cause before verification
Say "we are investigating X"	Guess at attacker identity or motive publicly
Commit to a concrete next-update time	Promise a fund-return timeline you don't control
Route all press and community questions to the spokesperson	Let engineers or support staff answer press messages individually
Correct errors quickly and visibly	Quietly edit a prior public statement without a changelog note

4.3.4 Secure Communication Channels (Encrypted Messaging)

The encryption properties chosen for internal coordination in Section 4.1.1 need to be applied consistently and deliberately, particularly around disappearing messages. Ephemeral messaging reduces the blast radius if the incident channel is itself later compromised, but it also destroys the audit trail that regulators, insurers, and forensic investigators will request afterward. Disable auto-delete for the duration of an active investigation and re-enable it only once the litigation hold or regulator inquiry window has closed; Part III's evidence-collection guidance and Part V's lessons-learned process both depend on that trail existing.

Signal implements the Signal Protocol with sealed-sender metadata minimization and is the common default for war-room coordination ([Signal Protocol documentation](#)). Wire adds enterprise features, including guest rooms for coordinating with external partners outside the internal roster. Matrix and its Element client offer a self-hosted option for teams, including DAOs, that want to own their server rather than trust a vendor to not become a subpoena target ([Matrix end-to-end encryption guide](#)). PGP or GPG-encrypted email remains the fallback for regulator or law enforcement contacts who do not use messaging apps. Provisioning discipline matters regardless of tool: create the group in advance, add responders by role, and verify safety numbers or key fingerprints out of band at least once.

Tool	Protocol	Disappearing messages	Best for
Signal	Signal Protocol, E2EE, sealed sender	Configurable	Core war-room coordination
Wire	Proteus/MLS, E2EE	Configurable, guest rooms	Cross-organization coordination with partners
Matrix / Element	Olm/Megolm, E2EE	Configurable	Self-hosted, DAO-owned infrastructure
PGP / GPG email	Asymmetric encryption, no forward secrecy	Not applicable	Async contact with regulators or law enforcement without messaging apps

Key controls for Part 2

- Provision an out-of-band, encrypted internal incident channel in advance, scoped by role, independent of the primary workspace and identity provider.
- Build an escalation policy that accepts monitoring-bot pages directly and auto-escalates on a missed acknowledgment, with a named Incident Commander for every Sev1/Sev2.
- Keep a pre-built contact sheet for regulators (SEC, FinCEN, CISA), law enforcement (IC3, Europol EC3), and OFAC obligations, mapped to jurisdiction and deadline.
- Notify integrated partners (oracles, bridge counterparties, exchanges, custodians, auditors) privately and before the public timeline.
- Maintain pre-approved, legally reviewed templates for acknowledgment, status update, resolution, and press, and commit to a fixed update cadence during active incidents.
- Name one designated spokesperson per incident and route every external inquiry through that person; state confirmed facts only.
- Disable disappearing messages on incident channels for the duration of the investigation to preserve the evidentiary trail.

Part III: Detection and Response

This part is the operational core of the handbook: the mechanics of noticing that something is wrong, deciding how wrong it is, and acting before the loss compounds. It works through the detection sources a Web3 team should wire up before launch, the response procedure that turns an alert into a contained and documented incident, and the preparation work that determines whether all of this runs smoothly or falls apart under pressure. Everything here follows the incident response lifecycle defined in the National Institute of Standards and Technology (NIST) [Special Publication \(SP\) 800-61 Revision 3](#), *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile* (April 2025), which folds incident response into the six functions of the [NIST Cybersecurity Framework \(CSF\) 2.0](#): Govern, Identify, Protect, Detect, Respond, and Recover.

5. Incident Detection

Detection is the function that starts the clock. A control that would have prevented a loss is worthless if nobody notices the loss happening, and in Web3 the interval between exploit and drained treasury is frequently measured in single-digit minutes, not the hours or days a traditional security operations center (SOC) can tolerate.

5.1 Key Components of Detection

A working detection program has four components, and a gap in any one of them creates a blind spot the others cannot compensate for. **Sources** are the systems that emit signal: chain state, application logs, infrastructure metrics, and human reports. **Collection** is the pipeline that pulls signal from those sources into a place you can query it, whether that is a managed platform or a self-hosted indexer. **Analysis** is the logic, rule-based or anomaly-based, that turns raw signal into a judgment call: is this normal, or does it warrant a human? **Alerting** is the mechanism that gets that judgment in front of a responder fast enough to matter.

Component	Web3-specific instance	Failure mode if missing
Sources	Contract events, mempool, remote procedure call (RPC) node metrics, front-end error logs, Discord/Telegram reports	Attack proceeds unobserved until funds are gone
Collection	Forta scan node, indexer (The Graph, custom listener), log aggregator	Signal exists but nobody is collecting it
Analysis	Threshold rules, anomaly detection, simulation-based bot logic	Noise drowns signal, real alert missed among false positives
Alerting	PagerDuty/Opsgenie escalation, Discord webhook, SMS	Analysis fires correctly but nobody is paged

Treat these four as a single pipeline you test end to end, not four independent purchases. A monitoring vendor contract does not constitute a detection program until you have proven, with a drill, that a real signal reaches a real human within your target window.

5.2 On-Chain Monitoring and Logging (Large Transfers, Minting, Ownership Changes)

On-chain monitoring watches the one data source an attacker cannot suppress: the chain itself. Every state-changing action a protocol takes, and every action an attacker takes against it, is a transaction with a permanent, publicly queryable record. The job of on-chain monitoring is to turn that record into alerts before the block finalizes rather than after a post-mortem.

Forta Network is the most widely deployed example of this pattern: independent detection bots run against a live mempool and block stream, evaluate each transaction against a rule, and publish alerts to a public network other consumers can subscribe to. **OpenZeppelin Defender's** Monitor module and **Tenderly Alerts** provide the equivalent capability scoped to your own contracts, with a UI for defining conditions without writing a bot from scratch.

The signal classes every protocol should monitor, regardless of platform:

- **Large transfers:** any `Transfer` event moving more than a configured percentage of a token's circulating supply, or more than a configured USD value, in a single transaction or a tight cluster of transactions from related addresses.
- **Minting and burning:** any call to a `mint()` or equivalent function outside an allowlisted caller, or any mint whose amount deviates from the protocol's normal issuance schedule.
- **Ownership and role changes:** `OwnershipTransferred`, `RoleGranted`, `RoleRevoked`, and any write to a storage slot holding an admin, owner, or upgrade-authority address.
- **Upgrade and configuration events:** proxy implementation changes, timelock queue and execute calls, and parameter changes on economically sensitive values (fees, collateral ratios, oracle addresses).

```

// Minimal viem-based watcher for the signal classes above.
// Production deployments should run this as a Forta bot or
// a Defender Sentinel so alerting survives a single host failing.
import { createPublicClient, websocket, parseAbiItem, formatUnits } from "viem";
import { mainnet } from "viem/chains";

const client = createPublicClient({ chain: mainnet, transport:
  websocket(process.env.WSS_RPC) });
const LARGE_TRANSFER_THRESHOLD = 500_000n * 10n ** 18n; // 500k tokens, 18 decimals

client.watchEvent({
  address: process.env.TOKEN_ADDRESS,
  event:
    parseAbiItem("event Transfer(address indexed from, address indexed to, uint256
    value)"),
  onLogs: (logs) => {
    for (const log of logs) {
      if (log.args.value >= LARGE_TRANSFER_THRESHOLD) {
        alert("large-transfer", {
          tx: log.transactionHash,
          amount: formatUnits(log.args.value, 18),
          from: log.args.from,
          to: log.args.to,
        });
      }
    }
  },
});

client.watchEvent({
  address: process.env.TOKEN_ADDRESS,
  event: parseAbiItem("event OwnershipTransferred(address indexed previousOwner,
    address indexed newOwner)"),
  onLogs: (logs) => logs.forEach((l) => alert("ownership-change", { tx: l.transaction-
    Hash, ...l.args })),
});

```

Every alert must carry the transaction hash, block number, and involved addresses so a responder can pivot straight to a block explorer or forensic tool without re-deriving context (see Section 6.3 on evidence collection).

5.3 Monitoring Objectives, Thresholds, and Alert Channels (Email, SMS, Messaging)

A threshold that is never tuned is a threshold that either floods the on-call responder with noise or stays silent through a real event. Set thresholds against a stated objective, not a round number picked at design time: “detect any single withdrawal exceeding 2% of protocol total value locked (TVL) within one block” is testable and revisable; “watch for big withdrawals” is not.

Route each severity to a channel matched to its urgency. A Critical alert that lands only in a Slack channel with notifications muted is functionally undetected. NIST SP 800-61r3 frames this as part of the Respond function’s communication planning, and the practical Web3 mapping looks like this:

Severity	Example trigger	Channel	Target time-to-acknowledge
Critical	Large unexplained outflow, admin key used from unrecognized address	Phone call/ SMS via PagerDuty, dedicated Signal war room	< 5 minutes
High	Oracle price deviation beyond bound, repeated failed admin auth attempts	SMS + push notification	< 15 minutes
Medium	Anomalous but plausible transaction pattern, infra latency spike	Team messaging (Discord/ Telegram/ Slack) with @here	< 1 hour
Low	Informational deviation, non-actionable community report	Ticket queue, reviewed at standup	Next business day

Redundancy matters as much as routing. A single alerting provider outage should never be a single point of failure for Critical severity: pair a primary paging service with an independent secondary channel (a second provider, or a phone tree) so a provider-side incident does not become your incident.

5.4 Infrastructure and Application Monitoring

On-chain signal tells you what happened to the contracts; infrastructure and application monitoring tells you what happened to everything the contracts depend on. RPC node health, front-end deployment integrity, API latency and error rates, and cloud account activity are all attack surfaces in their own right, and several of the largest incidents in the space started off-chain: a compromised front end serving a malicious swap target, a leaked application programming interface (API) key, or a cloud console takeover, well before any suspicious transaction appeared.

Standard site reliability engineering (SRE) tooling applies directly. Run Prometheus or an equivalent time-series system against node and service metrics, alert on deviations from baseline, and centralize logs (application, load balancer, cloud audit trail) in a single searchable store so a responder is not chasing five dashboards mid-incident.

```
# Prometheus alerting rule: RPC node falling behind chain head
# is an early indicator of eclipse attacks, provider outages,
# or a node silently forked off the canonical chain.
groups:
- name: rpc-health
  rules:
  - alert: RPCBlockLag
    expr: (time() - rpc_latest_block_timestamp) > 60
    for: 2m
    labels:
      severity: high
    annotations:
      summary: "RPC node {{ $labels.instance }} is more than 60s behind chain head"
  - alert: CloudConsoleAnomalousLogin
    expr: increase(cloudtrail_console_login_failure_total[5m]) > 5
    labels:
      severity: critical
    annotations:
      summary: "Repeated failed console logins on {{ $labels.account }}"
```

Cloud audit trails (AWS CloudTrail, Google Cloud Audit Logs) and web application firewall (WAF) logs deserve the same alerting discipline as on-chain signal, because a front-end compromise or a leaked deploy credential is frequently the precondition for an on-chain exploit, not a parallel unrelated event. The CDN and Front-End Supply Chain Security Handbook (01) covers the specific integrity controls (Subresource Integrity, Content Security Policy) that reduce this surface; this handbook's job is making sure a failure of those controls gets detected.

5.5 User and Community Reporting

Automated monitoring catches what you thought to look for. Users and community members regularly catch what you did not, and several major incidents were first reported by an ordinary user noticing a failed transaction before any internal alert fired. The [Ronin Bridge hack of March 2022](#) went undetected by Sky Mavis for six days and surfaced only because a user reported being unable to withdraw 5,000 ETH from the bridge, a gap the team's own monitoring never closed.

Build an intake path that does not depend on a user finding the right person to direct-message. A dedicated `#security` or `#bug-report` channel with a pinned reporting template, a `security.txt` file per [RFC 9116](#) at your domain root, and a bug bounty program on a platform like [Immunefi](#) each give a different class of reporter (community member, security researcher, competitor protocol) a clear, low-friction path in. Triage every inbound report against the same severity matrix used for automated alerts (Section 6.2) rather than a separate informal process, because a credible report of an in-progress exploit is functionally a Critical alert regardless of its source.

Reporting channel	Best suited for	Response commitment
<code>security.txt</code> / dedicated email	Independent researchers, coordinated disclosure	Acknowledge within 24 hours
Bug bounty platform (Immunefi, HackenProof)	Pre-exploit vulnerability disclosure	Per published program service-level agreement (SLA)

Reporting channel	Best suited for	Response commitment
Public Discord/Telegram report	Community members observing live anomalies	Triage within minutes during business hours, on-call otherwise
Direct message to known team member	Ad hoc, unreliable	Redirect to a tracked channel; never treat as the sole intake

5.6 Automated Alert Rules and Thresholds

Manual review does not scale to Web3's operating tempo, so the detection program's real output is a maintained library of automated rules, each with an owner, a documented rationale, and a review date. A rule with no owner decays: thresholds calibrated for a protocol at \$2 million TVL become useless noise, or worse, a blind spot, once the same protocol holds \$200 million.

Structure each rule with four fields so it survives a team handoff:

```
rule_id: LARGE-WITHDRAWAL-001
description: >
  Single withdrawal exceeding 2% of current vault TVL, or three
  withdrawals from related addresses exceeding 5% combined within
  one block.
severity: critical
owner: on-call-security
last_reviewed: 2026-06-01
action: page primary + secondary on-call, post to #incident-active, snapshot mempool
```

Two disciplines keep the rule library useful rather than a growing pile of stale conditions. First, review thresholds against current protocol state on a fixed cadence (monthly for TVL-relative rules, immediately after any parameter change that shifts what “normal” looks like). Second, track false-positive rate per rule and retire or retune anything crying wolf often enough that responders start ignoring its channel, since alert fatigue is itself a detection failure.

6. Incident Response Procedures

Response begins the moment detection hands off a confirmed or suspected incident to a human decision-maker, and everything from that moment forward follows a defined procedure rather than improvisation. NIST SP 800-61r3 groups this work under the Respond and Recover functions of CSF 2.0; the sequence below is that model adapted to a Web3 team where actions taken on-chain are irreversible and every minute of delay compounds loss.

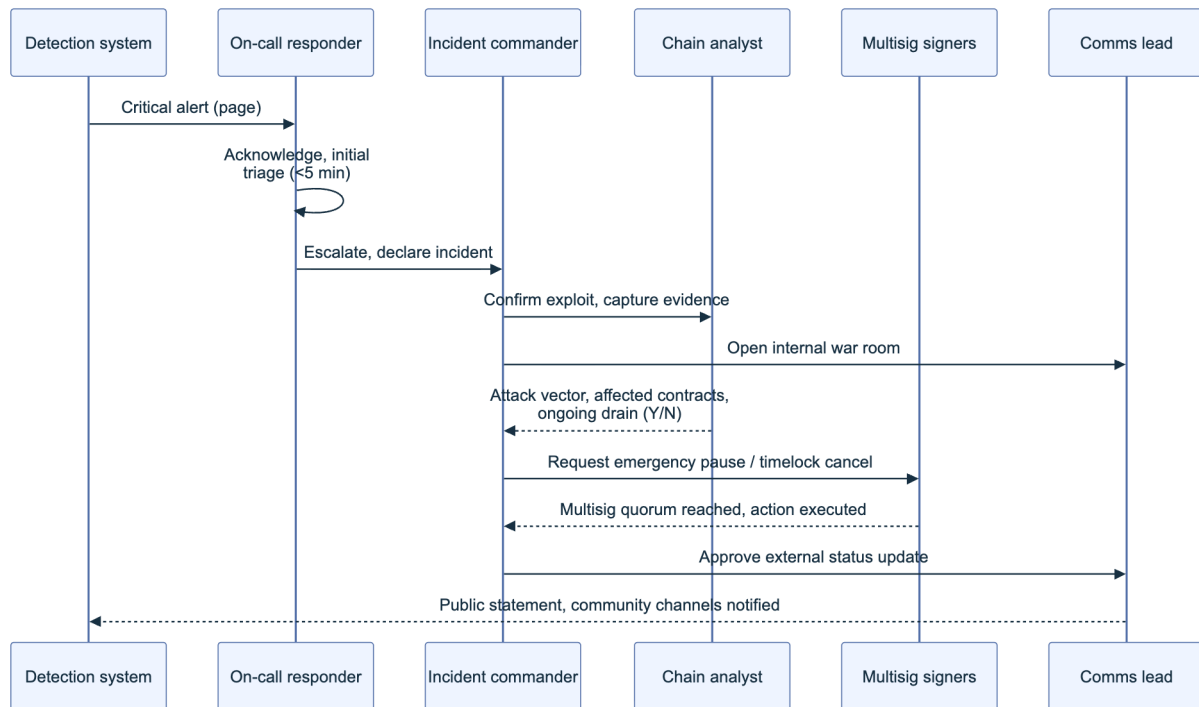


Figure 1. First-hour incident response sequence. Every arrow is a decision point with an owner; the diagram exists so no step depends on someone improvising who is responsible.

6.1 Key Components of Response

A response procedure has the same shape as detection: defined roles, a defined decision process, and defined actions, each rehearsed before they are needed. Part I of this handbook assigns the roles (incident commander, chain analyst, communications lead, and the rest of the team structure); this chapter assumes those roles exist and walks through what each does once an incident is declared.

Five activities recur across every incident regardless of type, and they are largely sequential though earlier ones sometimes continue in parallel with later ones: triage (classify severity and scope), evidence collection (capture state before it changes), containment (stop the bleeding), eradication and recovery (remove the root cause and restore normal operation), and documentation (maintain a timeline throughout, not only after). The remaining sections of this chapter take each in turn.

Activity	Primary owner	Can run in parallel with
Triage	On-call responder, then incident commander	Nothing (gates everything else)
Evidence collection	Chain analyst	Containment (must start before, continue during)
Containment	Incident commander + multisig signers	Documentation
Eradication and recovery	Engineering lead	Communications

Activity	Primary owner	Can run in parallel with
Documentation	Incident commander or scribe	All of the above, continuously

6.2 Triage and Severity Classification

Triage answers two questions fast: how bad is this, and who needs to know right now. Both answers can be wrong in the first five minutes and should be revised as better information arrives, but the initial classification determines which escalation path fires, so err toward overclassifying an ambiguous signal rather than under-reacting to it.

Ground severity in concrete, observable criteria rather than gut feel, following the same discipline [NIST SP 800-61r3](#) recommends for functional impact scoring, adapted to Web3-specific loss modes:

Severity	Criteria	Example
Critical	Active, ongoing fund loss; admin key compromise; contract logic exploited with funds still at risk	Ongoing drain via reentrancy; leaked deployer private key used on-chain
High	Exploit confirmed but contained or one-time; no ongoing drain; significant but bounded loss	Single flash-loan exploit already completed, funds moved to attacker address
Medium	Vulnerability confirmed exploitable but not yet exploited, or minor loss with no user fund impact	Researcher-reported bug with a working PoC, not yet seen in the wild
Low	Cosmetic, informational, or theoretical issue with no practical exploit path	Gas inefficiency, non-security-relevant event log gap

The full severity and triage matrix, including CVSS-adjacent scoring guidance for cases that need finer granularity than four tiers, lives in Appendix B of this handbook. During live triage, resolve ambiguity toward the higher tier: a suspected Critical that turns out to be High cost you an unnecessary page; a suspected Medium that turns out to be Critical cost you the response window.

6.3 Evidence Collection (Before Changing State)

Every state-changing action you take, whether pausing a contract, revoking a role, or migrating funds, potentially destroys evidence that would otherwise explain how the exploit worked, who else might be affected, and whether recovery is even possible. [NIST SP 800-86](#), *Guide to Integrating Forensic Techniques into Incident Response*, states the governing principle plainly: capture volatile and time-sensitive evidence before it is lost, and do so in a way that preserves its integrity for later analysis. Web3 adds a wrinkle the 2006 guide could not anticipate: some of the most relevant evidence, an unconfirmed transaction sitting in the mempool, exists for only seconds before it either confirms or disappears.

Capture, in this order, before taking any containment action that changes state:

```
# 1. Freeze the exact block and transaction context
cast tx <TX_HASH> --rpc-url $RPC_URL > evidence/tx-$(date +%s).json
cast block <BLOCK_NUMBER> --rpc-url $RPC_URL > evidence/block-$(date +%s).json

# 2. Capture full call trace, not just the top-level call
cast run <TX_HASH> --rpc-url $RPC_URL --trace > evidence/trace-$(date +%s).txt

# 3. Snapshot current contract state relevant to the incident
cast storage <CONTRACT_ADDR> --rpc-url $RPC_URL > evidence/storage-$(date +%s).txt

# 4. If the attack is ongoing, snapshot the mempool before it clears
cast rpc eth_getBlockByNumber pending true --rpc-url $RPC_URL > evidence/mempool-$(date +%s).json

# 5. Record wall-clock time and block number together, immediately
echo "$(date -u --iso-8601=seconds) block=$(cast block-number --rpc-url $RPC_URL)" >>
evidence/timeline.log
```

Write every captured artifact to the write-once evidence store described in Section 7.4, and record a SHA-256 hash of each file alongside it so later analysis can prove nothing was altered after capture. This is not bureaucracy for its own sake: the eventual root-cause writeup, any law-enforcement referral, and any insurance or recovery claim all depend on evidence whose chain of custody is intact.

6.4 Containment Options (On-Chain and Off-Chain)

Containment stops an incident from getting worse while eradication (Section 6.5) fixes the underlying cause. The two containment domains, on-chain and off-chain, have different mechanics, different speeds, and different points of no return, and a responder needs to know which lever is available before an incident forces the choice.

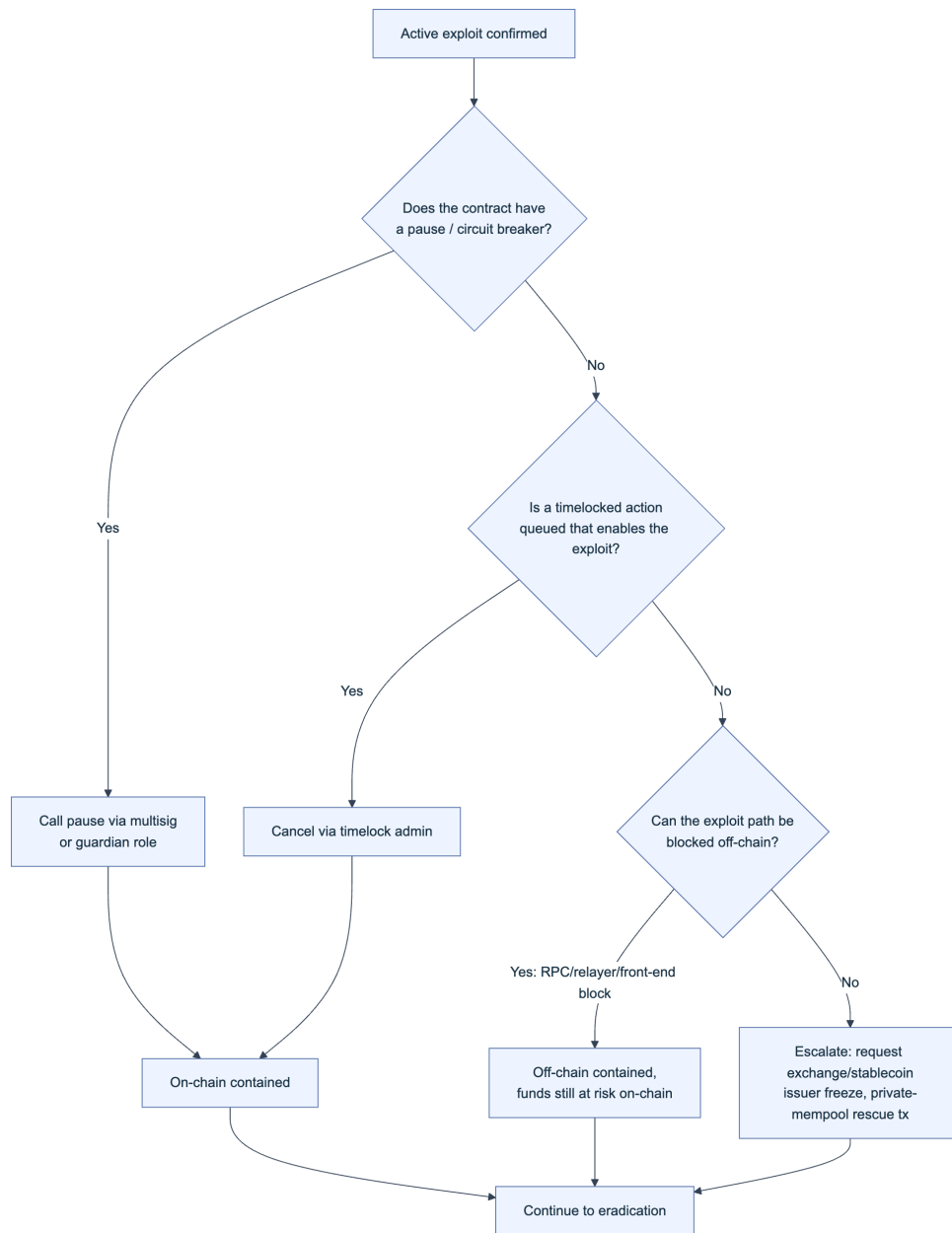


Figure 2. Containment decision tree. On-chain containment is preferred when available because it is enforced by the protocol itself rather than by cooperation from a third party.

On-chain containment options include invoking a `pause()` function built on OpenZeppelin’s `Pausable` pattern, canceling a queued timelock transaction before it executes, revoking a compromised role via access control, and routing a rescue transaction through a private mempool such as **Flashbots Protect** so it is not front-run by copycat exploiters watching the public mempool, the exact failure mode that turned the **Nomad Bridge hack of August 2, 2022** from a single exploit into a \$190 million free-for-all once other actors copy-pasted the original attacker’s calldata from Etherscan.

Off-chain containment includes pulling a compromised front end from the CDN, revoking leaked API keys and cloud credentials, blocking malicious contract addresses at the RPC or wallet-provider level, and requesting that a centralized stablecoin issuer freeze funds held at a known attacker address, a lever that has recovered funds in past incidents when the attacker moved stolen assets into a token with issuer-level freeze capability. Off-chain containment cannot stop an ongoing on-chain drain by itself, but it closes secondary damage paths (further phishing via the compromised front end, further use of a leaked key) while the on-chain path is being worked.

6.5 Eradication and Recovery

Eradication removes the root cause; recovery restores the system to trusted, normal operation. Confusing the two, patching quickly and reopening before the actual cause is understood, has caused repeat exploits against the same protocol within hours in real incidents. Eradication is not complete until you can state, with evidence, exactly why the exploit worked and confirm the fix addresses that cause rather than a symptom of it.

A practical eradication and recovery sequence:

1. **Confirm root cause with evidence**, not assumption, using the trace and state captured in Section 6.3. Do not proceed on a guess.
2. **Patch and independently review the fix**. A second engineer who did not write the fix reviews it against the confirmed root cause before it goes anywhere near mainnet.
3. **Test the fix against the exact exploit path**, ideally as a fork test that reproduces the original attack transaction and confirms it now reverts or behaves safely.
4. **Redeploy or upgrade** through the same governance and multisig process used for any privileged change, resisting the urge to skip review steps under incident pressure; a rushed emergency deploy is a common source of a second, self-inflicted incident.
5. **Recover funds where possible**: negotiate with the attacker (see the Safe Harbor and whitehat recovery playbook in Part IV), pursue a rescue transaction for at-risk but not-yet-drained funds, or coordinate with exchanges and issuers to freeze and eventually return stolen assets.
6. **Reopen gradually**, with tightened monitoring thresholds (Section 5.6) and, where feasible, a temporary transaction size cap, rather than immediately restoring full unrestricted operation.

Recovery outcomes vary enormously by incident. The [Euler Finance hack of March 13, 2023](#) ended with the attacker returning nearly all of the roughly \$197 million taken after direct on-chain negotiation, concluding within about two weeks. The [Poly Network hack of August 2021](#), at the time the largest crypto hack on record at roughly \$611 million, saw the attacker return the great majority of funds voluntarily within days after public identification pressure. Neither outcome was guaranteed, and Section 15 of Part IV covers the negotiation and safe-harbor mechanics that make voluntary return more likely.

6.6 Documentation and Timeline Maintenance

Documentation is not the report you write after the incident closes. It is a living timeline you maintain from the first alert, because memory of exact sequencing and exact timestamps degrades within hours, and a defensible post-mortem, a law-enforcement referral, or an insurance claim all depend on a record built during the incident rather than reconstructed from memory afterward.

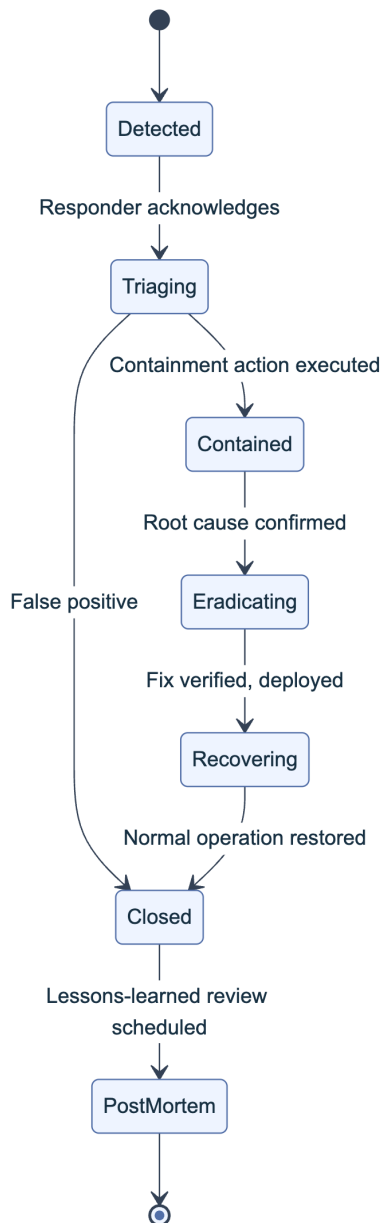


Figure 3. Incident status states. Every transition is a timestamped log entry with an owner; feeding directly into the timeline required for the post-mortem covered in Part V.

Assign a scribe role, separate from the incident commander, whose only job during the incident is logging each state transition, each decision, and each action taken, with a timestamp and an owner, to the incident channel or a dedicated log. Each entry should answer four questions: what

happened, when (both wall-clock and block number, per Section 6.3), who acted, and what evidence supports it. This log becomes the primary source for the lessons-learned process in Part V, for any regulatory or law-enforcement engagement, and for the after-action communication the community and any affected users are owed.

7. Preparation Checklist

Everything in Chapters 5 and 6 assumes infrastructure and agreements that exist before an incident starts. This chapter is that prerequisite list, meant to be worked through and signed off well ahead of any live event, ideally as part of onboarding for a new protocol or a quarterly readiness review.

7.1 Asset Inventory (Code, Infra, Keys)

You cannot protect, monitor, or contain what you have not inventoried. Maintain a single authoritative asset inventory covering every deployed contract address (with chain, version, and verification status), every piece of infrastructure (RPC providers, cloud accounts, domains, CDN configuration), and every privileged key (multisig signers and their contact methods, deployer keys, API keys, and what each one can do). Update it on every deployment and every personnel change, and treat a stale inventory as itself a finding in the readiness review.

Asset class	Fields to track	Review trigger
Contracts	Address, chain, deployer, admin/owner, upgrade authority, source verification link	Every deployment or upgrade
Infrastructure	Provider, account owner, access list, region, failover plan	Quarterly, or on provider change
Keys and credentials	Holder, scope of authority, storage method (hardware wallet, HSM, secrets manager), rotation date	Every personnel change, every 90 days minimum
Third-party dependencies	Oracle feeds, bridges, integrated protocols, upstream libraries	On any upstream incident disclosure

7.2 Log Pipeline and Reliable Clock

A response timeline is only as trustworthy as the clock it is built on, and correlating an off-chain log entry with an on-chain block requires both to share a reliable time reference. Synchronize every host that generates logs to Network Time Protocol (NTP), and record both wall-clock timestamp and block number together at every capture point, exactly as shown in the evidence collection commands in Section 6.3. [NIST's draft SP 800-92 Revision 1, Guide to Computer Security Log Management](#) (initial public draft, October 2023), frames log management as a full lifecycle: generation, transmission, storage, access, and disposal, and that lifecycle discipline is what turns scattered application logs into a usable pipeline during an incident rather than a frantic grep across a dozen hosts after the fact.

Centralize logs into a single searchable store before an incident, not during one. A pipeline that ships application logs, infrastructure metrics, and on-chain event logs into one queryable system, with retention long enough to cover your bug bounty program’s disclosure window, is the difference between a chain analyst answering “when did this start” in minutes versus hours.

7.3 Secure Comms Channels

An incident is exactly the wrong time to discover that your team’s usual chat tool is compromised, rate-limited, or being read by the same attacker who just breached your infrastructure. Pre-establish an out-of-band channel, separate from day-to-day tooling, reserved for incident coordination: an encrypted messaging platform such as Signal, with a dedicated “war room” group pre-populated with every incident-response role holder and their verified phone number, not just a username that could belong to anyone.

Pair the channel with pre-agreed verification steps for high-stakes actions. Any request to move funds, revoke a key, or issue a public statement during an incident should be verifiable through a second channel or a known callback number, because social engineering during the chaos of a live incident, impersonating a teammate to authorize an emergency action, is a documented attacker technique in its own right. The Security Alliance’s **SEAL 911** hotline, a 24/7 emergency response service staffed by volunteer Web3 security responders, is worth pre-registering with before an incident, not discovering mid-exploit; keep its contact path in your comms runbook alongside your internal war room.

7.4 Evidence Bucket (Write-Once Storage)

Evidence captured under Section 6.3 is only as good as its integrity guarantee. Store it in write-once-read-many (WORM) storage so that neither an attacker who has gained partial access nor a well-meaning but panicked teammate can alter or delete it after the fact. Cloud object storage with object-lock or legal-hold features provides this cheaply:

```
# Example: create a WORM-compliant evidence bucket (AWS S3 Object Lock, {.after-
printable-checklist}
# compliance mode, 90-day minimum retention) ahead of any incident.
aws s3api create-bucket --bucket incident-evidence-prod --region us-east-1
aws s3api put-object-lock-configuration \
  --bucket incident-evidence-prod \
  --object-lock-configuration '{
    "ObjectLockEnabled": "Enabled",
    "Rule": { "DefaultRetention": { "Mode": "COMPLIANCE", "Days": 90 } }
  }'
```

Restrict write access to the automation that captures evidence and to the chain analyst role, and restrict deletion entirely for the retention period, even from account administrators, which is what “compliance mode” enforces over the weaker “governance mode” available on the same feature. Set retention to at least the length of your longest plausible investigation, insurance claim,

or legal process, and document the bucket's existence and access procedure in the runbook referenced from Section 7.3's war room channel, so a first responder is not searching for it mid-incident.

7.5 Automated Alert Rules

Section 5.6 covers the discipline of maintaining alert rules; this entry is the pre-incident checklist confirming that discipline is actually in place before you need it. Before go-live, and again at every quarterly review, verify: every Critical and High severity scenario identified in your threat model (see Handbook 06 Part I) has a corresponding automated rule; every rule has a named owner and a last-reviewed date within the last quarter; every rule has been tested against a synthetic event, not merely deployed and assumed to work; and every rule's alert routes to a channel that has itself been tested for delivery, since a webhook silently failing is functionally identical to no rule existing at all.

Run this verification as a checklist item in the drill described in Section 7.6, because a tabletop exercise that assumes alerting works, rather than proving it, tests the team's response process while leaving the actual detection gap undiscovered.

7.6 Drill Schedule and Tabletop Exercises

A response procedure that has only ever been read, never rehearsed, will fail in ways a walk-through never predicted: a signer whose hardware wallet is in another city, a runbook link that 404s, a severity classification the team disagrees on in the moment. Tabletop exercises (TTX) surface these gaps cheaply, before an actual incident makes them expensive.

Run drills on a fixed cadence and vary the scenario so the team is not simply memorizing one script:

Drill type	Frequency	Focus
Tabletop exercise (discussion-based)	Quarterly	Walk a written scenario end to end verbally; test decision-making and role clarity, not tooling
Technical drill (live-fire)	Semi-annually	Execute real pause/timelock-cancel calls on a testnet fork; test that tooling and access actually work
Alert delivery test	Monthly	Trigger each Critical-severity rule synthetically; confirm the page arrives within SLA

Drill type	Frequency	Focus
Full incident simulation	Annually	Unannounced scenario injected by a designated red-teamer; test the whole pipeline from detection through documentation

Rotate scenario types across drills: a smart contract exploit, a compromised signer, a front-end supply-chain compromise, and an insider threat each exercise different parts of the procedure and different playbooks in Part IV. After every drill, feed findings into the same lessons-learned process (Part V) used for real incidents, closing the loop between preparation and continuous improvement rather than treating drills as a compliance checkbox.

Key controls for Part III

- Wire up on-chain, infrastructure, and community reporting as one detection pipeline, tested end to end, not three independent tools.
- Route every alert severity to a channel matched to its urgency, with a redundant path for Critical.
- Capture evidence (transaction, trace, storage, mempool, timestamped against block number) before any containment action changes state.
- Prefer on-chain containment (pause, timelock cancel) over off-chain measures when the contract supports it; use a private mempool for rescue transactions.
- Maintain a live, timestamped incident timeline from first alert through closure, owned by a scribe separate from the incident commander.
- Pre-stage a WORM evidence bucket, an out-of-band comms channel, and a tested asset inventory before you need any of them.
- Run quarterly tabletop exercises and at least one unannounced full simulation a year.

Part IV: Playbooks

This part is the operational library the rest of the handbook builds toward: one chapter per incident class, each written as a runbook a responder can open the moment an incident is declared. It assumes the roles from Part I, the communication rules from Part II, and the detection and evidence-collection mechanics from Part III (Chapters 5 through 7); what changes here is the level of altitude, from lifecycle theory down to the specific sequence of actions, checklists, and message templates a team executes under pressure. The same discipline (contain, preserve evidence, communicate, recover) repeats across every chapter because a responder mid-incident should never have to hunt across a document for the next step.

8. Playbook Overview

8.1 When to Use a Playbook

A playbook activates the moment triage (Part III, Chapter 6) confirms that an anomaly crosses the threshold defined in the severity and triage matrix (Appendix B), not before. Most alerts a monitoring stack generates are noise or near-misses: a large but legitimate withdrawal, a failed transaction from a misconfigured bot, a false-positive drainer signature. Opening a full incident playbook for every one of these burns the team's capacity for the incident that actually matters, and NIST Special Publication (SP) 800-61 Revision 3 frames this discipline explicitly, treating triage as a gate that sits between detection and the Respond function of the Cybersecurity Framework (CSF) 2.0 rather than a formality ([NIST SP 800-61r3, April 2025](#)). Once triage confirms an incident, the playbook is not optional reading, it is the default action; deviating from it requires the Incident Commander to say so explicitly and record why.

Signal at triage	Playbook to open
Confirmed unauthorized fund movement from protocol contracts	Chapter 9: Smart Contract and Protocol Hack
Multiple users reporting drained wallets after visiting a link or dApp	Chapter 10: Wallet Drainer and Phishing

Signal at triage	Playbook to open
Employee device shows signs of compromise (unexpected process, exfil traffic)	Chapter 11: Malware and Compromised Endpoints
Pattern matches known nation-state social engineering (fake job, fake call)	Chapter 12: Insider and Nation-State-Style Attacks
No single team owns the affected system or governance is distributed	Chapter 13: Decentralized Incident Response

8.2 Defining Incident Type, IOCs, Containment, and Recovery

Every playbook in this part follows the same four-field anatomy so that postmortems (Part V) and detection tuning can aggregate data across incidents instead of treating each one as a bespoke narrative. **Incident type** is a taxonomy tag, ideally mapped to the [OWASP Web3 Attack Vectors Top 15](#) codes (WA01 through WA15) so that this handbook's incidents and Handbook 10's attack-vector mapping speak the same language. **Indicators of compromise (IOCs)** are the concrete, shareable artifacts: transaction hashes, attacker addresses, malicious contract bytecode hashes, phishing domains, malware file hashes. **Containment** is the set of immediate, ideally reversible actions that stop the incident from getting worse. **Recovery** is the set of actions that restore normal operation once containment holds.

Field	Definition	Example (Chapter 10 incident)
Incident type	WA-code + short label	WA06: UI/UX Spoofing & Approval Phishing
IOCs	Addresses, hashes, domains	<code>0xAttacker...</code> , <code>evil-mint.xyz</code> , malicious <code>setApprovalForAll</code> calldata
Containment	Immediate reversible actions	Domain takedown request, wallet-provider blocklist submission
Recovery	Steps to restore normal state	User approval-revocation campaign, IOC feed to Part III detection

8.3 Best Practices for Playbook Design

A playbook that only exists as a page in a wiki hosted behind the same single sign-on the incident might have compromised is not a playbook, it is a liability. Design every playbook to be short enough to execute from memory under stress, action-first rather than explanatory, available offline or through a channel independent of primary infrastructure (a printed copy, an offline password-manager entry, a static page mirrored outside the main stack), and owned by a named role who is accountable for keeping it current. Version every playbook like code: date it, log changes, and re-run a tabletop exercise against it at a fixed cadence, since an untested playbook

is a hypothesis, not a procedure. NIST SP 800-61r3 treats preparation as continuous rather than a one-time document ([NIST SP 800-61r3](#)), and CISA's incident response guidance echoes the same point for the same reason: the plan degrades the moment it stops matching reality.

- Keep each playbook to one screen or one printed page per phase (immediate actions, parallel roles, containment, recovery).
- Store an offline, out-of-band copy that does not depend on the primary communication stack (Chapter 14 covers the backup channel).
- Assign a named owner and a review date; stale playbooks fail exactly when needed.
- Rehearse via tabletop exercise at least twice a year and after every material architecture change.
- Cross-link every playbook to the message templates in Appendix D so no one drafts a holding statement from scratch mid-incident.

9. Smart Contract and Protocol Hack

A live exploit against a smart contract or protocol is the highest-tempo incident this handbook covers: funds move in seconds, and every minute of delay is a minute an attacker has to bridge, mix, or offload stolen assets.

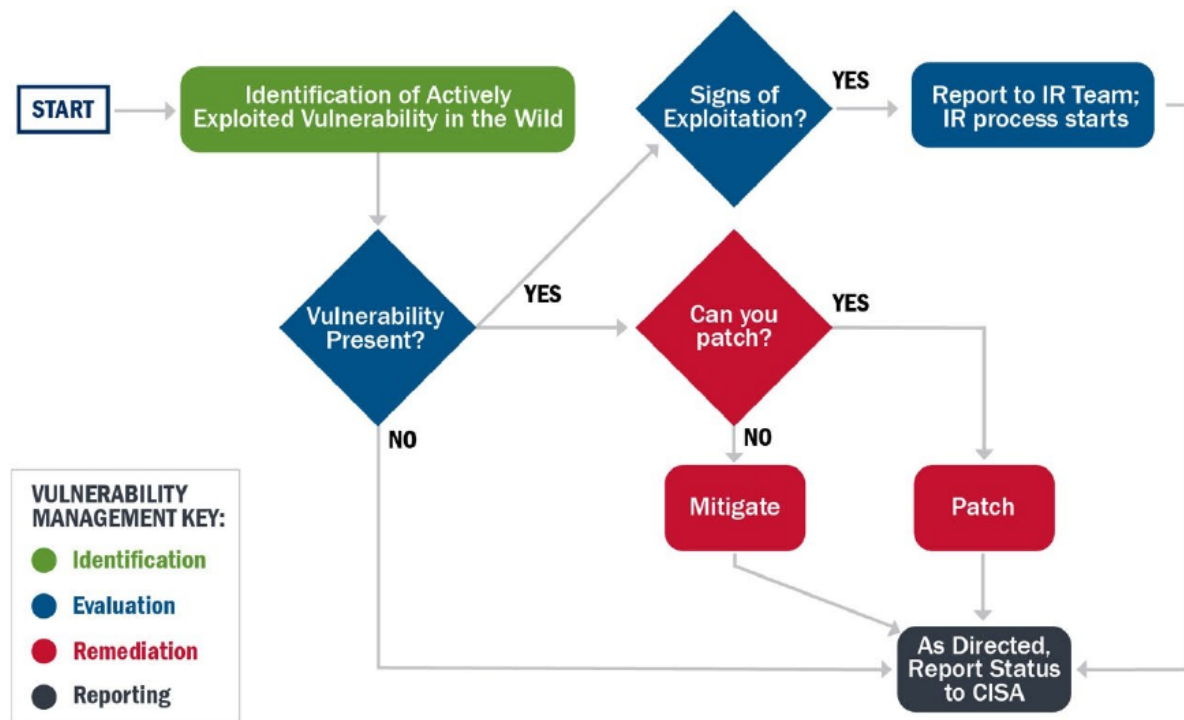


Figure 4: Vulnerability Response Phases

Figure. A vulnerability becomes an incident branch when exploitation is observed. Otherwise the decision remains exposure-driven: confirm presence, patch where possible, mitigate where it is not, and report status. For a smart contract, “can you patch?” may mean an authorized proxy upgrade; for immutable code it means containment, migration, or compensating controls instead. Source: *CISA Federal Government Cybersecurity Incident and Vulnerability Response Playbooks, Figure 4*, public domain (U.S. government work).

9.1 Immediate Actions Checklist

The first fifteen minutes decide how much of the loss is preventable. Confirm the alert against a second, independent data source before acting (a single monitoring bot's false positive should never trigger a public pause), because a wrong call here has its own cost in user trust. Then move in this order:

1. **T+0:** Confirm the exploit is real using a block explorer or simulation tool (Tenderly, Etherscan) against the flagged transaction.
2. **T+2:** Page the on-call Incident Commander and assemble the roles in Section 9.2.
3. **T+5:** Snapshot state before it disappears: the exploit block number, the full transaction trace, mempool activity, and any pending transactions from the attacker address.
4. **T+7:** If the protocol has an emergency pause and pausing does not itself trigger further loss, evaluate and execute it.
5. **T+10:** Check whether stolen funds are moving toward a centralized exchange or bridge; if so, alert that venue's compliance or security contact immediately, funds sitting in a deposit queue are the best chance at a freeze.
6. **T+15:** Decide, with Communications Lead and Legal, whether any public statement is needed yet; premature disclosure can tip off an attacker mid-exploit or invite copycats before containment holds.

9.2 Perform in Parallel by Role

Sequential handoffs cost minutes the team does not have, so once an incident is confirmed, every role below starts working simultaneously rather than waiting for a briefing to finish.

Role	Immediate parallel task
Incident Commander	Owns the decision to pause/not pause; coordinates the other roles; is the single point that resolves conflicting recommendations
Smart Contract Engineer	Simulates the exploit against a forked mainnet to confirm root cause and scope; drafts the pause or mitigation transaction
Forensics / On-Chain Analyst	Traces fund flow in real time (Nansen, Arkham, Chainalysis Reactor); flags exchange deposit addresses
Communications Lead	Drafts the holding statement; monitors social channels for user panic and misinformation
Legal / Compliance	Assesses sanctions exposure of the attacker address, notification obligations, and law-enforcement engagement criteria
Community / Support	Fields user reports; funnels new information back to Forensics without publicly confirming unverified details

9.3 Information Gathering (Issue, Timeline, Transactions, Addresses, Funds, Attacker)

Six categories of information must be captured continuously from the first minute, because this data feeds both the containment decisions made in real time and the forensics handoff to the EVM Forensics and DeFi Recovery Handbook (04) once the acute phase ends.

Category	What to capture	Typical tool
Issue	Root cause hypothesis, affected function/module	Source diff, static analysis, Tenderly simulation
Timeline	UTC timestamps for detection, each action taken, each fund movement	Append-only incident log (Section 14.3 format)
Transactions	Exploit transaction(s), any follow-on transfers, gas paid	Block explorer, Etherscan/ blockchain-specific explorer
Addresses	Attacker EOA/contract, any known-associated addresses, mixer or bridge hops	Arkham Intelligence, Chainalysis Reactor
Funds	Amount and asset stolen, current custody location, movement velocity	On-chain tracing dashboards, exchange compliance liaison
Attacker	Behavioral fingerprint (gas patterns, prior addresses, tooling), any negotiation contact	OSINT, prior incident database, SEAL ISAC intel sharing

9.4 On-Chain vs. Off-Chain Containment (Rescue Window for Non-Atomic Attacks)

Containment splits into two categories with different owners and different speed. **On-chain containment** acts on the protocol itself: pausing, upgrading a proxy, migrating liquidity, or blocklisting an address at the contract level where the code supports it. **Off-chain containment** acts on the surface around the protocol: pulling the front-end offline, revoking exposed API keys and RPC access, and asking exchanges and bridges to flag the attacker's deposit address. Neither substitutes for the other; a paused contract with a front-end still soliciting deposits, or a taken-down front-end with an unpaused vulnerable contract, both leave exposure open.

Not every exploit is a single atomic transaction, and that distinction matters for containment strategy. Some vulnerabilities are repeatable: any address that discovers the bug can trigger it independently against every remaining vulnerable pool, vault, or market. The Nomad Bridge incident of August 1, 2022 is the reference case: a misconfigured Merkle root validation treated an unproven message as automatically valid, and once the first exploit transaction was copied, the attack became a “free-for-all” that hundreds of independent addresses replicated against the remaining bridge balance before it was drained ([Nomad post-mortem coverage](#), [Rekt.news](#)). The Curve Finance incident of July 30, 2023 shows the defensive mirror image: a compiler-level reentrancy-guard bug in specific Vyper releases (0.2.15, 0.2.16, and 0.3.0) left several stablecoin pools exploitable, and independent MEV searchers, notably the address publicly associated with the handle `c0ffeebabe.eth`, raced ahead of remaining attackers to drain the still-vulnerable pools into safety and later returned the rescued funds ([Vyper security advisory](#)).

This is the **rescue window**: the interval between when a repeatable exploit is discovered and when every reachable instance of it has either been drained by an attacker or secured by a defender. Racing a rescue transaction through the public mempool exposes it to front-running by the same class of actor it is trying to outrun, so rescue transactions are typically submitted through a private relay such as [Flashbots Protect](#) that shields them from the public mempool until included.

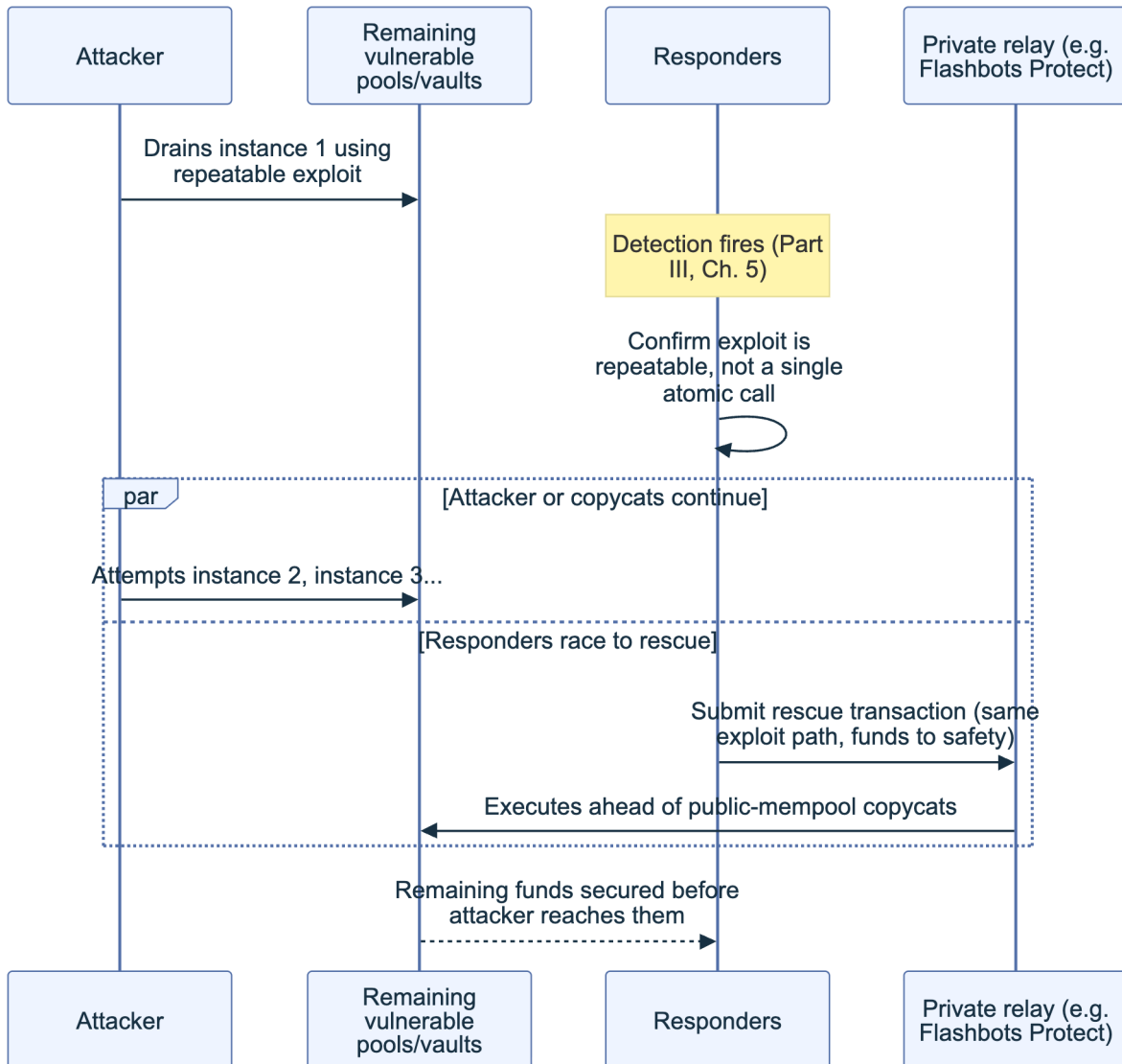


Figure 1. The rescue window for a non-atomic, repeatable exploit. Containment is a race, not a single button press, and the winner is whoever reaches each remaining instance first.

9.5 Post-Incident Actions

Once fund movement stops and containment holds, the incident shifts from acute response to stabilization. Hand the fund trace to a dedicated forensics workstream (Handbook 04 covers the methodology in full) so real-time responders can stand down. Develop and independently audit any patch before redeployment, no matter how time-pressured the situation feels; a rushed, unaudited fix that introduces a second bug is a documented failure mode in this space. If governance approval is required to redeploy or to authorize a treasury-funded user compensation plan, start that process immediately rather than after the technical fix is ready. Publish an initial public statement within the first hours (a holding statement, not a full analysis) and commit to a fuller post-mortem timeline, typically within one to three weeks once forensics and legal review

are complete. Assess regulatory notification obligations with Legal in parallel; several jurisdictions treat a material asset loss event as a reportable incident regardless of the protocol's legal structure.

9.6 Appendix: Key Roles, Tools, Message Templates

Category	Item
Roles	Incident Commander, Smart Contract Engineer, Forensics/On-Chain Analyst, Communications Lead, Legal/Compliance, Community/Support
On-chain tooling	Block explorer (Etherscan or chain equivalent), Tenderly (simulation and alerting), Chainalysis Reactor / Arkham Intelligence / Nansen (tracing)
Containment tooling	Multisig (Safe), Flashbots Protect (private rescue transactions), status page provider
Coordination	SEAL 911 (24/7 cross-protocol incident response) and SEAL's ISAC intelligence sharing (securityalliance.org)
Communication	Verified social account, status page, incident channel (Chapter 14)

****Initial holding statement template****

We are aware of unusual activity affecting `[protocol name]` and are actively investigating. As a precaution we have [paused the affected contract / disabled the affected front-end feature]. No further user action is required at this time. We will post a substantive update within `[X hours]` at `[status page URL]`. Do not interact with any address or link claiming to offer a "fix" or "refund"; we will never ask you to sign a transaction to recover funds.

10. Wallet Drainer and Phishing Incidents

Wallet drainers are turnkey kits, some sold as drainer-as-a-service, that construct malicious approval or signature requests designed to look like routine dApp interactions. The OWASP Web3 Attack Vectors Top 15 catalogs this as WA04 (Drainer Malware & Drainer-as-a-Service) and its companion WA06 (UI/UX Spoofing & Approval Phishing) ([OWASP Web3 Attack Vectors Top 15](#)); research from anti-scam trackers such as ScamSniffer has documented drainer kits including Inferno Drainer and Angel Drainer responsible for hundreds of millions of dollars in cumulative losses across 2023 and 2024.

10.1 Detection and Triage

Signals include a sudden spike in `approve`, `permit`, or `setApprovalForAll` transactions from many distinct users toward a single new contract, direct user reports of drained assets, alerts from wallet-integrated scam detection (Blockaid, ScamSniffer), or a spike in traffic to a newly registered look-alike domain. Triage classifies scope before response, because the correct action differs sharply by tier.

Scope	Trigger	Response tier
Single user	One report, no shared vector identified	Support-level guidance, log as data point
Mass campaign	Multiple unrelated users, same malicious contract/domain	Full playbook: takedown, blocklist, public notice
Official-channel compromise	Malicious link posted from a verified project account	Full playbook plus immediate account lockdown (Section 10.2)

10.2 Containment (Secure Remaining Assets)

Move on two fronts at once: stopping new victims and limiting damage to victims already exposed. File an abuse report with the domain registrar and submit the phishing URL to Google Safe Browsing, PhishFort, and Chainabuse to accelerate blocklisting across browsers and wallets. Coordinate with wallet providers (MetaMask, Phantom, and others expose blocklist submission channels) to flag the malicious contract address directly in-wallet. If the vector was a compromised official account, lock it down immediately: rotate credentials, enforce hardware-backed multi-factor authentication, and notify the platform's trust-and-safety team, since a compromised verified account is itself an active attack surface until reclaimed. For affected users, point them at an approval-revocation tool; checking exposure is a single read call:

```
# Example: check ERC-20 allowance a user granted to a suspected drainer contract
cast call $TOKEN_ADDRESS "allowance(address,address)(uint256)" $USER_ADDRESS $DRAIN-
ER_CONTRACT --rpc-url $RPC_URL
```

Direct users to a revocation interface (such as Revoke.cash) rather than asking them to run raw transactions themselves during an active incident.

10.3 User and Community Notification

Speed and accuracy trade off against each other here, and getting the balance wrong in either direction has real cost: too slow, and more users interact with the malicious link; too fast with unverified detail, and the team damages its own credibility or spreads panic. The notification should tell users exactly what happened, exactly what to do now (a direct link to the revocation flow, not just “check your approvals”), and exactly what not to do (never interact with a link

claiming to help, even to “test” it, since secondary phishing that impersonates the incident response itself is a documented follow-on pattern). Point users only to previously verified official channels and warn explicitly that DMs offering “manual recovery” are never legitimate.

10.4 Recovery and Follow-Up

Track which wallets were affected and offer direct support through the revocation process rather than a generic FAQ link. Monitor for delayed secondary drains: some drainer kits batch-sign multiple approvals during the initial interaction and execute the actual transfer later, so a user who “revoked in time” may still be exposed to a stale signature. Confirm takedown completion with the registrar and blocklist providers rather than assuming the initial report succeeded, and feed the IOCs (malicious contract address, phishing domain, wallet-drainer signature) into the detection rules maintained under Part III and into shared intelligence channels such as Chainabuse and SEALs ISAC so the next team facing the same kit starts from a known baseline.

11. Malware and Compromised Endpoints

A compromised employee or contributor device is a different tempo of incident than a live protocol exploit: the acute risk is usually key or session exfiltration rather than an in-progress drain, but the lateral-movement risk is often wider than it first appears.

11.1 Secure Crypto Assets First

The moment device compromise is suspected, before further diagnosis, move any funds accessible from that device (hot wallet keys, exchange API keys with withdrawal rights, treasury signer access) to a clean, verified-uncompromised destination. Every minute of delay is a minute malware documented in campaigns such as AppleJeuS (Lazarus Group’s cryptocurrency-targeting malware family, first catalogued by Kaspersky in 2018 and covered in [CISA advisory AA21-048A](#)) has to exfiltrate a seed phrase, hijack a clipboard-pasted address, or auto-sign a pending transaction. Treat any signature or approval made on that device during the suspected compromise window as attacker-influenced by default; verify each one rather than assuming it was legitimate.

1. Move hot-wallet funds to a hardware wallet or address generated on a known-clean device.
2. Revoke any API keys with withdrawal or transfer permissions issued to that device.
3. Flag every transaction signed on the device during the suspected window for independent review.
4. Only then proceed to diagnosis and forensic imaging.

11.2 Notify Colleagues and Affected Parties

Assume lateral exposure rather than waiting for forensics to confirm it. If the device had access to shared credentials, shared drives, or bot tokens for team communication platforms, notify every potentially exposed colleague immediately with the device identifier, the suspected compromise window, and any known indicators, so they can check their own exposure in parallel rather than in sequence. Silence during forensic triage is a common and costly mistake: the cost of a false alarm to a colleague is minutes, the cost of a missed lateral compromise is another full incident.

11.3 Secure Accounts and Credentials

Rotate every credential the compromised device had access to, prioritized by blast radius.

Credential type	Action	Priority
Password manager master credential	Rotate from a separate, known-clean device; force re-encryption	Immediate
SSH / deploy keys	Revoke and reissue; audit recent usage logs	Immediate
Cloud console / IaC access	Revoke active sessions; rotate API tokens	Immediate
Multi-factor authentication enrollment	Re-enroll on new hardware; prefer FIDO2/passkey over SMS or app-based OTP	High
SaaS session tokens (chat, ticketing, VCS)	Force logout across all sessions; rotate stored tokens	High

11.4 Notification of Authorities Where Appropriate

Engage law enforcement when theft exceeds a materiality threshold set in advance, when the pattern matches known nation-state tooling (Chapter 12 covers the coordination path in more depth for that case), or when the incident meets local mandatory-reporting criteria. In the United States, the FBI's Internet Crime Complaint Center (IC3) is the standard intake point for cryptocurrency theft (ic3.gov); most jurisdictions maintain an equivalent national cybercrime unit or computer emergency response team (CERT). Loop in Legal before making contact if there is any possibility the compromise involves a sanctioned actor, since that changes both the reporting obligation and what the organization may safely do next.

11.5 Device Replacement and Rebuild

Do not clean a compromised device and return it to sensitive use. For any role with treasury, signing, or production access, wipe and reimage from verified-clean media or replace the hardware outright; a device that hosted key-logging or memory-scraping malware cannot be trusted to be fully clean by inspection alone. Reinstall software only from sources with verifiable

checksums or signatures, re-enroll the rebuilt device in endpoint detection and response (EDR) tooling before it touches anything sensitive, and restore data only from backups confirmed to predate the compromise window.

11.6 Ongoing Vigilance

Compromise of one device is frequently a reconnaissance step rather than the whole campaign, particularly where nation-state tooling is involved (Chapter 12). Maintain heightened monitoring after rebuild: watch for the same malware family's indicators reappearing elsewhere in the fleet, unusual outbound connections from the rebuilt device, and repeat credential-audit sweeps at a shortened interval for at least one full cycle. Treat the incident as the trigger to accelerate any pending rollout of phishing-resistant, hardware-backed multi-factor authentication (FIDO2 passkeys) for high-privilege roles, since credential rotation alone does not close the gap that let the initial compromise succeed.

12. Insider and Nation-State-Style Attacks (e.g., DPRK Vectors)

State-linked actors, most visibly groups attributed to the Democratic People's Republic of Korea (DPRK) and tracked under the Lazarus Group umbrella, operate with patience and resourcing that generic malware playbooks do not anticipate. MITRE's [ATT&CK](#) knowledge base and its digital-asset-specific counterpart [AADAPT](#) (Adversarial Actions in Digital Asset Payment Technologies) both catalog these actors' tactics, techniques, and procedures, and this handbook maps directly to WA05 (Fake Interview & Video Call Social Engineering), WA12 (Insider Threats & Collusive Abuse), and WA15 (Nation-State Infiltration via Fake Hiring & Malicious OSS Contributions) in the [OWASP Web3 Attack Vectors Top 15](#).

12.1 Recognizing Attack Patterns (Fake Software, PDFs, Conferences)

The most damaging documented DPRK-attributed intrusion in this space is the Ronin Bridge hack of March 2022: an employee at Sky Mavis, the studio behind Axie Infinity, was targeted through a fake job offer that included a malicious PDF; opening it compromised the device and ultimately allowed the attacker to obtain enough validator signatures to authorize a fraudulent withdrawal of approximately \$625 million, an attack the FBI publicly attributed to the Lazarus Group ([FBI statement on Ronin/Lazarus attribution, April 2022](#)). A related pattern, documented by Palo Alto Networks' Unit 42 as the "Contagious Interview" campaign, has DPRK-linked actors pose as recruiters and induce software engineers to install a malicious npm package or a trojanized video-call application as part of a fake technical interview. A separate and increasingly common vector is direct infiltration: DPRK operatives using fabricated identities to obtain remote developer roles. Security awareness vendor KnowBe4 publicly disclosed in 2024 that it had unknowingly hired

such an individual, who received a company laptop that immediately began loading malware, and a joint US government advisory details the broader scheme ([FBI/CISA/State Department advisory on DPRK IT workers](#)).

Pattern	Delivery vector	Red flags	Case reference
Fake job offer	Malicious PDF or coding-test repository	Urgency, requests to open attachments outside normal channels	Ronin Bridge, March 2022
Fake interview / trojanized call software	Malicious npm package or fake video-conferencing app during a “technical interview”	Non-standard software required to join a call or complete a test	Unit 42 “Contagious Interview”
Fabricated remote-worker identity	Standard hiring pipeline, stolen or synthetic identity documents	Reluctance to appear on camera, laptop shipped to a freight-forward address	FBI/CISA/State DPRK IT worker advisory

12.2 Immediate Containment and Evidence Preservation

Nation-state intrusions typically have longer dwell time and more than one foothold, so a response scoped to a single device under-scopes the problem. Assume the possibility of a broader compromise until evidence says otherwise, and preserve volatile evidence before isolating a system, since isolating first can destroy exactly the evidence a forensics team trained in nation-state tactics, techniques, and procedures (TTPs) needs. [RFC 3227](#) defines the standard order of volatility for evidence collection, most perishable first:

1. Registers, cache, running process state
2. Routing table, ARP cache, kernel statistics, main memory
3. Temporary file systems
4. Disk
5. Remote logging and monitoring data relevant to the system
6. Physical configuration and network topology
7. Archival media

Engage forensics specialists with nation-state TTP experience before making the intrusion public in any channel; premature public disclosure can burn an active law-enforcement investigation before it produces attribution or recovery.

12.3 Coordination with Law Enforcement and Ecosystem

Route contact through the FBI’s IC3 and the relevant field office, CISA for infrastructure-level indicators, and Treasury’s Office of Foreign Assets Control (OFAC) when a sanctioned entity may be involved, since Lazarus-linked addresses and entities are OFAC-designated and that designation carries direct legal consequences for any organization that transacts with them, including a negotiated “bounty” payment. Ecosystem coordination runs through SEAL 911 for 24/7 cross-

protocol incident response and SEALs ISAC for intelligence sharing (securityalliance.org), and through exchange compliance teams for freezing or flagging attacker-controlled deposit addresses.

Contact	Trigger	What they need from you
FBI IC3 / field office	Confirmed theft, suspected nation-state involvement	Transaction hashes, addresses, timeline, indicators
OFAC (via counsel)	Suspected sanctioned-entity involvement	Legal review before any negotiation or payment
SEAL 911 / ISAC	Any live incident needing cross-protocol response capacity	Incident summary, IOCs, current containment state
Exchange compliance	Stolen funds moving toward a deposit address	Address, amount, transaction hash, urgency

12.4 Communication and Recovery

Public communication differs sharply from the other playbooks in this part: do not attribute an incident to a specific nation-state publicly until law enforcement or OFAC confirms it, since premature attribution can compromise an active investigation and exposes the organization to defamation risk if the attribution later proves wrong. Keep public messaging focused on concrete user-protection actions rather than attacker identity, and coordinate every external statement with Legal. Recovery in this class of incident is rarely a pure code fix; it usually exposes a process gap in hiring, offboarding, or credential lifecycle management, and the durable fix sits in the Hiring, Remote Work, and Insider Threat Handbook (05) and the Employee Lifecycle Security Handbook (03) rather than in this handbook alone.

13. Decentralized Incident Response

Some Web3 organizations have no single incident commander by design: authority is distributed across a multisig signer set, a decentralized autonomous organization (DAO), or a pseudonymous contributor community. This chapter adapts the same discipline to that structure.

13.1 Guiding Principles (Zero-Trust, Shared Responsibility, Evidence First)

Three principles anchor decentralized response. **Zero-trust identity:** no responder's claim is acted on purely because the account making it is recognized, because in a pseudonymous team a compromised "known" account issuing a fake emergency request is itself a plausible attack. **Shared responsibility:** authority should have no single point of failure for the same reason code should not, so no one signer, moderator, or contributor can unilaterally commit the whole response. **Evidence first:** every action taken during the incident is justified by evidence attached at the time

of the decision (a transaction hash, a simulation output, a signed statement), because a decentralized team typically reconstructs trust after the fact from that evidence trail rather than from an org chart.

13.2 Roles and Identities in Decentralized Teams

Decentralized structures still need the functions Part I (Chapter 3) defines, they just fill them differently.

Centralized role	Decentralized equivalent
Incident Commander	Multisig cosigner quorum acting as a body, or a rotating on-call from a pre-agreed responder pool
Verified employee identity	Reputation built from prior signed commits, ENS-linked history, and PGP or Keybase social proofs
Communications Lead	Distributed community moderators operating under a pre-agreed messaging protocol
Legal/Compliance	Retained counsel engaged by the foundation or steward entity, if one exists

13.3 Preparation Checklist for Distributed Response

- Publish an emergency contact list with independently verifiable identities (signed keys, not just handles).
- Pre-agree a multisig signer set and an emergency-action threshold distinct from routine governance (Section 13.5).
- Maintain a backup communication channel that does not depend on the primary infrastructure stack (Chapter 14).
- Where technically feasible, stage or pre-sign pause transactions so execution does not require drafting under pressure.
- Join a cross-protocol responder network, such as SEALs ISAC, so an unfamiliar team is not starting from zero on IOCs or TTPs.

13.4 Detection and Triage Flow

Declaring an incident without a single on-call manager requires a quorum mechanism rather than an individual judgment call.

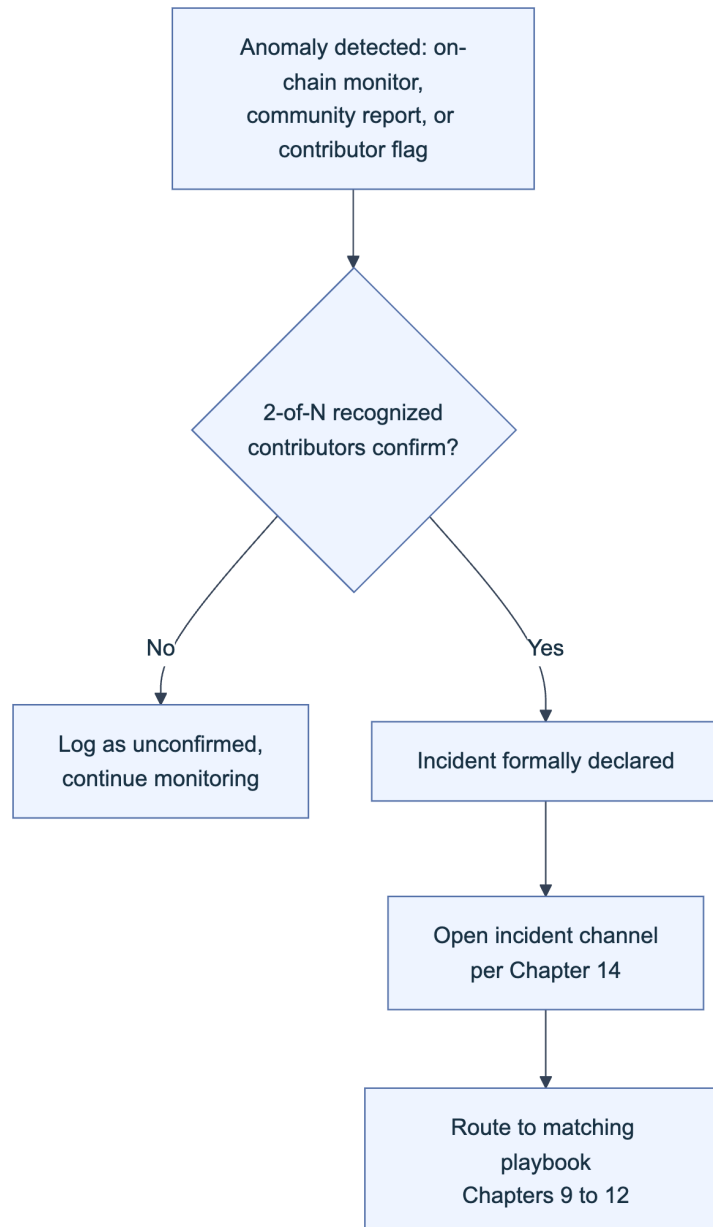


Figure 2. Decentralized detection and triage. Formal declaration requires independent confirmation from at least two recognized contributors, not a single report.

13.5 Containment Options Without Single Authority

The central tension in decentralized containment is that fast action normally needs one button, and decentralized governance intentionally removes single buttons. The resolution is tiered authority, pre-agreed before any incident. Arbitrum's Security Council illustrates the pattern in production: a defined multisig splits its threshold by action type, a lower bar for genuine

emergency pauses and a higher bar with a timelock for routine actions, so speed and safety are balanced by design rather than improvised mid-incident ([Arbitrum DAO governance documentation](#)). Where no multisig quorum is reachable in time, a designated guardian role can act unilaterally under strict conditions: the action must be publicly justified immediately, and it is subject to mandatory ratification once quorum re-forms. Where no on-chain lever exists at all, containment moves to the social layer: public warnings, delisting requests to front-ends and aggregators, and direct coordination with exchanges, none of which stops the contract itself but all of which reduce the pool of victims.

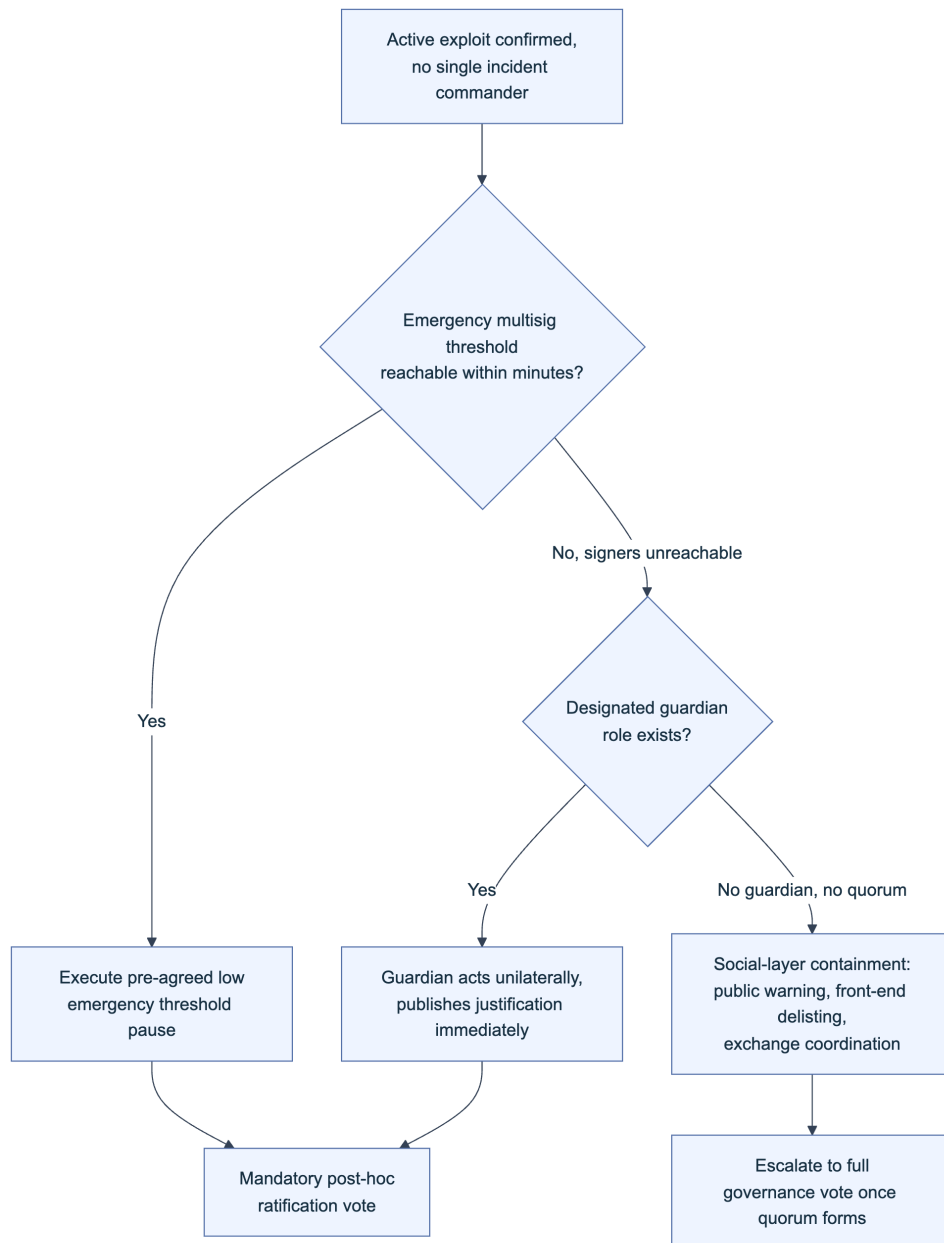


Figure 3. Containment options ranked by available authority when no single incident commander exists. Every path ends in public accountability, either immediate or through ratification.

13.6 Eradication and Recovery

Without a formal quality-assurance organization, patch review needs an explicit substitute: require sign-off from at least one independent contributor who did not write the fix, not just the original author's self-review. Stage redeployment with a public verification window before it goes fully live, and publish exactly which signer or guardian executed which recovery action, since that public record is the accountability mechanism a hierarchical org would otherwise get from an internal chain of command.

13.7 Post-Incident Actions and Quick-Start Templates

Publish the post-mortem to a canonical, censorship-resistant, permanent location (a content-addressed store such as IPFS, or a version-controlled repository) rather than relying on a single official blog that may itself have been part of the incident's attack surface. Reuse the message templates in Appendix D rather than drafting new ones mid-crisis; the quick-start value of a template is lost if it has to be written from scratch the first time it is needed.

13.8 Pros and Cons of Decentralized IR

Pros	Cons
No single point of compromise for response authority itself	Consensus takes longer exactly when speed matters most
Draws on a broader, distributed pool of technical expertise	Verifying pseudonymous identity mid-incident is inherently harder
Transparent, publicly auditable accountability trail	Legal liability for actions taken is often ambiguous
Resistant to a single compromised insider halting response	Coordination overhead across time zones and channels

14. War Room and Crisis Coordination

A war room is the synchronous coordination structure that every playbook in this part routes into once severity crosses a defined threshold; it exists so that the parallel roles in Section 9.2 (or their decentralized equivalents in Chapter 13) work from one shared source of truth instead of five disconnected threads.

14.1 When to Activate a War Room

Activate when severity reaches High or Critical on the matrix in Appendix B, when funds are actively moving, or when the incident spans more than one team and needs live coordination rather than asynchronous updates. Activation is a low-cost, reversible decision: standing a war room down once it turns out not to be needed costs a few minutes, while delaying activation on a genuine Critical incident costs the minutes containment depends on.

14.2 Template for Incident Channel and T-Minus Deadlines

Create a dedicated, clearly named incident channel (for example, `#incident-YYYYMMDD-short-name`) with a pinned message stating the Incident Commander, current status, and a link to the live timeline document. Attach a T-minus deadline table to the pin so every participant knows what update is due when, without asking.

Deadline	Action
T+15 min	First internal status update to the incident channel
T+60 min	First external holding statement (Section 9.6 template)
T+4 hours	Substantive update: scope, containment status, next milestone
T+24 hours	Detailed update or interim report
T+1 to 3 weeks	Full public post-mortem (Part V)

14.3 Information Gathering and Timeline

Maintain a single append-only timeline document for the duration of the incident; retroactively editing entries destroys the evidentiary and after-action value of the log. Every entry is timestamped in UTC and links its supporting evidence.

Timestamp (UTC)	Actor	Action	Evidence link
2026-01-01 03:12	Monitoring bot	Alert fired: large outflow from vault contract	Tx hash / dashboard link
2026-01-01 03:15	On-call engineer	Confirmed via Tenderly simulation	Simulation link
2026-01-01 03:19	Incident Commander	Declared incident, opened war room	Channel link

14.4 Handoff to Forensics and Long-Term Recovery

Stand the war room down once fund movement has stopped, containment holds, and no further real-time decisions are pending, then hand the incident to the sustained forensics and recovery workstream. Fund tracing and recovery mechanics beyond this point are the province of the EVM Forensics and DeFi Recovery Handbook (04), and the post-incident review that closes the loop belongs to Part V of this handbook.

15. Safe Harbor and Whitehat Recovery

An ethical hacker who spots an in-progress exploit faces a genuine dilemma: intervening to rescue funds may itself require executing the same exploit code, which without prior authorization looks indistinguishable from the attack it is trying to stop. Safe Harbor frameworks exist to remove that dilemma before an incident, not during one.

15.1 Pre-Authorizing Whitehats for Active Exploits

Security Alliance's Safe Harbor is the leading concrete implementation of this idea: a protocol adopts it through a governance decision made in advance, granting whitehat responders standing legal authorization to intervene in an active exploit under defined conditions, described by SEAL as legal protection for whitehat rescues during active exploits ([Security Alliance Safe Harbor](#)). The core conditions typically require that intervention be necessary to prevent loss or further loss, that recovered funds be returned to the protocol minus an agreed bounty, and that the whitehat not exploit the authorization for gain beyond those agreed terms. Pre-authorization matters because it converts a split-second legal judgment call, made mid-exploit by someone with no time to consult counsel, into a decision the protocol already made calmly, in advance.

15.2 Recovery Addresses, Bounty Terms, and Return of Funds

A workable Safe Harbor adoption needs concrete, discoverable mechanics, not aspirational language a whitehat has no time to interpret mid-incident.

Field	Typical value
Recovery address	Pre-designated multisig, published in protocol docs and repository so it is discoverable without delay
Bounty terms	Percentage of rescued funds, commonly aligned with the protocol's existing bug bounty ceiling
Return deadline	Fixed window after the exploit (commonly on the order of months) to return principal minus the agreed bounty
Scope	Explicit conditions under which the authorization applies (active exploit only, not a standing invitation to test in production)

Real recoveries show both how well this can work and how badly it fails without concrete terms. In the Poly Network incident of August 2021, an attacker who extracted roughly \$610 million voluntarily returned nearly all of it after a public, on-chain negotiation, and was later informally credited as "Mr. White Hat." In the Euler Finance incident of March 2023, an attacker who extracted approximately \$197 million through a flawed donation-to-reserves mechanism returned the funds in tranches over several weeks following sustained on-chain pressure and negotiation, with Euler ultimately honoring an agreed bounty. By contrast, the KyberSwap incident of November 2023 (roughly \$48 million) saw negotiations collapse after the attacker demanded gov-

ernance control of the DAO as a term, a demand no protocol can safely grant, and the funds were not returned. The lesson across all three: terms need to be concrete, bounded, and agreed before the incident, exactly what Safe Harbor is designed to provide.

15.3 Relationship to Bug Bounty and Responsible Disclosure

Safe Harbor and bug bounty programs cover adjacent but distinct scenarios. A bug bounty (Immunefi, HackenProof, Cantina, and similar platforms) covers the pre-exploit case: a researcher finds a vulnerability and has time to disclose it responsibly before anyone exploits it. Safe Harbor covers the active-exploit case, where there is no time for that process because someone, possibly the researcher, possibly an attacker, is already exploiting it. A well-designed program references both explicitly, so a researcher who stumbles onto a bug already being exploited in the wild knows immediately that the rescue path is pre-authorized rather than legally ambiguous. The Mango Markets exploit of October 2022 is the cautionary counter-example: the attacker publicly framed the roughly \$114 million extraction as a “legal” trading strategy and negotiated a settlement to keep a portion as a bounty, yet was subsequently criminally charged and convicted on market manipulation and fraud counts in United States federal court, a reminder that an informal, after-the-fact negotiation carries none of the legal certainty a written, pre-agreed Safe Harbor provides.

16. Vulnerability Disclosure and Bug Bounty Linkage

The incident channel and the vulnerability-disclosure channel need a clear boundary between them, because routing a not-yet-exploited bug report through incident response wastes the urgency that channel exists for, and routing a live incident through the slower disclosure triage queue costs time the response cannot afford.

16.1 When to Escalate to Disclosure and Security Contact

An incident, by definition, has already been exploited. A vulnerability report describes something that has not. When a live incident investigation surfaces a second, related but not-yet-exploited bug, escalate it out of the incident channel and into the dedicated security-contact or bug-bounty triage path rather than letting it linger inside incident response. A published `security.txt` file, per [RFC 9116](#), is the standard, machine-discoverable way to expose that contact point so a researcher, or an internal team member, always has an unambiguous place to send it.

Situation	Route to
Already exploited, funds or data affected	Incident channel (Chapters 9 to 14)
Discovered but not yet exploited, no active loss	Security contact / bug bounty triage
Discovered mid-incident, related but distinct bug	Flag to Incident Commander, then route to bug bounty triage once the acute phase ends

16.2 Post-Incident Disclosure and Communication

Once an incident resolves, publish a coordinated disclosure of the root-cause vulnerability, even when it was not originally reported through the bug bounty program, so the wider researcher and protocol community can learn from it rather than rediscover it independently. Credit any whitehat or researcher involved, with their consent, and update the bug bounty program's scope and reward structure to reflect any gap the incident revealed, whether that is an unlisted contract, an underpriced severity tier, or a vulnerability class the program had not previously considered in scope. This step closes the loop into Part V's continuous-improvement cycle: a disclosure that never updates the program that should have caught the bug first is a missed opportunity repeated at the next incident.

Key controls for Part 4

- Match every declared incident to exactly one playbook using the decision table in Section 8.1, gated by triage, not raw alert volume.
- Run the six roles in Section 9.2 in parallel, never sequentially, and capture the six information categories in Section 9.3 continuously.
- Treat non-atomic, repeatable exploits as a rescue-window race: use a private relay for rescue transactions, not the public mempool.
- Adopt Security Alliance's Safe Harbor (or an equivalent pre-authorization) before an incident, with a discoverable recovery address and bounded bounty terms.
- Pre-agree tiered containment authority (emergency multisig threshold, guardian role, social-layer fallback) for any team without a single incident commander.
- Stand up the war room from Chapter 14 the moment severity crosses High/Critical, and hand off to forensics and Part V once containment holds.

Part V: Lessons Learned and Improvement

An incident that ends at recovery and never reaches a documented root cause will recur, often against the same class of weakness that just cost the protocol money. This part closes the loop between the playbooks in Part IV and the detection and preparation work in Part III: it sets out how to run a post-incident review that produces action items instead of blame, how to turn a root cause into an updated playbook or detection rule, and how to keep drills, runbooks, and threat-intelligence relationships current enough that the next incident is faster to contain than the last one.

17. Lessons Learned

A lessons-learned process is the mechanism that converts one team's bad day into every future incident being handled better. [NIST Special Publication \(SP\) 800-61 Revision 3](#), published in April 2025 as *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*, retired the older four-phase model (preparation, detection and analysis, containment and eradication and recovery, post-incident activity) in favor of mapping incident response work across all six functions of the [NIST Cybersecurity Framework \(CSF\) 2.0](#): Govern, Identify, Protect, Detect, Respond, and Recover. Under that model, lessons learned is not a phase that happens after Recover finishes; it is the mechanism by which Recover feeds directly back into Govern and Identify, so that the next incident starts from a stronger baseline rather than the same one.

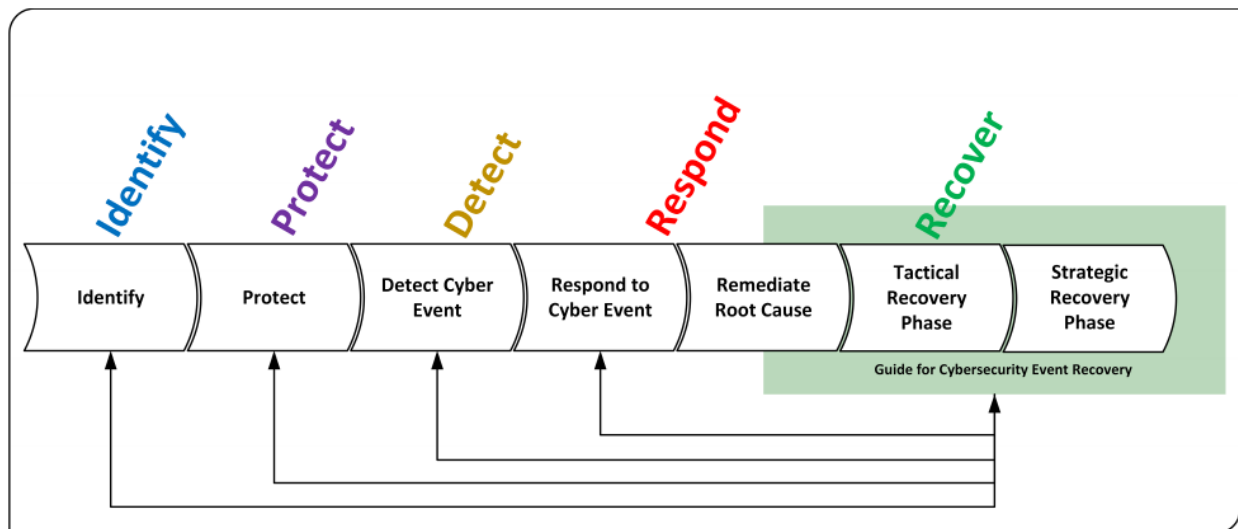


Figure 3-1: NIST SP 800-184 Guide for Cybersecurity Event Recovery Relationship with the NIST CSF

Figure. NIST separates tactical recovery—restoring the affected service—from strategic recovery, which removes the conditions that made the incident damaging and feeds improvements back into every earlier function. Declaring success after a contract is unpaused completes only the tactical half. Source: *NIST SP 800-184, Figure 3-1*, public domain (U.S. government work).

17.1 Conducting Post-Incident Reviews

A post-incident review (PIR) is a structured, scheduled reconstruction of what happened, run separately from the pressure of active response. Two timing windows matter. A short “hot wash” debrief, held within 24 to 72 hours while memory is fresh, captures raw impressions and immediate fixes. The formal review, held one to two weeks later once logs, transaction traces, and communication records are assembled, produces the document that outlives the incident. Running the formal review too soon starves it of evidence; running it too late lets the team’s memory of decision points decay.

The review must be blameless. This principle traces to Etsy’s engineering culture writing under John Allspaw, who argued that punishing the individual who made a mistake teaches everyone else to hide mistakes rather than report them, which destroys the very information a security team depends on (*Etsy, Code as Craft, “Blameless PostMortems and a Just Culture”*). *PagerDuty’s postmortem guidance* and Google’s Site Reliability Engineering practice both operationalize the same idea: the review asks “what in the system allowed this to happen,” never “who caused this.” A facilitator who was not part of the incident response should run the session, because a facilitator who was in the fire has a stake in a particular narrative.

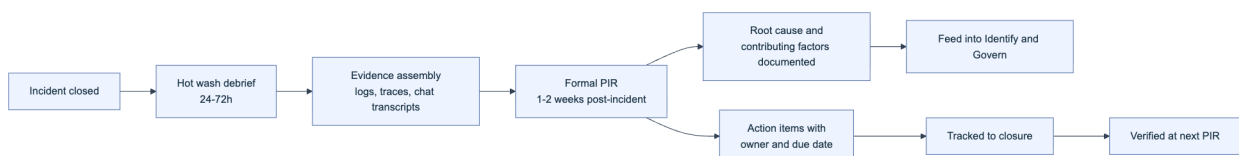


Figure 1. The post-incident review timeline. The formal review is deliberately separated from the hot wash so evidence, not adrenaline, drives the findings.

A minimum PIR agenda covers: a factual timeline built only from logs, monitoring alerts, and transaction hashes (never memory alone); a stated business and financial impact, including funds at risk versus funds lost; what detection and response did well; what slowed the team down; the root cause and contributing factors (Section 17.2); and a list of action items, each with a named owner, a due date, and a tracking ticket. Publish the review internally to everyone with a stake in the outcome, and, following the norm set by public post-mortems such as Euler Finance’s and Nomad’s after their 2023 and 2022 incidents, consider a redacted external version when the incident affected user funds. External transparency after a Web3 incident has become close to an expectation rather than an option, and a well-run PIR is what makes that publication possible without exposing operational detail.

17.2 Documenting Root Cause and Contributing Factors

A root cause is the single condition that, if removed, would have prevented the incident. A contributing factor made the incident worse, more likely, or harder to detect, but removing it alone would not have stopped the attack. Conflating the two produces reviews that fix a monitoring gap and never touch the code defect that caused the loss, or the reverse. Two techniques keep the distinction disciplined: the Five Whys, which repeatedly asks why the immediately preceding cause occurred until the chain terminates in a systemic condition rather than a person, and the Ishikawa (fishbone) diagram, which sorts candidate causes into categories such as code, process, people, and environment before the team commits to a root cause. Fault tree analysis is worth adding when a Critical incident had multiple independent conditions that all had to hold simultaneously, since a single linear “why” chain understates that kind of compound failure.

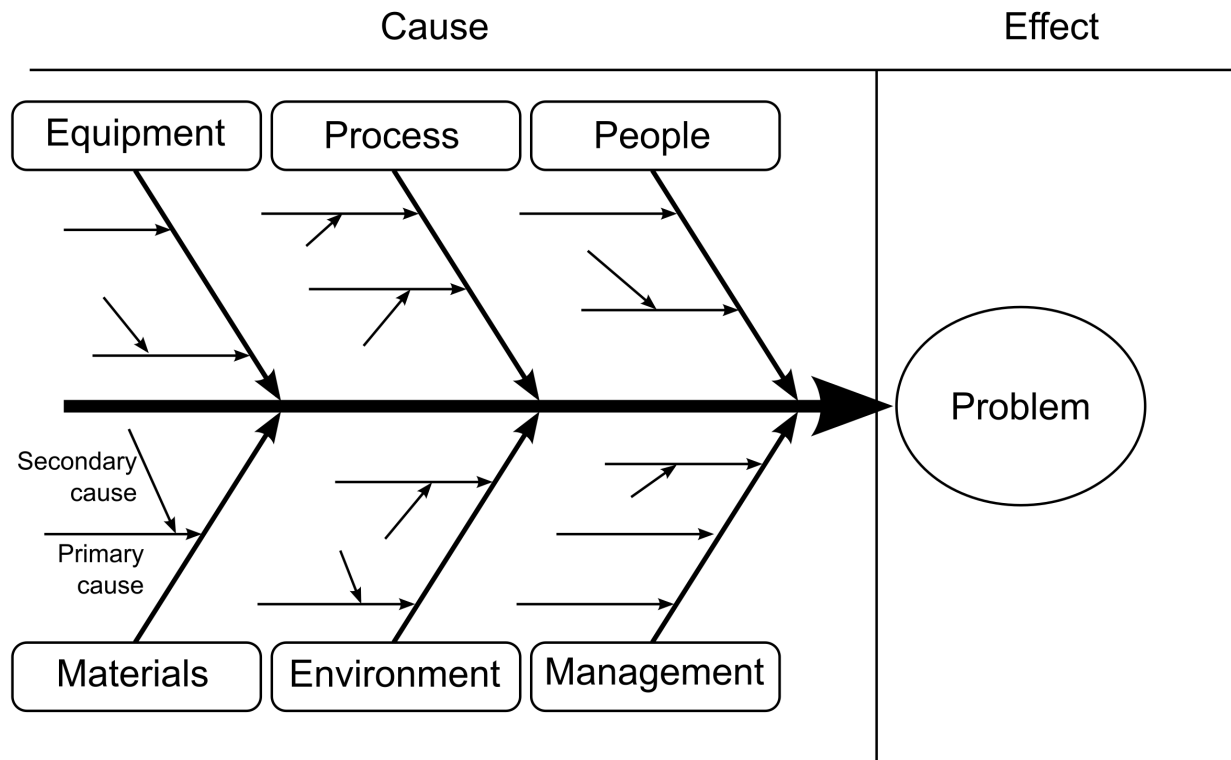


Figure. The fishbone structure this section references, forcing a reviewer to check every cause category (code, process, people, environment) before settling on a single root cause rather than stopping at the first plausible one. Source: [FabianLange](#), via [Wiki-media Commons](#), CC BY-SA 3.0.

Two real incidents show how root cause and contributing factors separate in practice. The [March 13, 2023 Euler Finance hack](#) lost roughly \$197 million to a flaw in the `donateToReserves` function introduced by an earlier upgrade (EIP-14): the function let a user donate eTokens to protocol reserves without a health check on the donor's own borrowing position, letting the attacker manufacture an unhealthy account and then self-liquidate it at a discount. The root cause was the missing health check on that single code path. Contributing factors included the change having shipped without a targeted audit pass on the new donation logic and the absence of a real-time monitor for accounts crossing into unhealthy collateralization outside a normal liquidation flow. The attacker ultimately returned nearly all recoverable funds in tranches after Euler's team combined on-chain forensics with direct on-chain messaging, with the bulk of assets back under Euler's control by early April 2023.

The [August 2, 2022 Nomad Bridge hack](#) lost about \$190 million in roughly two and a half hours after a routine contract upgrade set the zero address as a "trusted" message root, meaning any message with an unset or zero value was accepted as pre-verified by the `process()` function. The root cause was that single misconfigured trusted root. The contributing factor that turned a single exploitable bug into a \$190 million loss was the bug's trivial replayability: once the first attacker's transaction was visible in the mempool, anyone could copy its calldata and drain the bridge themselves, and Nomad's team took roughly three hours to halt the contract during an incident that unfolded openly on-chain.

Category	Question it answers	Euler (Mar 2023)	Nomad (Aug 2022)
Root cause	The single condition that, removed, prevents the incident	Missing solvency check in <code>donateToReserves</code>	Zero address accepted as a trusted message root
Contributing factor: code	Adjacent code that widened the blast radius	No isolation between donation and liquidation logic	<code>process()</code> had no additional signer or nonce check
Contributing factor: process	Governance or SDLC gap that let it ship	New donation path added without a dedicated audit pass	Upgrade shipped without a replay-safety review
Contributing factor: detection	Why it was not caught before major loss	No real-time solvency-anomaly monitor	Exploit was trivially copyable once visible on-chain

Record root cause and contributing factors as data, not prose alone, so the fields are queryable across a growing incident history: tag each finding against the **OWASP Smart Contract Weakness Enumeration (SCWE)** category it belongs to when the root cause is code-level, and against the relevant **MITRE AADAPT** tactic when it involves adversary tradecraft rather than a pure code defect. That tagging is what lets Section 18.3's threat-intelligence integration and a future PIR spot a recurring pattern across incidents that look unrelated on the surface.

17.3 Updating Playbooks and Detection Rules

A PIR that does not change an artifact has not actually closed the loop. Every root cause and every contributing factor from Section 17.2 must resolve to one of three concrete updates: a revised procedure in a Part IV playbook, a new or tuned detection rule, or, if neither applies, an explicit and documented decision to accept the residual risk. Track playbooks and detection content in version control alongside the code they protect, so a playbook change is a reviewable diff with an author, a linked PIR, and a commit message, not an edit to a static document nobody re-reads.

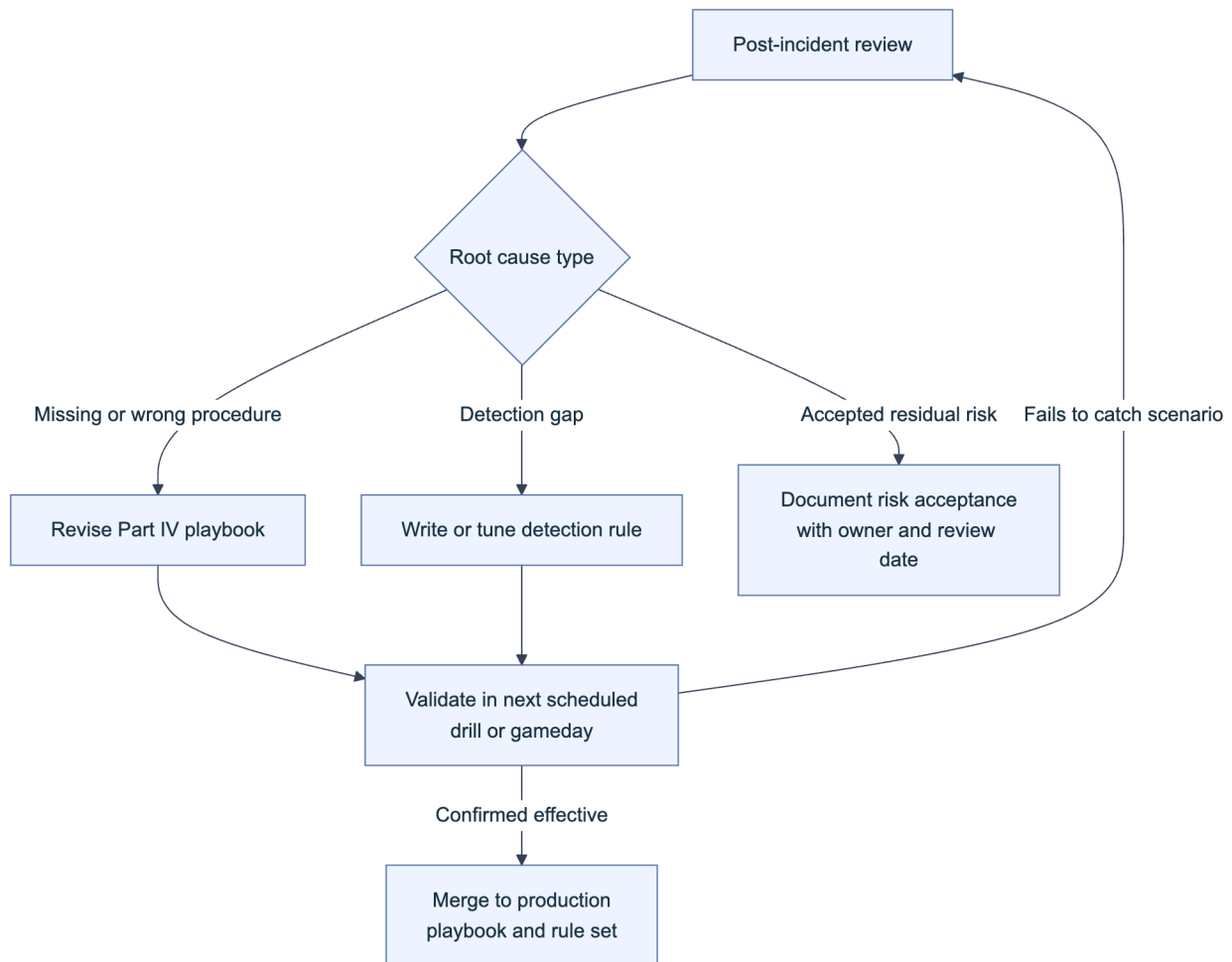


Figure 2. The feedback loop from review finding to validated control. A drill (Section 18.1) is the mechanism that proves an update actually works before the next real incident does.

For detection rules, express the logic in a portable format so it survives a tooling migration. **Sigma**, the open, YAML-based generic signature format maintained by SigmaHQ, is the widely used pattern for log and event-based detections; the same discipline of “condition plus context plus severity” applies to on-chain monitors even though most chain-monitoring platforms use their own rule syntax rather than Sigma directly. A rule written after a PIR should encode the specific precondition the incident revealed, not just the attack’s surface symptom:

```
title: Unhealthy-position self-liquidation via reserve donation path
id: 6f1a2e4c-postincident-euler-pattern
status: production
description: >
  Detects a donate-to-reserves or equivalent balance-adjusting call from an
  address followed, within the same transaction, by that address's own
  position being liquidated. Written after PIR-2023-EULER-01.
logsource:
  category: onchain_call_trace
detection:
  donate:
    function_selector: "donateToReserves"
  same_tx_liquidation:
    function_selector: "liquidate"
    liquidated_account: "%donate.caller%"
  condition: donate and same_tx_liquidation
level: critical
tags:
  - scwe.missing-precondition-check
  - aadapt.fraud
```

Route every merged rule update through the same detection pipeline covered in Part III, and record in the rule's metadata which PIR produced it. That traceability is what lets a future audit or a new team member answer “why does this rule exist” without archaeology, and it is what lets you retire a rule confidently once its underlying code path is fixed, rather than letting the detection surface grow forever with rules nobody remembers the purpose of.

17.4 Best Practices for Retrospectives

A retrospective that reliably produces useful output follows a small set of disciplined habits rather than an elaborate process. Keep it blameless in language and in structure: phrase every finding as a system condition (“the deploy pipeline allowed an unaudited change to reach production”), never as an individual failing. Timebox it, typically 60 to 90 minutes for a Medium-severity incident and up to half a day for a Critical one, because an unbounded retrospective drifts into unrelated grievances. Assign a facilitator and a scribe who are not the same person, and pick both from outside the responding team where the org is large enough to allow it. Open every retrospective by reviewing the status of action items from the previous one before discussing the new incident, since an organization that never checks whether its last fix actually shipped will keep rediscovering the same gap.

Checklist for a well-run retrospective:

- ☐ Facilitator is not a primary responder to this incident
 - ☐ Timeline built from logs and transaction data, reviewed for accuracy before the meeting
 - ☐ Impact stated in concrete terms (funds at risk, funds lost, users affected, downtime)
 - ☐ Prior retrospective’s action items reviewed for completion status first
 - ☐ Root cause and contributing factors separated per Section 17.2 and tagged (SCWE / AAD-APT)
 - ☐ Every action item has a named owner, a due date, and a tracking ticket
 - ☐ At least one action item maps to a playbook or detection rule change (Section 17.3), or the residual risk is explicitly accepted in writing
 - ☐ Decision made on whether to publish an external, redacted summary
 - ☐ Retrospective document stored with the incident record, not only in chat history
-

18. Continuous Improvement

Lessons learned repairs what a specific incident exposed. Continuous improvement is the standing program that finds gaps before an incident does, by rehearsing response under controlled conditions and keeping the operational scaffolding, runbooks, contacts, and external relationships, current between incidents rather than only after one.

18.1 Drill and Gameday Schedules

The GameDay format traces to Amazon, which began deliberately triggering major failures in production on a schedule starting in 2003 to build confidence that the systems and the people would survive the real thing, drawing explicitly on firefighter training methodology ([Gremlin, “How to Run a GameDay”](#)). Google’s internal Disaster Recovery Testing (DiRT) program followed a similar model from 2006, and Netflix’s Chaos Monkey, built in 2011 and open-sourced in 2012, automated the same idea by randomly terminating production instances to force resilient design as a default rather than an afterthought. The common thread across all three is that failure is rehearsed on a schedule the team controls, so the first time a given failure mode occurs is not during a real incident at 3 a.m.

Web3 teams need drills that cover attack classes generic chaos engineering does not, because the failure modes are financial and cryptographic, not only infrastructural. A useful schedule mixes cadence and drill type so that every major response path gets exercised at least once a quarter:

Drill type	Frequency	Participants	What it validates
Tabletop exercise (scenario walkthrough, no systems touched)	Quarterly	IR team, protocol leads, legal, comms	Decision-making and communication under a written scenario, following a CISA-style Tabletop Exercise Package (CTEP) structure
Multisig / key-ceremony rehearsal	Quarterly, and after any signer change	All active signers, security lead	Signers can actually produce a valid signature under time pressure, and the quorum is reachable
Pause-mechanism drill (testnet or fork)	Monthly	On-call engineer, IC (incident commander) rotation	Emergency pause function works, and the on-call engineer can execute it without escalation delay
Full incident simulation (injected exploit on a forked mainnet)	Semiannually	Entire IR org, including external partners (Section 18.3)	End-to-end detection, triage, containment, and communication chain from Parts III and IV
Chaos/GameDay on supporting infrastructure	Monthly	SRE / DevOps	Infrastructure resilience underneath the protocol: RPC failover, monitoring uptime, alert delivery

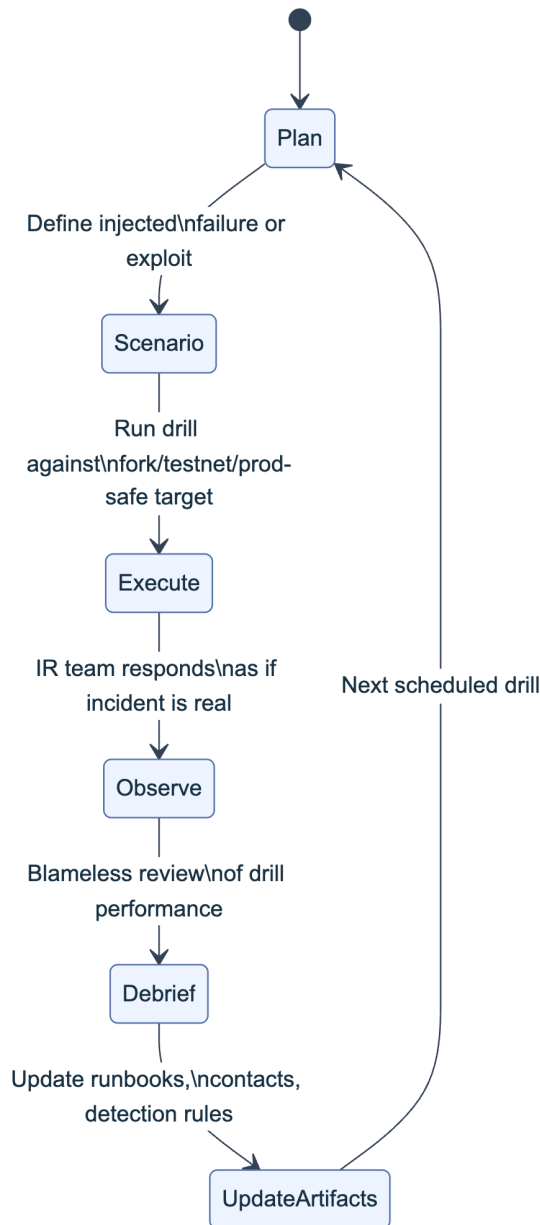


Figure 3. The gameday exercise lifecycle. A drill that never updates an artifact afterward is a rehearsal with no memory; it must feed the same update pipeline a real PIR does (Figure 2).

Every drill closes with the same blameless debrief format as Section 17.4, and every gap it surfaces gets a tracked action item exactly like a real incident's. Treat a drill finding with the same severity discipline as a live finding: a tabletop exercise that reveals the IC role is unclear during a weekend incident is not a minor process note, it is a preparation gap that will cost real response time next time.

18.2 Updating Runbooks and Contact Lists

A runbook that has not been opened since it was written is a liability disguised as preparation, because procedures, contract addresses, dashboard URLs, and personnel all drift while the document stands still. Assign every runbook an owner and a mandatory review cadence, and stamp both facts into the document itself so staleness is visible at a glance rather than discovered mid-incident:

```
runbook: smart-contract-exploit-response
owner: security-lead@protocol.example
last_reviewed: 2026-06-01
next_review_due: 2026-09-01
last_tested_in_drill: 2026-05-14
related_playbook: part4-playbooks.md#9-smart-contract-and-protocol-hack
```

Review on a fixed calendar (quarterly is typical) and, separately, immediately after any incident or drill that touched the runbook's procedure. A runbook is only proven current once it has been executed, either in a real incident or in the drills from Section 18.1; a review that only re-reads the text without exercising it will miss a stale dashboard link or a deprecated multisig address just as easily as no review at all.

Contact lists fail the same way and carry higher cost when they do, because a phone number or Telegram handle that no longer reaches a signer during an active drain is a minute-by-minute cost. Maintain the list as structured data with an explicit verification date, not a document someone updates when they remember:

Role	Primary channel	Backup channel	Escalation SLA	Last verified
Incident commander (on-call)	PagerDuty / Opsgenie rotation	Phone	5 min	2026-07-15
Multisig signer (each)	Signal	Phone	15 min	2026-07-15
Primary exchange security contact	Named vendor emergency channel	Email	30 min	2026-06-01
Blockchain forensics partner	Named account manager	Emergency hotline	30 min	2026-06-01
Legal counsel	Phone	Email	30 min	2026-06-01
Security Alliance (SEAL) 911	Telegram emergency bot	Email	Immediate, 24/7	2026-06-01

Verify every entry on the same cadence as the runbooks they support, and verify it for real: a message that confirms the channel is live and monitored, not an assumption that last quarter's number still rings.

18.3 Integration with Forensics and Threat Intelligence

An incident response program that operates only on its own history is working with a small sample size. Web3-specific threat intelligence closes that gap by pooling patterns across many protocols and many incidents, and it needs to feed both directions: your PIR findings should reach the wider ecosystem, and the ecosystem's findings should reach your detection rules before an attacker reuses a known pattern against you.

MITRE AADAPT (Adversarial Actions in Digital Asset Payment Technologies), released by MITRE in 2024 as a complement to the established **MITRE ATT&CK** framework, gives Web3 teams a shared vocabulary for classifying blockchain and digital-asset attacks by tactic, from resource development and initial access through to fraud and impact, in the same structured way ATT&CK classifies conventional intrusions. Tagging each PIR's root cause against an AADAPT tactic (Section 17.2) is what makes cross-incident pattern-matching possible: a protocol that tags three unrelated-looking incidents under the same AADAPT technique has just found a trend its own history alone would not reveal.

On-chain forensics vendors (Chainalysis, TRM Labs, and Elliptic are the three most commonly engaged in Web3 incidents) provide fund-tracing, mixer and bridge-hop analysis, and exchange-freeze coordination that a protocol's own team rarely has the tooling or law-enforcement relationships to perform alone; the EVM Forensics and DeFi Recovery Handbook (04) covers that engagement in depth. Structured threat-information sharing follows the practice NIST lays out in **SP 800-150, Guide to Cyber Threat Information Sharing**: define what you share, with whom, and under what governance before an incident forces an improvised answer, using standard exchange formats such as STIX for the content and TAXII for the transport where your sharing community supports them. In Web3 specifically, the **Security Alliance (SEAL)**, a non-profit industry body, operationalizes this through SEAL 911, a 24/7 emergency channel connecting a protocol facing an active incident to vetted external responders, alongside broader threat-intelligence coordination across member protocols.

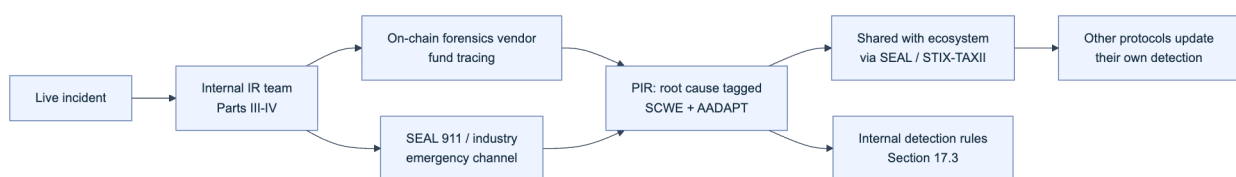


Figure 4. Threat intelligence flows in both directions: forensics and industry channels inform your review, and your tagged findings strengthen the detection baseline for everyone else.

Close the loop by reviewing, on the same quarterly cadence as the runbook review in Section 18.2, whether your detection rule set has been updated against the AADAPT techniques and SCWE categories that appeared in incidents across the wider ecosystem this quarter, not only your own. A protocol that only learns from its own incidents will always be one incident behind the ones that learn from everyone's.

Key controls for Part 5

- Run a blameless post-incident review for every incident, hot wash within 72 hours and a formal review within two weeks, with action items owned, dated, and tracked to closure.
- Separate root cause from contributing factors explicitly, tag each against SCWE and MITRE AADAPT, and record findings as structured, queryable data.
- Resolve every PIR finding into a version-controlled playbook update, a detection rule update, or a documented, owned risk acceptance, and validate the update in a scheduled drill.
- Run a mixed cadence of tabletops, key-ceremony rehearsals, pause drills, and full simulations, at minimum quarterly, and treat every drill finding with real-incident severity discipline.
- Stamp runbooks and contact lists with owner, last-reviewed date, and last-tested date, and verify contacts on the same cadence rather than assuming they still work.
- Tag incidents against MITRE AADAPT and share sanitized findings through industry channels such as SEAL 911, so detection improves ecosystem-wide, not only locally.

Part VI: Appendices

Part 1, Section 1.4 told a team already mid-incident to skip straight to the matching playbook and “the appendix templates.” This is where those templates live. This part collects the reference material a responder reaches for during an incident rather than reads once and sets aside: a glossary consolidating every acronym and role name defined across Parts 1 through 5, the full severity and triage matrix that Part 3, Section 6.2 promised but deferred, a vetted list of monitoring, forensics, and communication tooling, and a set of copy-ready message templates for the channels Part 2 defines. Appendix E closes the handbook by pointing to the complete, categorized bibliography rather than duplicating it here.

19. Appendix A: Glossary

A responder should not have to search five parts to remember what an acronym meant the first time it appeared. This glossary gathers every term and role name used in the handbook into one lookup table, grouped by the kind of work each belongs to, with a pointer back to where it was first introduced.

Lifecycle and governance

Term	Definition
IRP	Incident Response Plan: the living document naming every role, its authority, and the procedure it follows (Part 1, 3.1).
IRT	Incident Response Team: the cross-functional group with pre-assigned authority to act during an incident (Part 1, 3.1).
NIST	National Institute of Standards and Technology, the U.S. federal body publishing SP 800-61 and its companion guides that this handbook is built on.
CSF 2.0	NIST Cybersecurity Framework version 2.0, the six-function model (Govern, Identify, Protect, Detect, Respond, Recover) that SP 800-61 Rev. 3 nests incident response inside (Part 1, 2.1).

Term	Definition
SP 800-61 Rev. 3	NIST Special Publication 800-61 Revision 3, <i>Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile</i> (April 2025), the handbook's primary lifecycle reference.
TTX	Tabletop exercise: a discussion-based drill that walks a written scenario end to end without executing live actions (Part 3, 7.6).
Postmortem	The blameless, written review conducted after an incident closes, covering root cause, timeline, and remediation items (Part 5, 17).
MTTD	Mean time to detect: the average interval between an incident's start and its first confirmed detection, the metric the Ronin case study's six-day gap makes concrete (Part 1, 1.1).
MTTR	Mean time to respond or recover: the average interval between detection and confirmed containment or restoration.

Roles and identity

Term	Definition
First Reporter	Whoever, or whatever, raises the first flag: an on-chain monitor, a whitehat, or a community member (Part 1, 3.2).
Triage Lead	The first internal decision-maker who confirms a report and assigns its initial severity (Part 1, 3.3).
Comms Lead	Owns every word the organization says publicly and internally about an incident (Part 1, 3.4).
Containment Lead	Holds the technical authority to execute pause, revoke, or upgrade actions (Part 1, 3.5).
Recorder	Maintains the single authoritative, timestamped incident timeline (Part 1, 3.6).
GPG	GNU Privacy Guard, an implementation of the OpenPGP standard (RFC 4880) used to sign statements so authenticity is checkable even if official channels are compromised (Part 1, 3.7).
OIDC	OpenID Connect, an identity layer over OAuth 2.0 used to bind roles such as Triage Lead to durable, revocable single sign-on credentials (OpenID Connect Core 1.0 ; Part 1, 3.7).
Multisig	A wallet or contract requiring signatures from multiple independent keyholders before executing a transaction; the mechanism binding the Containment Lead role to a signer key such as a Safe{Wallet} owner (Part 1, 3.7).

Technical and on-chain

Term	Definition
RPC	Remote Procedure Call: the interface a client uses to query or submit transactions to a blockchain node.
TVL	Total Value Locked: the aggregate value of assets a protocol holds, used as the denominator for percentage-based alert thresholds (Part 3, 5.3).

Term	Definition
Timelock	A contract mechanism that delays a queued action by a fixed period, giving responders a window to cancel it before it executes.
Pausability / circuit breaker	A contract-level mechanism, commonly built on OpenZeppelin's Pausable , letting an authorized role halt state-changing functions during an incident (Part 3, 6.4).
Private mempool	A transaction relay, such as Flashbots Protect , that withholds a submitted transaction from the public mempool until it is mined, preventing a rescue transaction from being front-run (Part 3, 6.4).
Rescue transaction	A transaction moving at-risk but not-yet-drained funds to safety ahead of an ongoing exploit.
WORM	Write-Once-Read-Many storage: an object-storage mode, such as AWS S3 Object Lock in compliance mode, that prevents evidence from being altered or deleted for a set retention period (Part 3, 7.4).
IOC	Indicator of Compromise: an observable artifact, an address, a bytecode hash, a phishing domain, associated with an attacker and shareable so other teams can detect the same actor.
Chain of custody	The documented, unbroken record of who captured, handled, and stored a piece of evidence, required for that evidence to survive later legal or insurance scrutiny (NIST SP 800-86 ; Part 3, 6.3).

Communication, legal, and recovery

Term	Definition
DPRK	Democratic People's Republic of Korea, referenced for the state-linked threat actor patterns covered in Part 4, Chapter 12.
Safe Harbor	A pre-agreed legal and operational framework authorizing a whitehat to intervene against an active exploit without exposing themselves to liability (Part 4, 15.1).
Whitehat	A security researcher who identifies or intervenes against a vulnerability to protect funds or disclose responsibly, as distinct from a malicious attacker.
Drainer	Malicious code, typically injected into a phishing site or a compromised front end, engineered to request and execute maximal-value wallet approvals the instant a victim connects and signs.
SEAL / SEAL 911	Security Alliance, a cross-protocol Web3 security coordination body; SEAL 911 is its 24/7 emergency incident-response hotline (securityalliance.org ; Part 3, 7.3).
WA (Web3 Attack Vector)	The WA01 through WA15 classification the OWASP Web3 Attack Vectors Top 15 uses to categorize an incident once triaged (Handbook 10).
SCSVS / SCSTG / SCWE	The Smart Contract Security Verification Standard, Testing Guide, and Weakness Enumeration: the OWASP SCS Project's pre-incident standards this handbook complements (Part 1, 1.3).

20. Appendix B: Severity and Triage Matrix

Part 3, Section 6.2 introduced the four-tier severity model every triage call in this handbook runs through and promised the full, standalone version here. Copy this table into your own runbook rather than re-deriving it from memory during a live incident; the Triage Lead role (Part 1, 3.3) exists specifically to apply it under pressure, not to invent it on the spot.

Severity	Criteria	Scope of impact	Escalation	Target time-to-acknowledge
Critical	Active, ongoing fund loss; admin key or signer compromise; exploit live with funds still at risk	Protocol-wide, unbounded until contained	Page the full IRT, activate the war room (Part 4, 14)	Under 5 minutes
High	Exploit confirmed and executed, but bounded or already contained; no ongoing drain	Significant, but a realized and fixed amount	Page Triage Lead, Containment Lead, and Comms Lead	Under 15 minutes
Medium	Vulnerability confirmed exploitable (for example, a working proof of concept) but not yet exploited, or a minor loss with no user-fund impact	Isolated component, no active user harm	Notify on-call, schedule same-day review	Under 1 hour

Severity	Criteria	Scope of impact	Escalation	Target time-to-acknowledge
Low	Cosmetic, informational, or theoretical, with no practical exploit path	None	Route to ticket queue	Next business day

A four-tier model resolves which channel and which people get paged, but it does not rank two Critical findings against each other, and a live incident often produces more than one candidate at the same tier simultaneously. Part 3, Section 6.2 called this “CVSS-adjacent scoring guidance for cases that need finer granularity”; the table below borrows the base-metric structure of the **FIRST CVSS v4.0 specification** (Common Vulnerability Scoring System) without computing a full CVSS score, adapting it to on-chain incident triage rather than pre-disclosure vulnerability rating.

Dimension	0 (none)	1 (low)	2 (moderate)	3 (severe)
Exploitability	Requires a fully-trusted actor to misbehave	Requires a semi-trusted role’s key	Requires specific, narrow preconditions	Permissionless, anyone can trigger it right now
Scope	Single, isolated function	Single contract	Protocol-wide	Cross-protocol or cross-chain
Fund impact	None	Partial, bounded, quantifiable	Majority of at-risk funds	Total loss of at-risk funds
Reversibility	Fully reversible, no action taken yet	Recoverable via pause or timelock cancel	Recoverable only via negotiation or exchange freeze	Irreversible once mined

Sum the four dimension scores (0 to 12) to break ties within a severity tier: two Critical findings both score in the 9 to 12 range by definition of the tier’s criteria, and the Containment Lead works the higher composite score first when resources cannot address both simultaneously. This is a prioritization aid, not a formal CVSS calculation, and it never overrides the tier assigned by the primary matrix above.

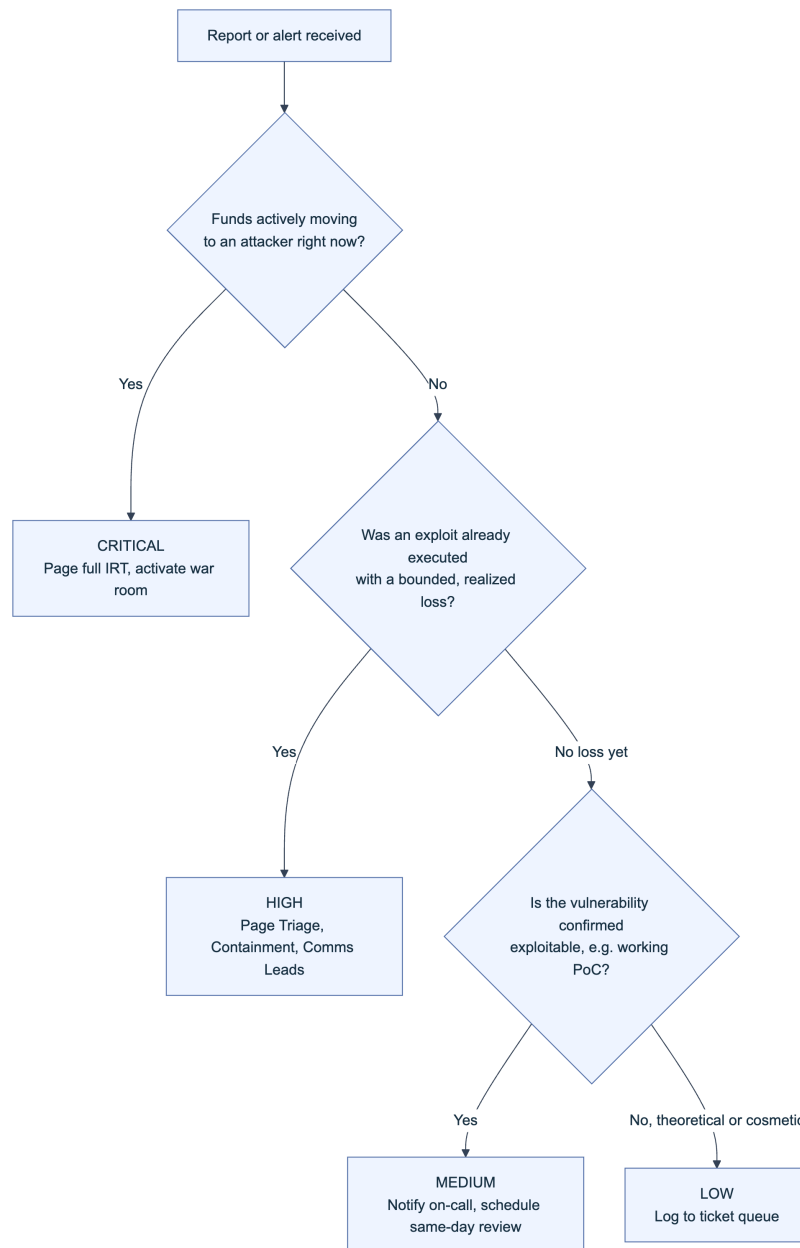


Figure 1. Triage decision tree. Each branch resolves to one severity tier from the primary matrix; the fine-grained scoring table only breaks ties among findings that land in the same tier.

A copy-ready worksheet turns this into something the Triage Lead fills out live rather than holds in their head:

```
# triage/worksheet_template.yaml
incident_id: ""
reported_at_utc: ""
reported_by: ""
first_signal_block_number: ""
funds_actively_moving: null      # true/false
loss_confirmed_bounded: null    # true/false
exploit_confirmed_reproducible: null # true/false
initial_severity: ""            # Critical/High/Medium/Low
fine_grained_scoring:
  exploitability: null          # 0-3
  scope: null                  # 0-3
  fund_impact: null             # 0-3
  reversibility: null           # 0-3
escalated_to: []
war_room_activated: null
```

21. Appendix C: Suggested Tools and Platforms

None of the entries below are endorsements. They are the categories of capability the readiness checklist in Part 3, Chapter 7 assumes exist somewhere in your stack, with named examples so a team starting from zero has a concrete place to begin evaluating rather than a blank page. Weigh each candidate against your own asset inventory (Part 3, 7.1) and severity matrix (Appendix B) rather than adopting a tool because a peer protocol uses it.

On-chain monitoring and alerting

Tool	What it does
Forta Network	Decentralized network of detection bots evaluating live mempool and block data against subscribable rules (Part 3, 5.2).
OpenZeppelin Defender	Contract-scoped monitoring (Monitor) paired with automated response actions (Sentinel).
Tenderly Alerts	Contract-scoped monitoring with a no-code condition builder.

Infrastructure and application monitoring

Tool	What it does
Prometheus	Open-source time-series metrics collection and alerting, used for RPC node health and infrastructure signal (Part 3, 5.4).
PagerDuty	Paging and escalation orchestration matched to the severity-to-channel routing in Appendix B.
Opsgenie	Alternative paging and escalation orchestration.

Forensics and chain analysis

Tool	What it does
Foundry (cast)	Local transaction, trace, and storage inspection, the basis for the evidence-collection commands in Part 3, 6.3.
Chainalysis	Commercial blockchain forensics and attribution platform.
TRM Labs	Commercial blockchain forensics and risk-scoring platform.
Arkham Intelligence	Public on-chain intelligence and entity-labeling platform.

Evidence and secrets storage

Tool	What it does
AWS S3 Object Lock	WORM-compliant object storage underlying the evidence bucket pattern in Part 3, 7.4.

Secure communications

Tool	What it does
Signal	End-to-end encrypted messaging, the reference “war room” channel in Part 3, 7.3.
Matrix	Federated, encryption-capable protocol for teams that need self-hosted control over their comms infrastructure.

Bug bounty and responsible disclosure

Tool	What it does
Immunefi	Web3-focused bug bounty and vulnerability disclosure platform (Part 3, 5.5).
HackenProof	Bug bounty platform.
Cantina	Competitive audit and bug bounty marketplace.

Threat intelligence and community response

Tool	What it does
Security Alliance (SEAL) / SEAL 911	Cross-protocol incident response coordination and 24/7 emergency hotline (Part 1, 2.6.3; Part 3, 7.3).
MITRE ATT&CK	Adversary tactics and techniques knowledge base for general enterprise threat modeling.
MITRE AADAPT	Adversary tactics and techniques knowledge base built specifically for digital asset and payment ecosystems.

Signer and fund-movement authorization

Tool	What it does
Safe{Wallet}	The most widely used multisig implementation for treasury and Containment Lead signer keys (Part 1, 3.7).
Flashbots Protect	Private mempool relay for rescue transactions, immune to public-mempool front-running (Part 3, 6.4).

22. Appendix D: Message and Communication Templates

Part 2, Section 4.3.1 established that every incident communication should start from a pre-approved template rather than an improvised draft written under pressure. The six templates below are that starting point. Fill in the bracketed fields, keep the structure, and route every one of them through the Comms Lead (Part 1, 3.4) before it goes anywhere near a public channel; a template used unreviewed is only marginally better than no template at all.

Template 1: Internal incident declaration (paging message)

```
[SEVERITY] Incident declared: <one-line description>
Declared by: <Triage Lead>
Time (UTC): <timestamp> | Block: <block number>
Affected: <contract(s) / system(s)>
War room: <link or channel>
Next update: <time>
```

Template 2: Public holding statement (first hour)

```
We are aware of a potential security incident affecting <protocol/component>.
We have <action taken, e.g. "paused the affected contract"> as a precaution.
No further action is required from users at this time. / Users should
<specific action> until further notice.
We will post a further update by <fixed time, UTC>.
```

Template 3: Scheduled status update (recurring cadence)

```
Status update <N>, <timestamp UTC>

What we know: <verified facts only, no speculation>
What we have done: <containment/eradication actions completed>
What is next: <planned next step>
Next update by: <fixed time, UTC>
```

Template 4: Resolution and all-clear statement

```
As of <timestamp UTC>, the incident affecting <protocol/component> is resolved.
Root cause: <one-sentence summary; full postmortem link below>
Funds affected: <amount/status, e.g. "fully recovered", or "partial loss
of $X, restitution plan below">
Restitution: <link, or "not applicable">
Full postmortem: <link, published within the window committed to in Part 5>
```

Template 5: GPG-signed authenticity notice

Use this whenever official channels may themselves be compromised, the exact scenario Part 1, Section 3.7 built the fingerprint-publication requirement for.

```

-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA512

This statement is authentic and was issued by <organization>.
Verify against fingerprint: <published GPG fingerprint>
Statement: <the exact text of the public statement being authenticated>
-----BEGIN PGP SIGNATURE-----
<signature block>
-----END PGP SIGNATURE-----

```

Template 6: Exchange or stablecoin issuer freeze request

Subject: Urgent, security incident, freeze request for address <0x...>

<Exchange/Issuer security team>,

We are <organization>, operator of <protocol>. At <timestamp UTC>, block <N>, an exploit moved approximately <amount, asset> to address <0x...>. Supporting evidence (transaction hash, trace, public incident statement) is attached or linked below.

We request an emergency freeze or enhanced monitoring on this address and any funds that move from it. Point of contact for follow-up: <name, verified channel, GPG fingerprint>.

Reference: <public incident statement URL>



Figure 2. Which template fires at which incident phase, mapped against the state diagram in Part 3, Figure 3. Template 5 (GPG authenticity) and Template 6 (freeze request) are not phase-bound: use Template 5 whenever a statement's authenticity might be questioned, and Template 6 the moment an attacker address is known, regardless of phase.

23. Appendix E: References and Further Reading

This handbook's complete, deduplicated source list lives in `references.md`, grouped into Standards and Frameworks, Incidents and Case Studies, Tools, and Further Reading, following the series convention (the series conventions). It is not reproduced here, so that the two documents never drift out of sync; treat `references.md` as the current bibliography and this section as a pointer to the five publications the handbook leans on most.

23.1 Standards

The lifecycle model in Part 1, the supplemental practices in Part 1, Section 2.6, and the evidence-handling discipline in Part 3 all trace back to a small set of NIST publications. Everything else this handbook cites, RFCs, incident post-mortems, and tool documentation, supports a specific claim in a specific chapter and is listed in full in `references.md` rather than here.

23.1.1 NIST SP 800-61r3, CSF 2.0, SP 800-92, SP 800-184, SP 800-150

Five NIST documents anchor this handbook's structure. [SP 800-61 Revision 3](#) (April 2025) supplies the core lifecycle, nesting incident response inside the [Cybersecurity Framework 2.0](#)'s six functions (Part 1, 2.1). [SP 800-92 Revision 1](#), an initial public draft dated October 2023, governs the log management lifecycle behind Part 1, Section 2.6.1 and Part 3, Section 7.2. [SP 800-184](#) (December 2016) grounds the recovery-rehearsal argument in Part 1, Section 2.6.2 and the drill schedule in Part 3, Section 7.6. [SP 800-150](#) (October 2016) frames the threat-information-sharing relationships, SEAL 911 among them, referenced in Part 1, Section 2.6.3. A sixth publication outside this named set, [SP 800-86](#), *Guide to Integrating Forensic Techniques into Incident Response*, underlies the evidence-collection ordering in Part 3, Section 6.3 and is listed alongside these five in [references.md](#).

Key controls for Part 6

- Keep the glossary, severity matrix, and tool list from this part inside your own runbook, not just this handbook, so nobody is searching for definitions mid-incident.
- Have the Triage Lead fill out the Appendix B worksheet live during triage; the fine-grained scoring table breaks ties, it never overrides the primary four-tier matrix.
- Select and pre-provision the Appendix C tool categories before an incident; evaluating a monitoring or forensics platform for the first time during a live drain is a preparation failure, not a response one.
- Route every Appendix D template through the Comms Lead and, where authenticity may be questioned, through the GPG signature workflow from Part 1, Section 3.7, before it reaches a public or partner channel.
- Treat [references.md](#) as the single current bibliography for this handbook; update it rather than letting citations diverge across parts.

Appendix C: References and Further Reading

This appendix collects every external source cited across the handbook's parts, grouped by category for easier lookup.

Standards and Frameworks

- NIST Special Publication (SP) 800-61 Revision 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management
- NIST SP 800-61r3 (full PDF)
- NIST SP 800-86, Guide to Integrating Forensic Techniques into Incident Response
- NIST SP 800-92 Revision 1 (Initial Public Draft), Guide to Computer Security Log Management
- NIST SP 800-150, Guide to Cyber Threat Information Sharing
- NIST SP 800-184, Guide for Cybersecurity Event Recovery
- NIST Cybersecurity Framework (CSF) 2.0
- FIRST CVSS v4.0 Specification
- MITRE ATT&CK
- MITRE AADAPT
- RFC 3227, Guidelines for Evidence Collection and Archiving
- RFC 4880, OpenPGP Message Format
- RFC 9116, A File Format to Aid in Security Vulnerability Disclosure (security.txt)
- OpenID Connect Core 1.0
- CISA CIRCIA (Cyber Incident Reporting for Critical Infrastructure Act)
- SEC Cybersecurity Risk Management, Strategy, Governance, and Incident Disclosure Rule
- FinCEN SAR Filing Guidance
- OWASP Smart Contract Weakness Enumeration (SCWE)
- OWASP Web3 Attack Vectors Top 15
- Security Alliance Safe Harbor

Incidents and Case Studies

- Ronin Bridge Hack of March 2022, Rekt.news
- Ronin Network, Wikipedia
- Halborn, "Explained: The Ronin Hack (March 2022)"

- [FBI Statement on Attribution of Malicious Cyber Activity Posed by the DPRK \(Ronin\), April 2022](#)
- [CISA Advisory AA21-048A](#)
- [FBI/CISA/State Department Advisory on DPRK IT Workers, 2024](#)
- [Poly Network Hack of August 2021, Rekt.news](#)
- [Poly Network Exploit, Wikipedia](#)
- [Nomad Bridge Hack of August 2, 2022, Rekt.news](#)
- [March 13, 2023 Euler Finance Hack, Rekt.news](#)
- [Euler Finance](#)
- [Vyper Security Advisory \(GHSA-g2q6-65hc-h6hh\)](#)
- [Uber Security Update](#)

Tools and Documentation

- [Foundry](#)
- [Cantina](#)
- [HackenProof](#)
- [Immunefi](#)
- [Matrix](#)
- [Matrix End-to-End Encryption Implementation Guide](#)
- [Signal](#)
- [Signal Protocol Documentation](#)
- [Prometheus](#)
- [Tenderly Alerts](#)
- [Forta Network](#)
- [OpenZeppelin Defender](#)
- [OpenZeppelin Contracts](#), [Pausable](#)
- [Flashbots Protect](#)
- [Safe{Wallet}](#)
- [AWS S3 Object Lock](#)
- [PagerDuty](#)
- [Opsgenie](#)
- [Arkham Intelligence](#)
- [Chainalysis](#)
- [TRM Labs](#)
- [IC3 \(FBI Internet Crime Complaint Center\)](#)
- [Sigma](#)
- [Arbitrum DAO Governance Documentation](#)
- [Security Alliance \(SEAL\) / SEAL 911](#)

- [Security Alliance \(SEAL\) / Telegram Emergency Bot](#)

Further Reading

- [Etsy, Code as Craft, “Blameless PostMortems and a Just Culture”](#)
- [PagerDuty, Postmortem Guidance](#)
- [Gremlin, “How to Run a GameDay”](#)
- [CISA Tabletop Exercise Package \(CTEP\)](#)